VILNIUS GEDIMINAS TECHNICAL UNIVERSITY
FACULTY OF FUNDAMENTAL SCIENCES
DEPARTMENT OF INFORMATION SYSTEMS

Practical Assignment No. 2
«Information Technology Security Methods»

Prepared by: Ulan Baigaskin, Ivann Aureus, Ravan Gajievi (ITSfmu-26)

VILNIUS 2026

1. Introduction

This work builds a small web application whose login rests on salted, slow password hashing with Argon2id, and proves with tests that the defences around it hold.

Item	Value
University	Vilnius Gediminas Technical University (VILNIUS TECH), Faculty of Fundamental Sciences, Department of Information Systems
Course	Information Technology Security Methods (ITSM), 2026–2027 fall semester
Practical work	No. 2: Web authentication security
Group	ITSfmu-26
Team members	Ulan Baigaskin, Ivann Aureus, Ravan Gajievi
Roles	Ulan Baigaskin (builder); Ivann Aureus (tester); Ravan Gajievi (threat model, checklists, report)
Topic	Hash (password + salt) with Argon2id
Repository	github.com/Baigaskin-Ulan/itsm-hash-auth

The goal is that a leaked user database does not reveal passwords. Each password is stored only as an Argon2id hash with its own random salt, so equal passwords give different records and every guess costs the attacker 64 MiB of memory.

The full solution adds backup codes, credential reset and centralised revocation. The report answers, from a threat model, whether these additions raise or lower security.

A second team member, who did not write the code, then attacked the finished application. His twenty penetration tests (PT-01 to PT-20) are summarised in section 11 and reproduced in full in Appendix A.

2. Task and assessment

The task is to implement a simple web application and secure its user authentication with one mechanism, here password hashing with a salt. Design can be minimal; the mechanism is the focus.

The full assignment adds three features on top of the working mechanism:

1. A backup authentication method (one-time backup codes).
2. Credential reset for users.
3. Centralised revocation or reset of user and group credentials by the infrastructure owner.

For each addition the report must state what it lets an attacker try and how that was blocked, and conclude whether security went up or down.

Part	Points
Mechanism works and is demonstrated live	4
Tests prove each defence; ASVS checklist is honest	2
Recovery and revocation designed against the threat model	2
Report is clear, sourced and reproducible	2
Base part only (working mechanism)	max 7
Complete solution (base + backup, reset, revocation)	max 10

Required deliverables:

- a one-page STRIDE threat model of the login and recovery flows (section 8);
- an OWASP ASVS 5.0 Level 1 checklist for V6 Authentication and V7 Session Management (section 9);
- a security test log showing each defence working (section 10);
- an independent penetration test of the finished application (section 11 and Appendix A);
- a Git repository whose history shows each team member's contribution (sections 4.4 and 4.5);
- a live demonstration and this report in PDF.

The lecture also asks for a one-sentence result from each in-class mini task (section 12).

3. Theory: salted, slow password hashing

Passwords must be stored with a one-way, deliberately slow and memory-hard function plus a unique salt. Fast hashes such as SHA-256 let an attacker test huge numbers of guesses offline after a database leak [2].

3.1 Hashing, not encryption

Encryption is two-way: whoever obtains the key recovers every password. A hash is one-way, so the server stores only the hash and, at login, hashes the typed password again and compares the results. A stolen hash cannot be used to log in directly.

3.2 Salt

A salt is a unique random value mixed into each password before hashing. Two users with the same password then get different hashes, so an attacker cannot see who shares a password. Precomputed lookup tables (rainbow tables) become useless, because each salt would need its own table.

The salt is not a secret. Argon2 libraries generate it automatically and store it inside the hash string (section 3.5).

3.3 Pepper

A pepper is a secret value shared by all passwords and kept outside the database, for example in an environment variable or a vault. It protects passwords when only the database leaks, such as through SQL injection or a stolen backup. If the pepper leaks, it must be replaced, which forces every user to reset their password. In this project the pepper is optional.

3.4 Why the function must be slow

The defender hashes one password per login; the attacker hashes billions of guesses. A cost that is negligible for one login multiplies across every guess. Argon2id adds a memory cost to the time cost, which limits how many guesses a GPU can run in parallel.

Function	Salt	Cost per guess	Limitation	OWASP recommendation
Argon2id	built in	memory + iterations	none relevant here	first choice: at least 19 MiB, 2 iterations, parallelism 1

Function	Salt	Cost per guess	Limitation	OWASP recommendation
bcrypt	built in	work factor	input cut at 72 bytes	legacy systems only; work factor at least 10
PBKDF2-HMAC-SHA256	separate	iterations only	no memory cost	when FIPS is required; at least 600,000 iterations
SHA-256	none	very fast, no memory	built for speed	not suitable for passwords

The 72-byte limit of bcrypt also conflicts with ASVS requirement V6.2.8, which demands that passwords be verified exactly as received, without truncation [4]. This is a second reason to prefer Argon2id.

3.5 Argon2id in this project

The project uses the `argon2-cffi` library. Its `PasswordHasher` defaults follow the second recommended option of RFC 9106, the low-memory profile [3], [6]. Hashing the password `test` produced:

```
$argon2id$v=19$m=65536,t=3,p=4$JWcqK90GhcMecSuX73imrw$VK0L2YR0x2GQ7dY2//6CoNvxa+lw0zLwXeLzLTgOY/Q
```

Field	Value	Meaning
`argon2id`	variant	hybrid Argon2, resistant to GPU and side-channel attacks
`v=19`	version	Argon2 version 1.3 (0x13)
`m=65536`	64 MiB	memory used per hash
`t=3`	3	passes over the memory
`p=4`	4	parallel lanes
`JWcqK90G…`	16 bytes	the salt, Base64-encoded
`VK0L2YR0…`	32 bytes	the hash itself

These parameters exceed the OWASP minimum (19 MiB, 2 iterations). Because the salt and parameters travel inside the string, the database needs one column per user. `verify()` reads them back from the string and recomputes the hash; `check_needs_rehash()` tells after a successful login whether the stored hash uses outdated parameters, so it can be upgraded while the plaintext is available.

3.6 Password policy

The policy follows NIST SP 800-63B-4 as summarised in the course lecture [1]:

- at least 15 characters when the password is the only factor, and at least 64 allowed;
- no composition rules (upper case, digits, symbols);
- no periodic forced change, only on evidence of compromise;
- a blocklist of breached, dictionary and context-specific words;
- paste and password managers allowed; no hints or security questions.

3.7 Client-side hashing and PAKE

A related question is whether the browser should hash the password in JavaScript before sending it, so the plaintext never leaves the client. It helps a little as defence in depth: if TLS is terminated at a proxy or the password lands in a log, only a hash is exposed. But it does not replace server-side hashing, for three reasons:

- Whatever the client sends is the credential. If the server stored that value as received, a leaked database would let an attacker replay it and log in. The server must still hash what it receives with Argon2id.
- It does not replace TLS: without TLS the client-side hash can simply be replayed from a capture.
- The client needs a salt to compute the hash, usually the username or a salt the server hands out first, which can leak which usernames exist.

The principled way to never send the password is a password-authenticated key exchange (PAKE) such as SRP or OPAQUE: the server never sees the password, even at registration, and a captured exchange cannot be replayed or brute-forced offline. This project keeps the standard model, server-side Argon2id with a salt over TLS, which is correct for the topic; client-side hashing is noted here as defence in depth, not a substitute.

4. Environment and reproducibility

The project runs on Windows with Python 3.15.2 in an isolated virtual environment; anyone can rebuild it from the repository with the commands below.

Component	Version / value	Purpose
Python	3.15.2	runtime
Flask	3.1.3	web framework, routing, sessions
argon2-cffi	25.1.0	Argon2id password hashing
Flask-Limiter	4.1.1	login rate limiting
Flask-WTF	1.3.0	CSRF protection for forms
Git	2.56.0	version control
Editor	Visual Studio Code + Python extension	development
Database viewer	DB Browser for SQLite	inspecting stored hashes

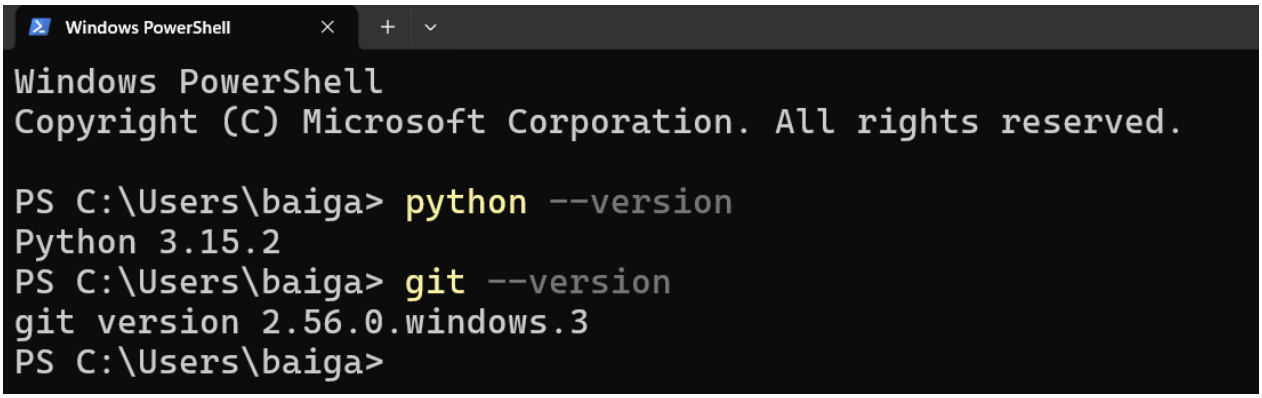


Figure 1. Python 3.15.2 and Git 2.56.0 installed on the Windows host.

The virtual environment `.venv` keeps the project's libraries apart from the system Python. It is excluded from Git by `.gitignore`, together with `__pycache__/`, the SQLite database, the `.env` secrets file and `auth.log`. Exact library versions are pinned in `requirements.txt`.

4.1 Setup commands

```
python --version
git --version
python -m venv .venv
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
.\.venv\Scripts\Activate.ps1
pip install flask argon2-cffi flask-limiter flask-wtf
pip freeze | Set-Content -Encoding ascii requirements.txt
python check.py
git init
git add .
git commit -m "Project setup: venv, requirements, argon2 check"
git remote add origin https://github.com/Baigaskin-Ulan/itsm-hash-auth.git
git push -u origin main
```

`Set-ExecutionPolicy` allows the local activation script to run; Windows blocks `.ps1` scripts by default (Figure 2). `requirements.txt` is written with `Set-Content -Encoding ascii` because the PowerShell > redirect saves UTF-16, which Git treats as a binary file.

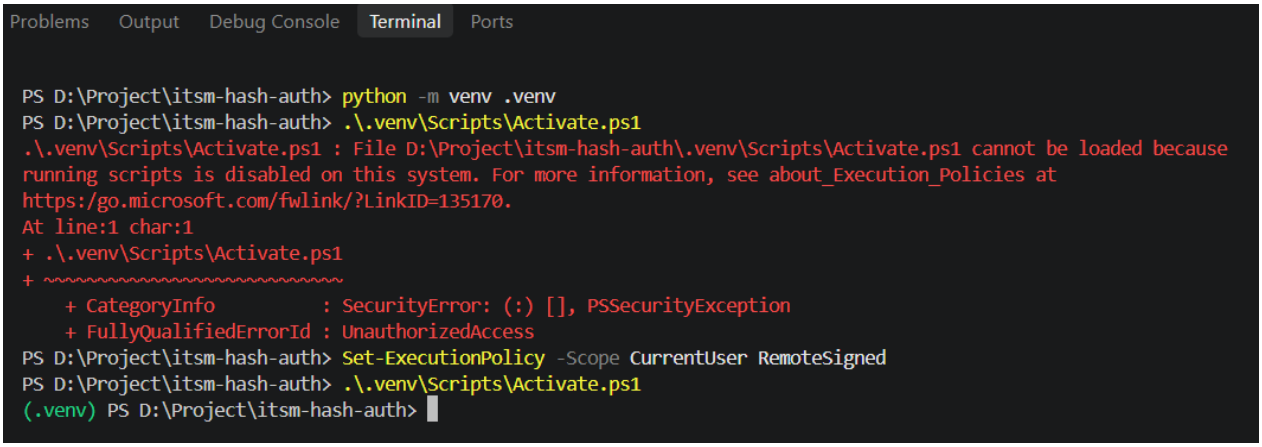


Figure 2. Creating the virtual environment: the first activation is blocked by the execution policy; after Set-ExecutionPolicy the prompt shows (.venv).

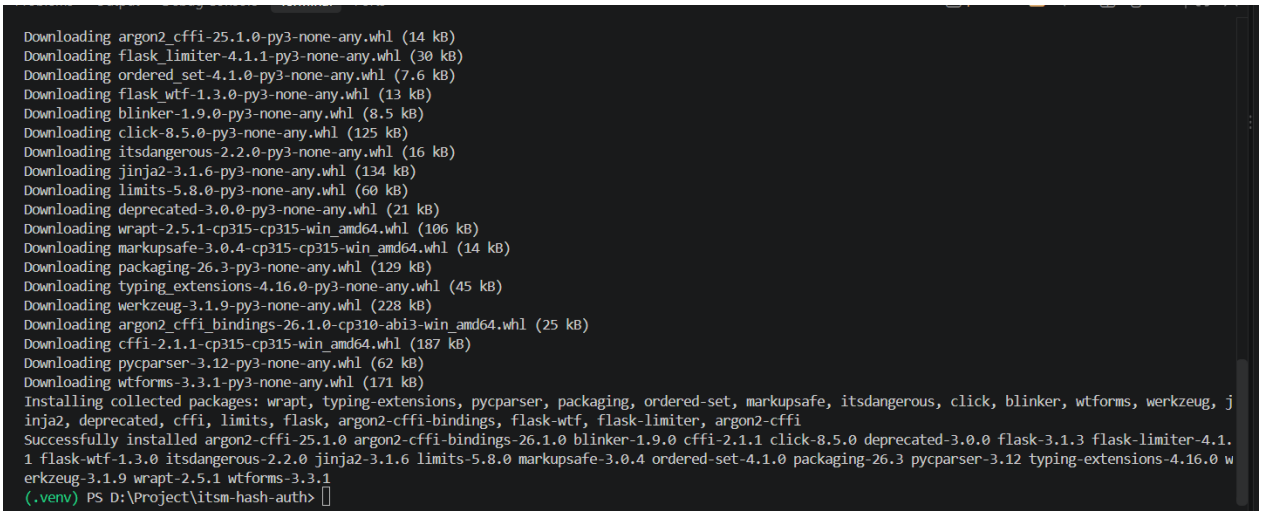


Figure 3. Flask, argon2-cffi, Flask-Limiter and Flask-WTF installed into the virtual environment.

4.2 Rebuilding from the repository

```
git clone https://github.com/Baigaskin-Ulan/itsm-hash-auth.git
cd itsm-hash-auth
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python init_db.py
python app.py
```

4.3 First check of the mechanism

`check.py` hashes the same password `test` on every run (Figure 4). Each run gives a different string, because a new 16-byte salt is generated each time; the parameters are explained in section 3.5.

```
from argon2 import PasswordHasher
ph = PasswordHasher()
print(ph.hash("test"))
```

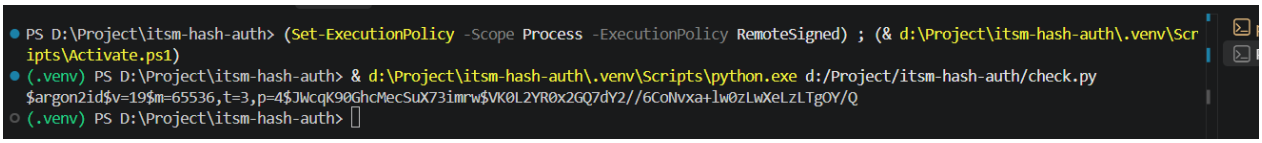


Figure 4. check.py prints an Argon2id hash of the password "test": parameters m=65536, t=3, p=4 and a random salt.

4.4 Repository and commit history

The code is published on GitHub; Git Credential Manager signed in through the browser (Figure 5) and the first push created branch `main` (Figure 6). Every step of the work is a separate commit, which is the builder's contribution history:

Commit	Message
`f2826f5`	Project setup: venv, requirements, argon2 check
`7233d11`	Add database schema: users and server-side sessions
`15e7fc3`	Add registration, login and logout with Argon2id and sessions
`3bf4b6b`	Add login rate limiting per IP and per username
`72c91a7`	Block common and username-based passwords at registration
`cb90ac2`	Add authentication event logging without secrets
`ffc96c2`	Add crack.py demo: SHA-256 (no salt) vs Argon2id
`314423e`	Add security headers, secure session cookie, request size limit
`aaa102f`	Add password change requiring the current password
`8c650a4`	Add password reset by single-use, time-limited token
`e6ba692`	Add one-time backup codes stored as Argon2id hashes
(latest)	Add admin panel: revoke, force reset, disable by user or group

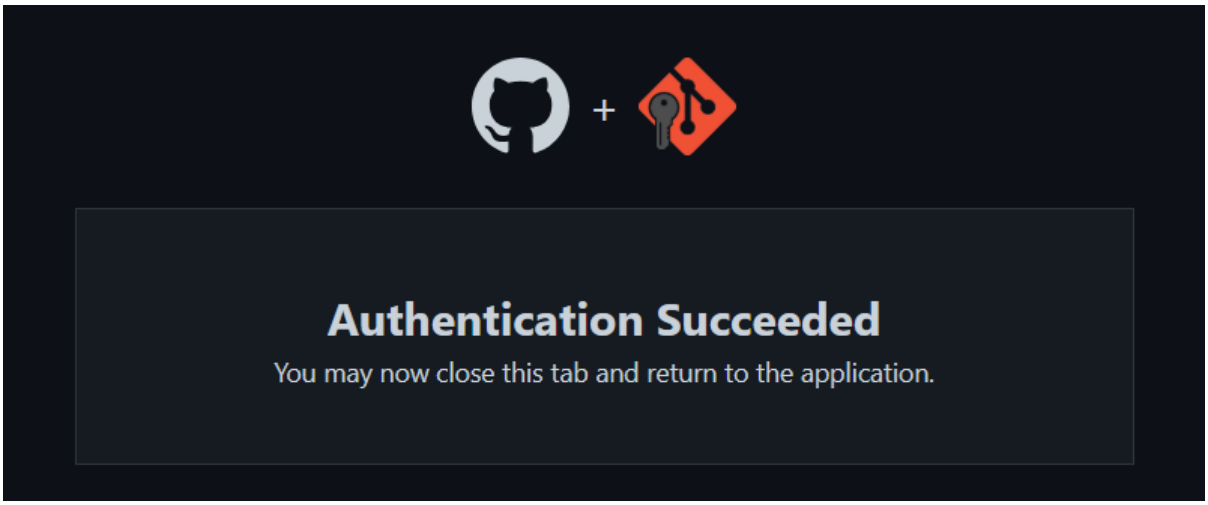


Figure 5. Git Credential Manager: authentication to GitHub succeeded.

```
(.venv) PS D:\Project\itsm-hash-auth> git remote add origin https://github.com/Baigaskin-Ulan/itsm-hash-auth.git
(.venv) PS D:\Project\itsm-hash-auth> git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (8/8), 1.13 KiB | 1.13 MiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/Baigaskin-Ulan/itsm-hash-auth.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
(.venv) PS D:\Project\itsm-hash-auth>
```

Figure 6. The local repository linked to GitHub and the first push of branch main.

4.5 Team roles and contributions

The lecture suggests three roles for a team of three: builder, breaker and writer. The team kept to this split, so the person who attacked the application is not the person who wrote it.

Member	Role	Contribution	In this report
Ulan Baigaskin	Builder	Flask application, Argon2id mechanism, sessions, rate limiting, backup codes, credential reset, admin panel; internal test log T1–T22	sections 4–7 and 10; commits in section 4.4
Ivann Aureus	Breaker	independent penetration test of the finished application: 20 test cases on 3 November 2026, four test reports with evidence	section 11, Appendix A
Ravan Gajievi	Writer	STRIDE threat model, ASVS checklist, mapping of all test evidence to threats, mini-task results, final report	sections 8, 9, 11–14

5. Architecture and base implementation

The application is one Flask file, a set of Jinja templates and an SQLite database. Passwords exist only as Argon2id strings and sessions only as SHA-256 hashes of random tokens.

File	Role
`app.py`	routes, Argon2id hashing, server-side sessions, CSRF, rate limiting, admin panel
`schema.sql`, `init_db.py`	database schema and its creation
`blocklist.txt`, `make_blocklist.py`	common-password blocklist and the script that builds it
`crack.py`, `make_admin.py`	salt-vs-slow-hashing demo; grant the admin role
`templates/*.html`	pages, each form with a hidden CSRF token
`app.db`	SQLite database; excluded from Git

5.1 Database schema

Table	Purpose
`users`	username, email, Argon2id `password_hash`, role, group, is_active, must_reset
`sessions`	SHA-256 of the session token, owner, created and last-seen times
`password_resets`	SHA-256 of the reset token, owner, expiry
`backup_codes`	Argon2id hash of each one-time backup code
`audit_actions`	actor, action, target and time of every admin action

```sql
CREATE TABLE IF NOT EXISTS users (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    username      TEXT    NOT NULL UNIQUE,
    email         TEXT    NOT NULL UNIQUE,
    password_hash TEXT    NOT NULL,
    role          TEXT    NOT NULL DEFAULT 'user',
    user_group    TEXT    NOT NULL DEFAULT 'students',
    is_active     INTEGER NOT NULL DEFAULT 1,
    must_reset    INTEGER NOT NULL DEFAULT 0,
    created_at    TEXT    NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS sessions (
    token_hash  TEXT    PRIMARY KEY,
    user_id     INTEGER NOT NULL REFERENCES users(id),
    created_at  REAL    NOT NULL,
    last_seen   REAL    NOT NULL
);
```

Figure 7. schema.sql: the users table and the server-side sessions table.

```python
import sqlite3

con = sqlite3.connect("app.db")
with open("schema.sql", encoding="utf-8") as f:
    con.executescript(f.read())
con.close()
print("Database created: app.db")
```

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\Project\itsm-hash-auth> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\Project\itsm-hash-auth\.venv\Scripts\Activate.ps1)
(.venv) PS D:\Project\itsm-hash-auth> & d:\Project\itsm-hash-auth\.venv\Scripts\python.exe d:/Project/itsm-hash-auth/init_db.py
Database created: app.db
(.venv) PS D:\Project\itsm-hash-auth>
```

Figure 8. init_db.py creates app.db from schema.sql (top: the script; bottom: its output).

Every SQL statement passes user input as `?` parameters, never by string concatenation, so SQL injection is not possible.

5.2 Registration

1. The server reads the username, email and password from the form.
2. It checks that the password is 15 to 128 characters long. The `minlength` attribute in the form is only a convenience; the server check is the control.
3. It rejects passwords from the blocklist and passwords that contain the username (section 6.2).
4. `ph.hash(password)` generates a fresh 16-byte salt and computes the Argon2id hash.
5. The row is inserted. A duplicate username or email triggers the `UNIQUE` constraint, and the user sees a neutral message.

5.3 Login

Login always costs one Argon2id verification, whether the user exists or not (Figure 9). For an unknown username the server verifies the password against `DUMMY_HASH`, a hash computed once at start-up. Both failure cases return the same text, *Invalid username or password*, so neither the message nor the response time reveals which usernames exist.

After a successful match the server checks `is_active`, upgrades the stored hash if `check_needs_rehash()` reports outdated parameters, deletes any session the browser already had, and issues a new token. Issuing a new token at login defeats session fixation.

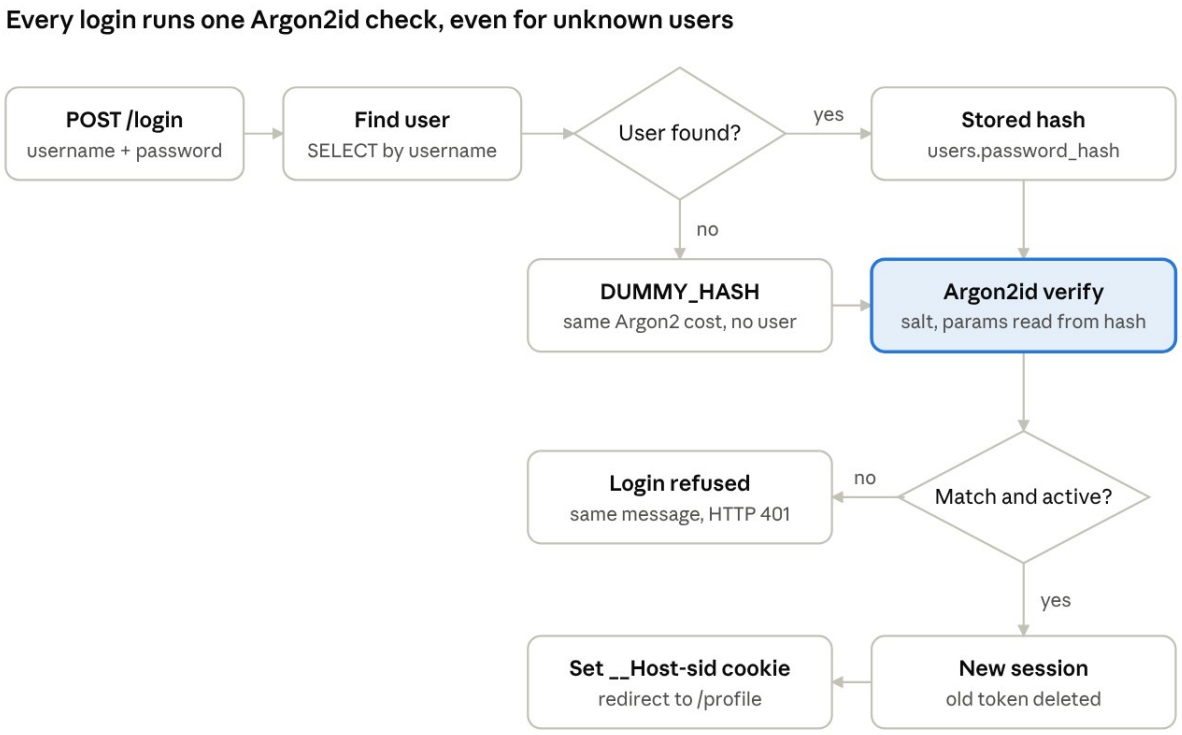


Figure 9. Login flow: an unknown user takes the DUMMY_HASH branch, so both failure paths cost the same time and end in the same message.

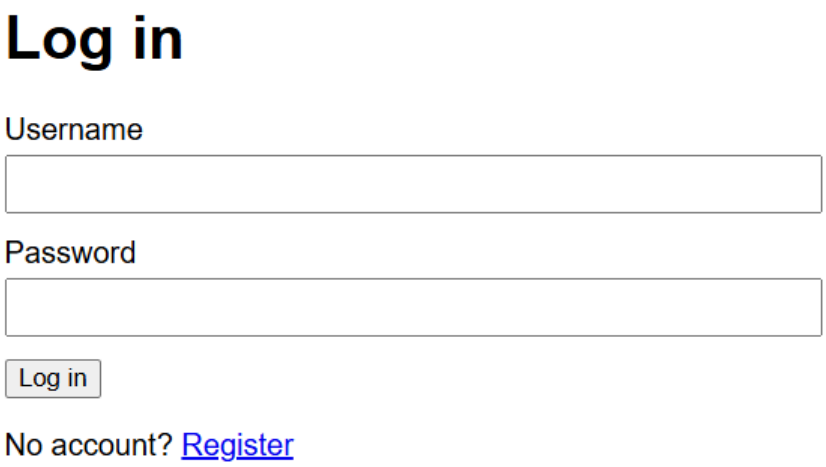


Figure 10. The login page at http://localhost:5000/login.

5.4 Sessions and logout

- The token is `secrets.token_urlsafe(32)`: 256 random bits, sent to the browser in the cookie `__Host-sid` with `Secure`, `HttpOnly`, `SameSite=Lax` and `Path=/`.
- The database stores only the token's SHA-256. A fast hash is safe here because 256 random bits cannot be guessed, unlike a human password; a leaked `sessions` table gives no usable cookie.
- Before every request `load_user()` finds the session, ends it after 30 minutes idle or 8 hours in total, or when the account is disabled, and otherwise updates `last_seen`.
- Logout is a POST with a CSRF token. It deletes the session row on the server, so a copied cookie stops working at once.

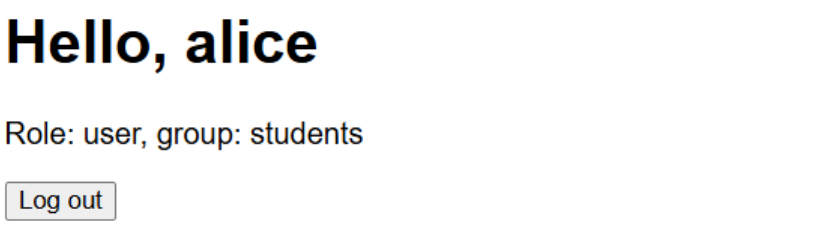


Figure 11. Profile page after a successful login as alice.

6. Security controls around the mechanism

Argon2id protects passwords after a database leak; further controls limit guessing against the live login form and harden the rest of the flow.

6.1 Login rate limiting

Flask-Limiter counts POST requests to `/login` in memory and applies two limits:

Limit	Key	Stops
5 per minute	client IP address	fast guessing from one machine
10 per hour	submitted username	slow, distributed guessing against one account from many IPs

A failed login answers HTTP 401; a request over either limit answers HTTP 429 with *Too many login attempts. Try again later.* (Figure 23). Opening the login page (GET) is not counted.

The limit also bounds a denial-of-service risk created by the mechanism itself: every attempt costs 64 MiB of memory for Argon2id, so an unthrottled login form could exhaust the server.

The per-username limit has a cost: anyone who knows a username can block that user's login for up to an hour with ten wrong attempts. The block is temporary rather than a permanent lockout, and this trade-off is analysed in the threat model (section 8).

6.2 Blocklist of common passwords

ASVS V6.2.4 (Level 1) requires new passwords to be checked against at least the 3,000 most common passwords that satisfy the policy [4]. `make_blocklist.py` downloads the SecLists list of the one million most common leaked passwords [7] and keeps those of 15 characters or more: 10,908 entries, saved as `blocklist.txt` (Figure 12).

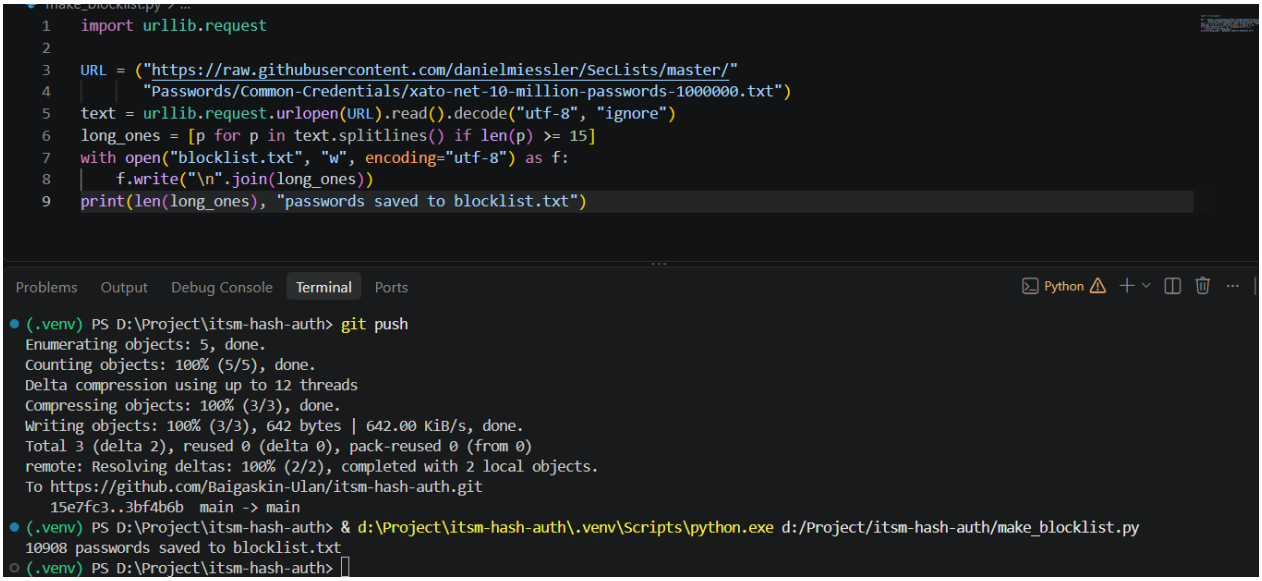


Figure 12. make_blocklist.py saves 10,908 common passwords of at least 15 characters to blocklist.txt.

The application loads the list once at start-up into a set and compares the new password case-insensitively, so `QAZWSXEDCRFVTGB` is rejected like `qazwsxedcrfvtgb`. Registration also refuses a password that contains the username, a context-specific word in the sense of V6.2.11 (Level 2).

6.3 Logging, headers and password change

Three more controls complete the base part.

- Authentication logging. Logins, failures, logouts, rate-limit hits, password changes and admin actions are written to `auth.log` with the user and IP, never the password. The event and username are logged with `%r`, so a username containing a newline cannot forge a second log line (log injection). This covers OWASP A09.

- Security response headers, set on every response: a `Content-Security-Policy` that allows scripts only from the site and forbids framing, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: same-origin` and `Cache-Control: no-store`. The Flask session cookie also carries `Secure`, and a 16 KB request-size limit bounds the memory a flood of login attempts can cost.
- Password change (`/change-password`) requires the current password, applies the same policy to the new one, and signs out every session, so a stolen session cannot quietly take over the account. This satisfies ASVS V6.2.2 and V6.2.3.

7. Backup authentication, credential reset and revocation

The full solution adds three features on top of the working mechanism. Each one is a new way into the account, so each is built to be as controlled as the password itself.

7.1 Backup codes

A logged-in user can generate ten one-time backup codes, shown only once (Figure 13). The codes are the chosen backup authentication method: a way to sign in when the password is unavailable (Figure 14). They fit the topic because a code is, in effect, a second password and is stored the same way, as an Argon2id hash with a salt.

- Codes are random, `secrets.token_hex(5)` grouped as `abcde-fghij`.
- Only the Argon2id hash of each code is stored, in the `backup_codes` table; the plaintext is shown once and never saved.
- A code works once: it is deleted on use. Generating new codes replaces any old ones.
- Sign-in with a code is on a separate page and is rate-limited like the password login.

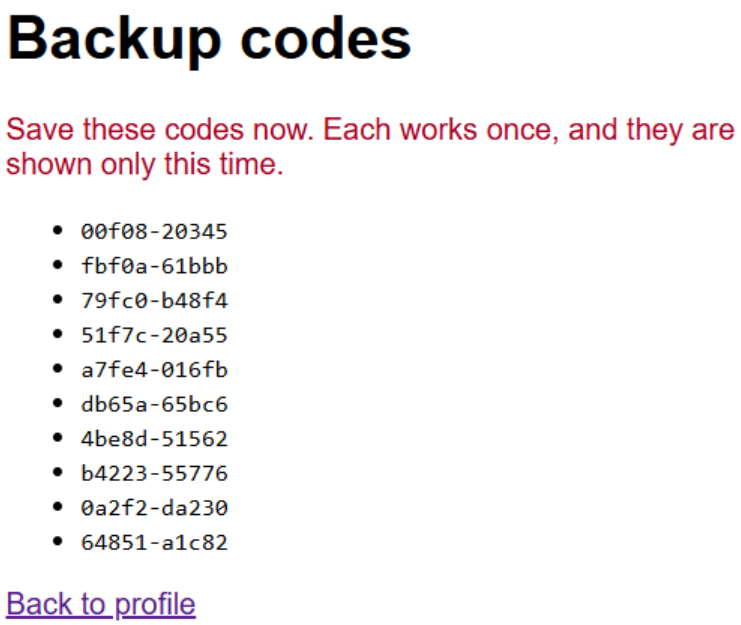


Figure 13. Ten one-time backup codes, shown only once; only their Argon2id hashes are stored.

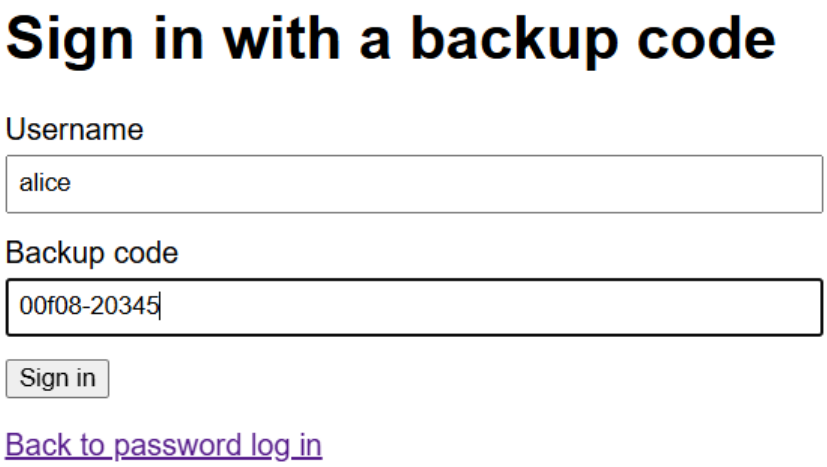


Figure 14. Signing in with a username and one backup code.

7.2 Credential reset

A user who has forgotten the password requests a reset by email (Figure 15).

- The reset token is `secrets.token_urlsafe(32)`, 256 random bits. Only its SHA-256 is stored, in `password_resets`, with a 30-minute expiry. A fast hash is enough here because the token is already random, unlike a human password.
- The response is the same whether or not the email has an account, so the reset flow cannot be used to discover which emails are registered.
- The lab has no mail server, so the reset link is printed to the server console, as the course suggests (Figure 16).
- The token is single-use and deleted after the reset. After a successful reset the new password is set, all of the user's sessions are deleted, and the forced-reset flag is cleared.

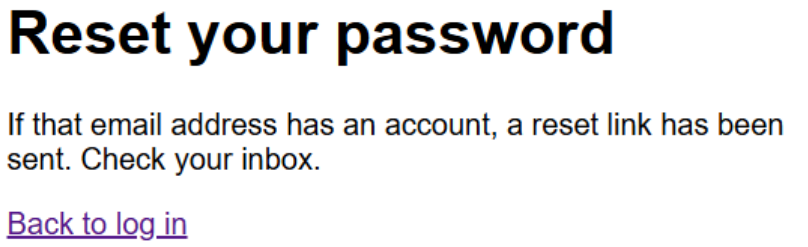


Figure 15. The forgot-password page returns the same message for any email.

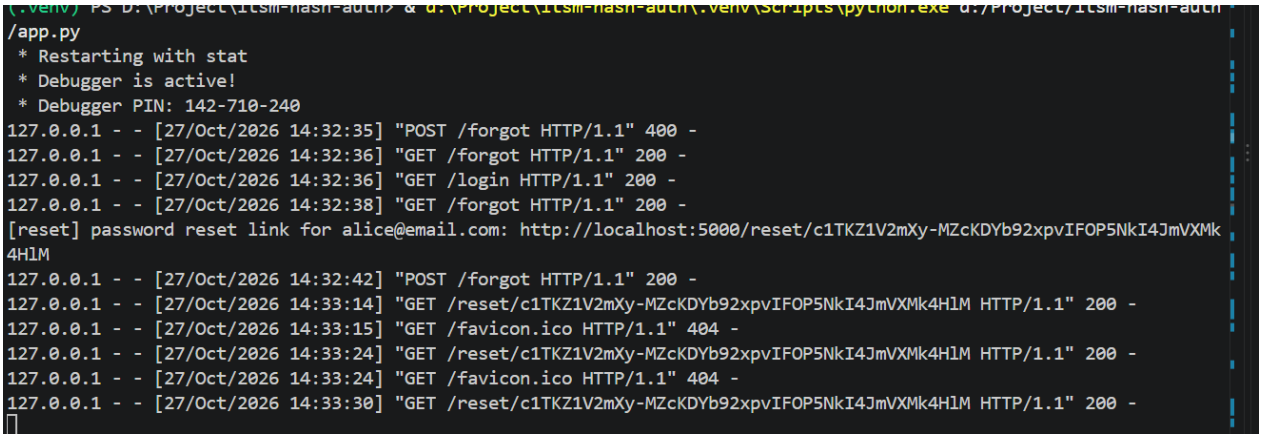


Figure 16. With no mail server, the reset link is printed to the server console.

7.3 Centralised revocation by the owner

An administrator (role `admin`, set once in the database with `make_admin.py`) has a panel that acts on one user or a whole group (Figure 17):

Action	Effect
Revoke sessions	Deletes all sessions, so the target is signed out everywhere at once
Force password reset	Sets a flag, deletes sessions and backup codes; the user cannot log in until they reset via the email flow (Figure 18)
Disable account	Blocks login and deletes sessions; a disabled account's open sessions also end on their next request
Enable account	Re-enables a disabled account

- Every action requires the administrator to re-enter their own password, so a stolen admin session cannot act on its own.
- Every action is written to an `audit_actions` table (actor, action, target, time) and to the log file.
- Group scope is the live demonstration: the administrator revokes the sessions of a group, and an open session in that group ends on the next request.

One limitation: an administrator who revokes their own group is signed out too, because by default every account is in the same group. In the demo the administrator is moved to a separate group.

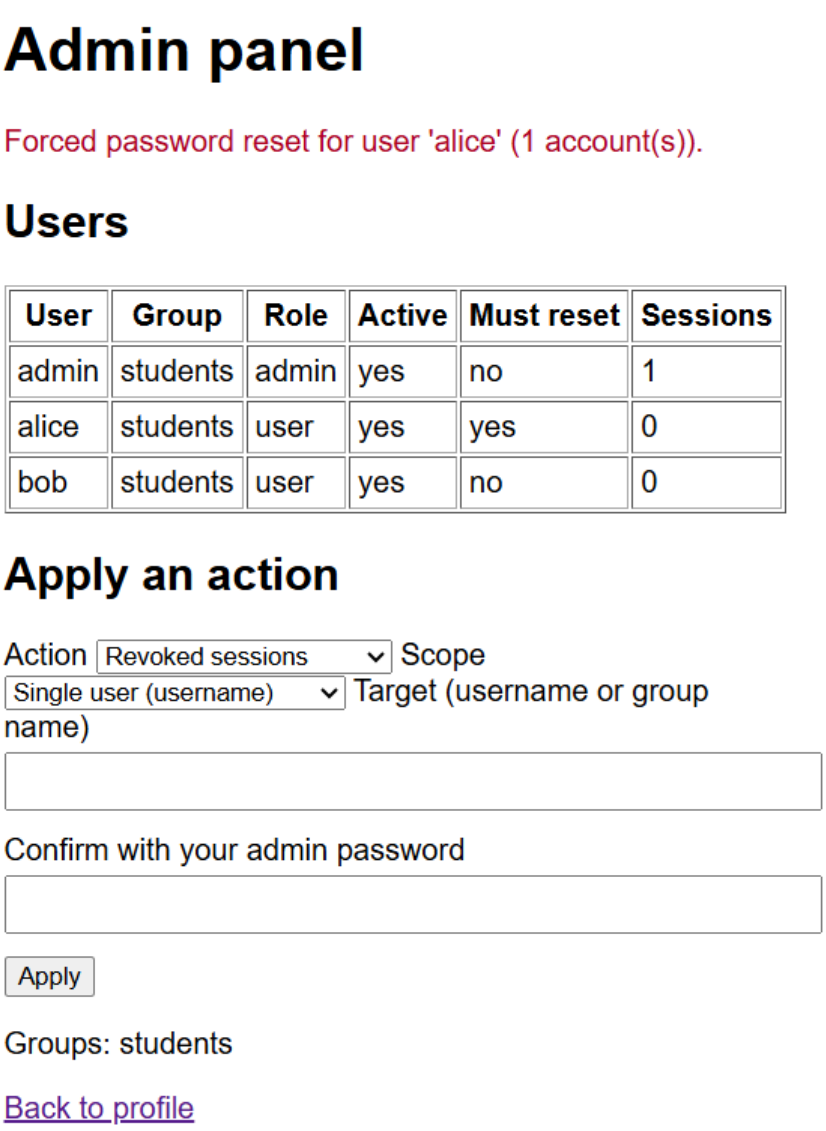


Figure 17. The admin panel: user table and the action form, confirmed with the admin password.

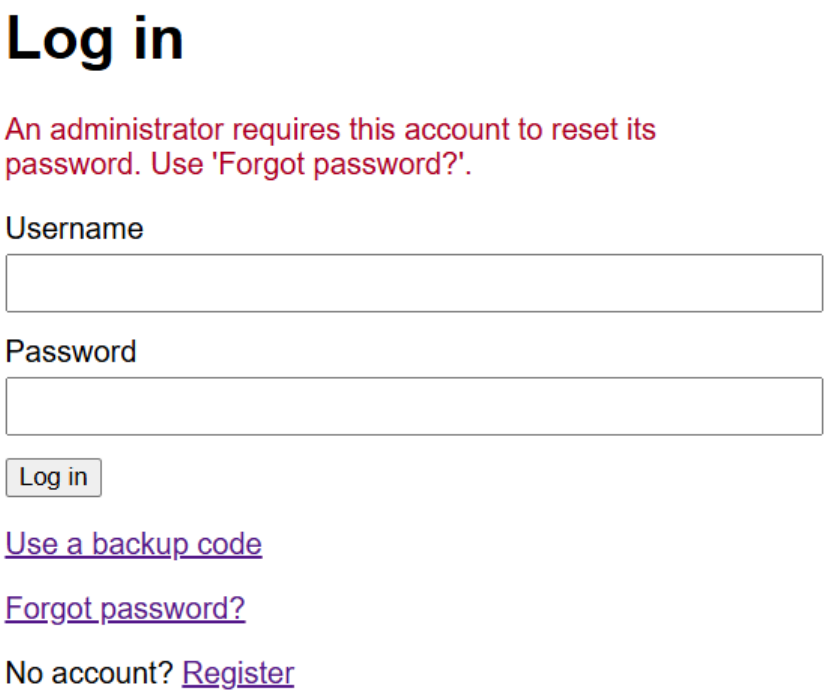


Figure 18. After a forced reset, the user cannot log in until they reset the password.

8. STRIDE threat model

STRIDE is a checklist of six threat types. Applied to the login and recovery flows, it shows what an attacker can try at each step and how the application blocks it. The data-flow diagram below marks the two trust boundaries the data crosses: the network between the browser and the server, and the boundary between the application and its database.

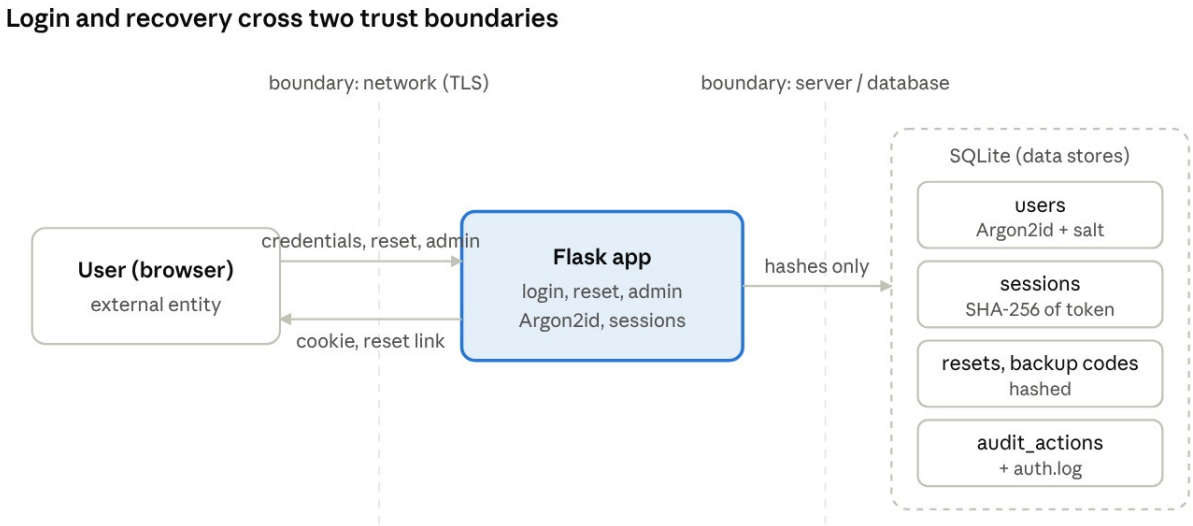


Figure 19. Data-flow diagram of login and recovery, with the two trust boundaries.

The table takes each STRIDE category in turn. The last column points to the tests that show the defence working: T-numbers are the builder's test log (section 10), PT-numbers the independent penetration test (section 11).

STRIDE threat	What the attacker tries here	Defence in this project	Test
Spoofing, pose as another user	Guess passwords; reuse breached passwords; guess which usernames exist	Argon2id slows guessing; rate limiting per IP and per account; one generic error; a dummy hash so timing does not leak usernames	T1, T5, T6, T8, T9, PT-02, PT-03, PT-04, PT-09, PT-13
Tampering, alter data in transit or at rest	Forge or edit a session cookie; submit a form from another site	Server-side session keyed by a random token; cookie flags Secure, HttpOnly, SameSite, __Host-; CSRF token on every form	T13, T14, PT-10, PT-11, PT-18
Repudiation, deny an action	An admin revokes accounts and later denies it; a login cannot be traced	auth.log records logins, resets, changes and failures; audit_actions records every admin action with actor and time	T15, PT-15
Information disclosure, leak secrets	Steal the database and read passwords, tokens or codes	Passwords and backup codes are Argon2id hashes with salts; session and reset tokens are stored only as SHA-256; shown by crack.py	T1, T17, PT-19, PT-20
Denial of service, make the service unavailable	Flood the login form; Argon2id memory cost could exhaust the server; lock a real user out	Rate limiting bounds attempts and memory; a 16 KB request cap; the per-account limit is temporary throttling, not a lockout	T8, T9, PT-08
Elevation of privilege, gain rights you should not have	A normal user opens the admin panel; a stolen admin session acts on its own	Admin panel checks the admin role (403 otherwise); every admin action re-asks for the admin password	T22, PT-05, PT-14, PT-16, PT-17

Two residual risks remain, and we accept them. The per-account rate limit lets someone throttle a known user's login for up to an hour (a denial-of-service trade-off, chosen over a permanent lockout). And an administrator who revokes their own group is signed out too, since accounts share a group by default.

The penetration test found a third one: an admin action was saved twice when the browser sent the admin form again (finding F1, section 11.2). We only noticed it because every admin action goes into the audit table.

9. OWASP ASVS 5.0 Level 1 checklist

This checklist covers the Level 1 requirements of chapter V6 (Authentication) and chapter V7 (Session Management) that apply to a password-based login. Password storage sits in chapter V11.4 (Cryptography) rather than V6; because it is the core of this topic, V11.4.2 is included even though it is a Level 2 requirement. Status is reported honestly: Met, Partly, Not met, or N/A with a reason.

9.1 V6 Authentication

ID	Requirement	Level	Status	Evidence / note
V6.2.1	Minimum password length (>=8, 15+ recommended)	L1	Met	15–128 enforced server-side
V6.2.2	User can change their password	L1	Met	/change-password

ID	Requirement	Level	Status	Evidence / note
V6.2.3	Change needs current and new password	L1	Met	current password verified
V6.2.4	Checked against >=3000 common passwords	L1	Met	blocklist.txt, 10,908 entries
V6.2.5	Any character composition allowed	L1	Met	no composition rules
V6.2.6	Password field masked	L1	Partly	type=password; no show-password toggle
V6.2.7	Paste and password managers allowed	L1	Met	autocomplete set, no paste block
V6.2.8	Verified as received, no truncation	L1	Met	Argon2id; the reason bcrypt (72-byte cap) was not used
V6.2.12	Checked against breached passwords (HIBP)	L2	Not met	out of scope; noted as a Level 2 improvement
V6.3.1	Controls against brute force / stuffing	L1	Met	rate limiting per IP and per account; PT-08
V6.3.2	No default or built-in accounts	L1	Met	none; admin is created explicitly
V6.3.3	Multi-factor authentication	L2	N/A	topic is password + salt; MFA is a separate topic
V11.4.2	Passwords stored with an approved KDF	L2	Met	Argon2id, 64 MiB / t=3 / p=4 (core of this work)

9.2 V7 Session Management

ID	Requirement	Level	Status	Evidence / note
V7.2.1	Token verified by a trusted backend	L1	Met	server-side sessions table
V7.2.2	Dynamic tokens, not static secrets	L1	Met	new token_urlsafe(32) per login
V7.2.3	Unique, CSPRNG, >=128 bits of entropy	L1	Met	secrets.token_urlsafe(32) = 256 bits
V7.2.4	New token at login; old one terminated	L1	Met	rotation at login; old session deleted
V7.4.1	Session unusable after logout or expiry	L1	Met	deleted server-side; idle and absolute timeouts; PT-06, PT-12
V7.4.2	Disabling an account ends its sessions	L1	Met	admin disable deletes sessions; inactive dropped on next request; PT-16
V7.3.1	Inactivity timeout	L2	Met	30-minute idle timeout (exceeds L1)
V7.3.2	Absolute session lifetime	L2	Met	8-hour absolute timeout (exceeds L1)
V7.4.5	Admin can end sessions for one or all users	L2	Met	admin panel, by user or group (exceeds L1); PT-17

The gaps are small. V6.2.6 is marked Partly: the field is masked with type=password, but there is no show-password toggle, which the standard allows as a usability option. V6.2.12, checking new passwords against a breached-password service such as Have I Been Pwned, is a Level 2 requirement and is not implemented. Several Level 2 controls are already met; they are noted rather than claimed as Level 1. Where the independent penetration test confirmed a requirement, its PT-number is added to the evidence.

10. Security test log

Each defence is shown working by a test run against the local application at `http://localhost:5000`; evidence is a screenshot or command output. The first table covers the base mechanism, the second the full-solution features. These tests were run by the builder; section 11 repeats and extends them independently.

10.1 Base-mechanism tests

ID	Defence	Steps	Expected	Result	Evidence
T1	Unique salt per password	Register alice and bob with the same password; open users in DB Browser	Different password_hash values	Pass	Figure 20
T2	Password length, browser	Enter an 8-character password in the form	Browser refuses to submit (minlength=15)	Pass	Figure 21
T3	Password length, server	Remove minlength in DevTools, submit 8 characters	Server answers: Password must be 15 to 128 characters long	Pass	Figure 26
T4	Correct login	alice with her password	Profile page: Hello, alice	Pass	Figure 11
T5	Generic error, wrong password	alice with a wrong password	Invalid username or password, HTTP 401	Pass	Figure 22
T6	Generic error, unknown user	nobody with any password	Same message as T5	Pass	Figure 27
T7	Server-side logout	Log out, then replay the old __Host-sid cookie	Redirect to /login	Pass	Figure 28
T8	Rate limit per IP	Six wrong logins within one minute	Attempts 1–5: 401; attempt 6: 429	Pass	Figure 23
T9	Rate limit per username	Eleven wrong logins for one name within an hour	Attempt 11: 429	Pass	Figure 30
T10	Common password blocked	Register with 1qaz2wsx3edc4rfv	This password is too common	Pass	Figure 24
T11	Username in password blocked	Register with a password that contains the username	Password must not contain the username	Pass	Figure 25
T12	Session rotation at login	Note __Host-sid before and after login	Value changes	Pass	Figure 29
T13	Cookie flags	DevTools -> Network, Set-Cookie on login	Secure, HttpOnly, SameSite=Lax on __Host-sid	Pass	Figure 32
T14	CSRF protection	POST to /login without a token	HTTP 400	Pass	Figure 31

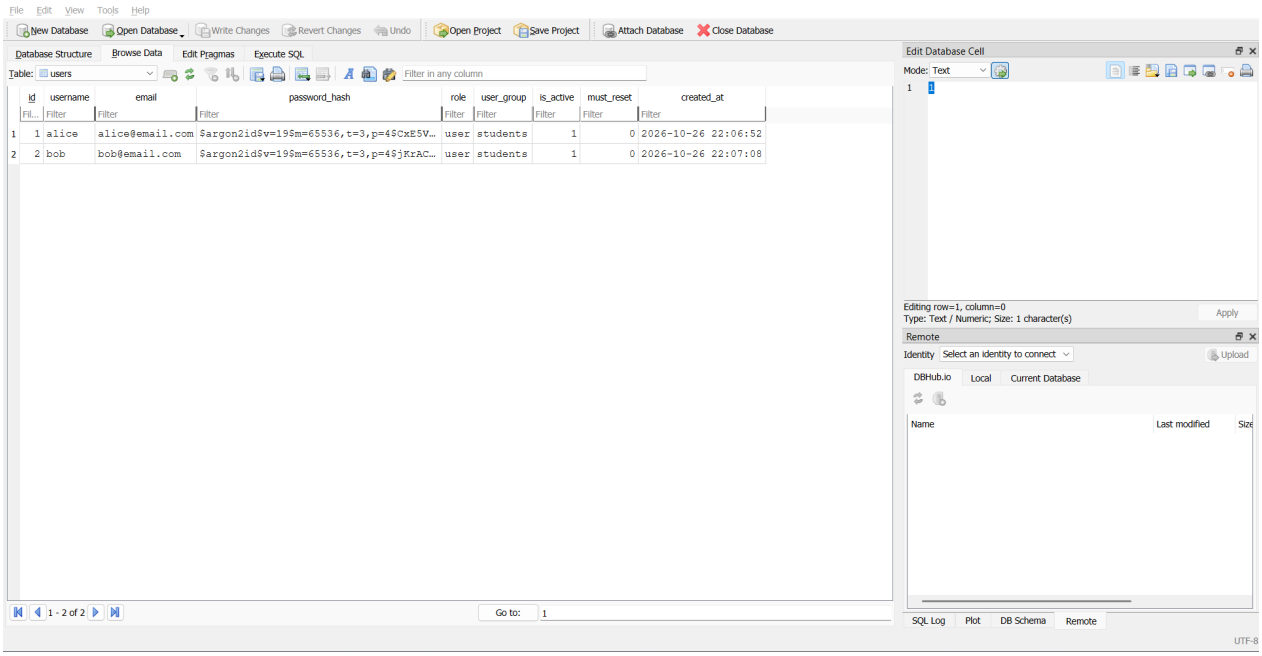


Figure 20. T1: alice and bob share a password but have different salts and hashes in the users table.

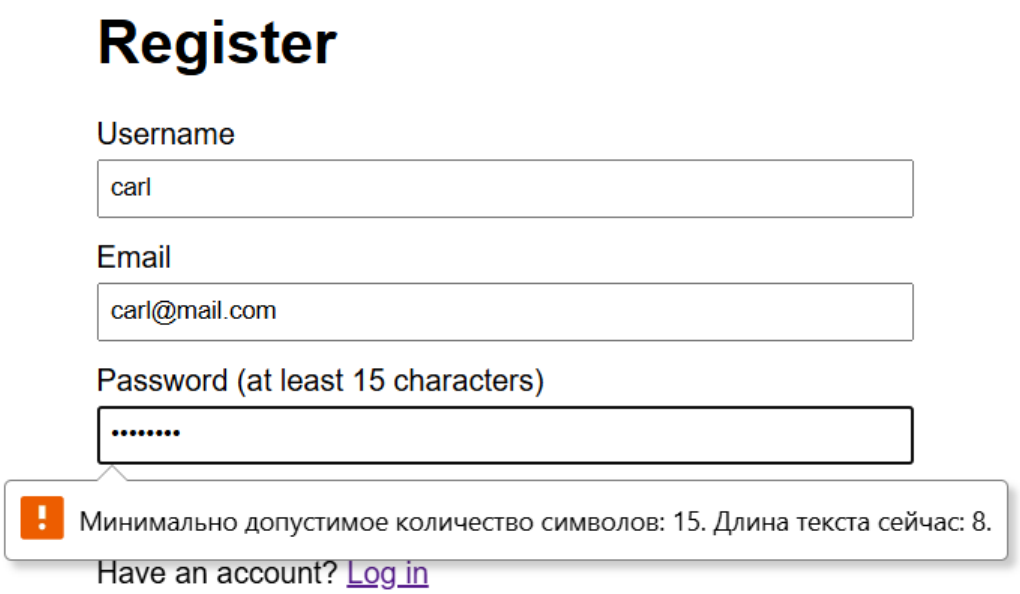


Figure 21. T2: the browser refuses an 8-character password because of minlength=15.

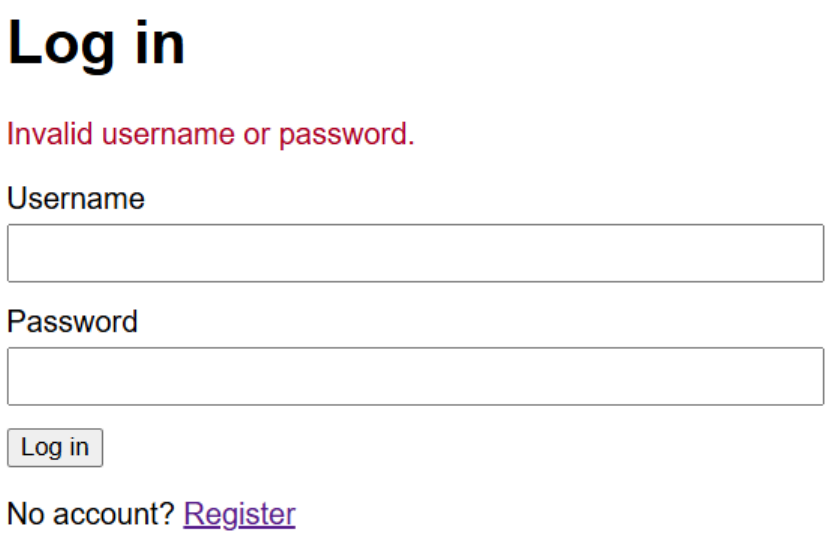


Figure 22. T5: a wrong password returns the generic message.

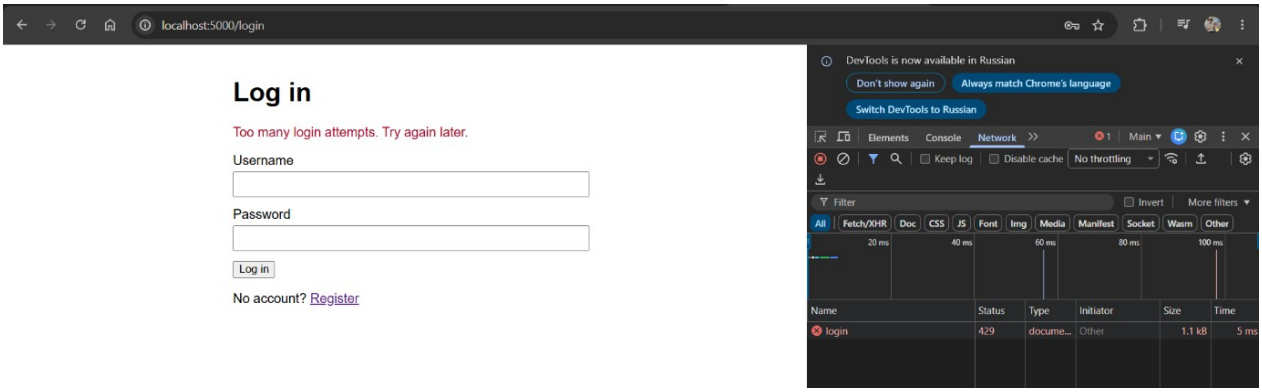


Figure 23. T8: the sixth wrong login within a minute is answered with HTTP 429 (DevTools, Network tab).

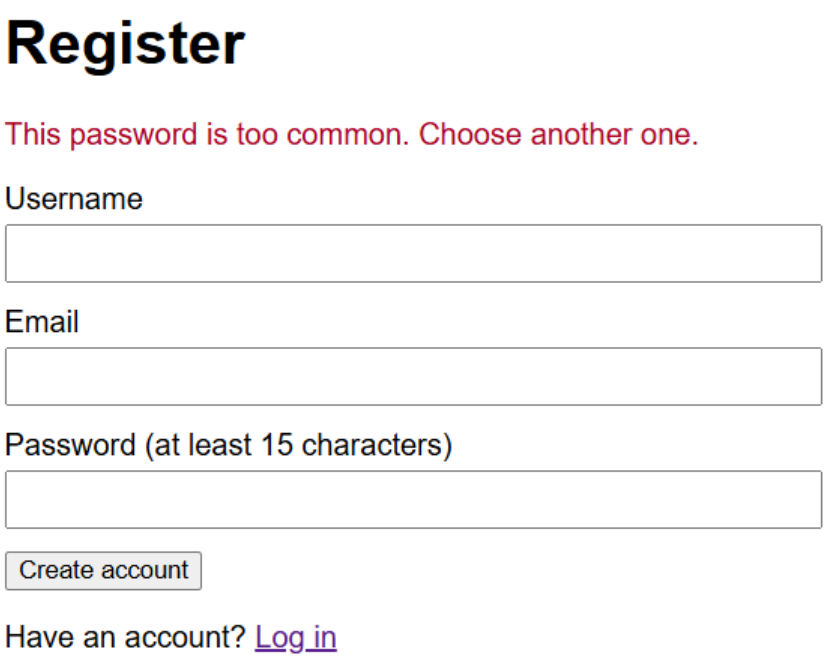


Figure 24. T10: registration with 1qaz2wsx3edc4rfv is rejected as a common password.

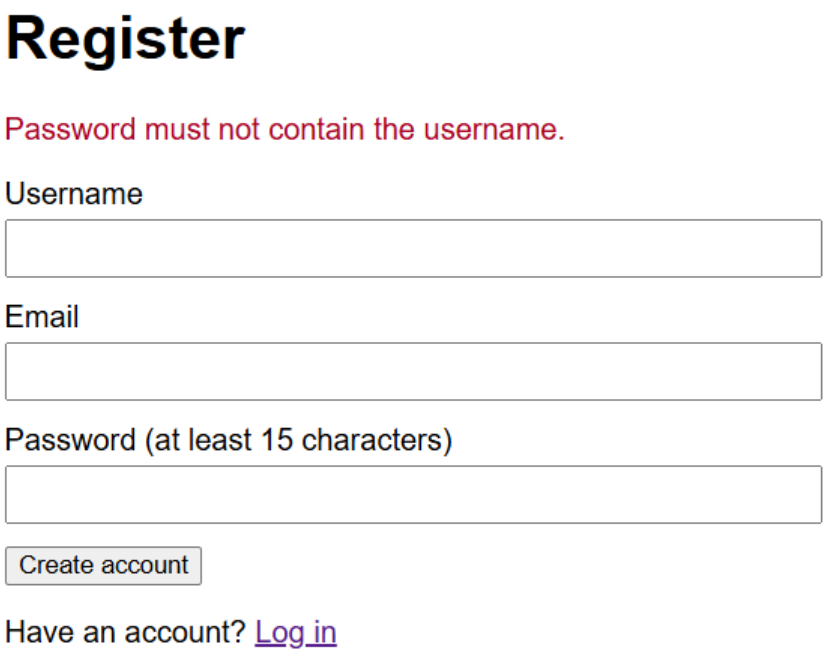


Figure 25. T11: a password that contains the username is rejected.

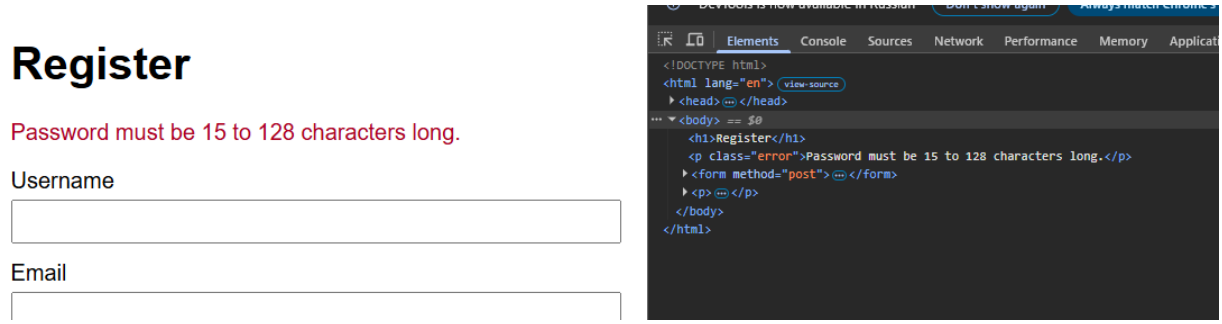


Figure 26. T3: with minlength removed, the server still rejects the 8-character password.

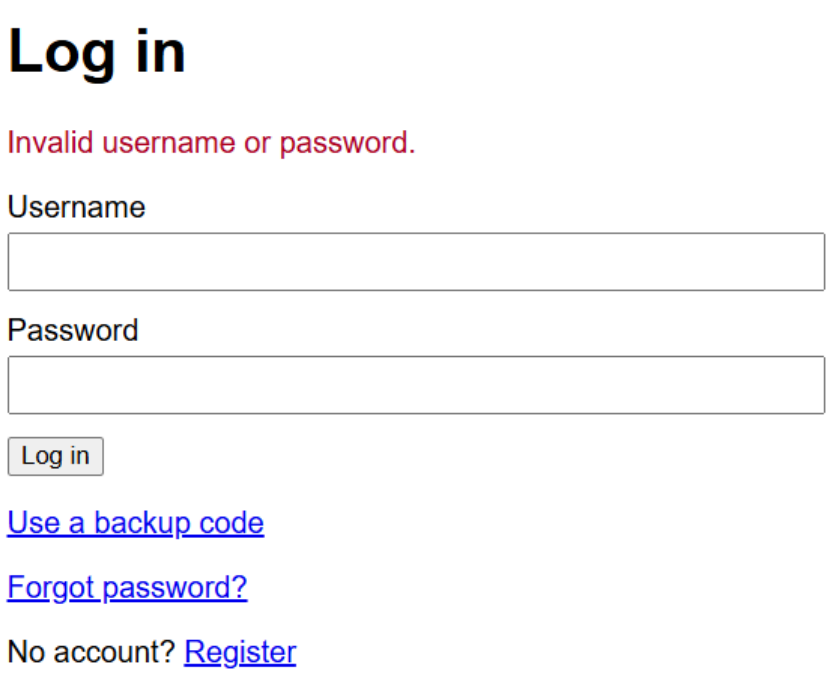


Figure 27. T6: an unknown user gets the same message as a wrong password.

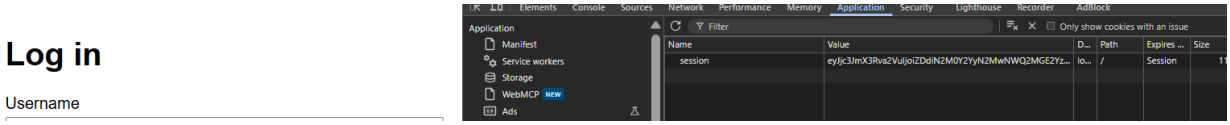


Figure 28. T7: after logout the old __Host-sid is rejected and the user is sent to the login page.

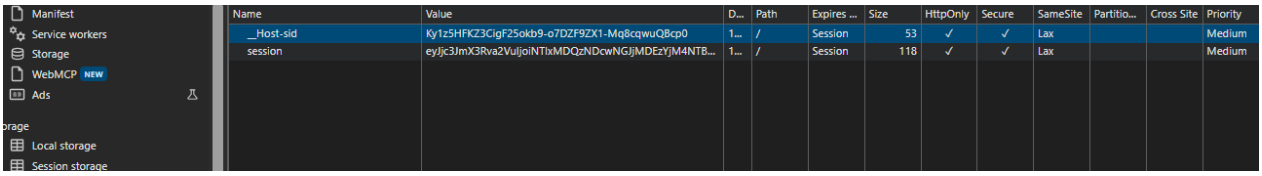


Figure 29. T12: __Host-sid appears only after login, with a fresh value (session rotation).

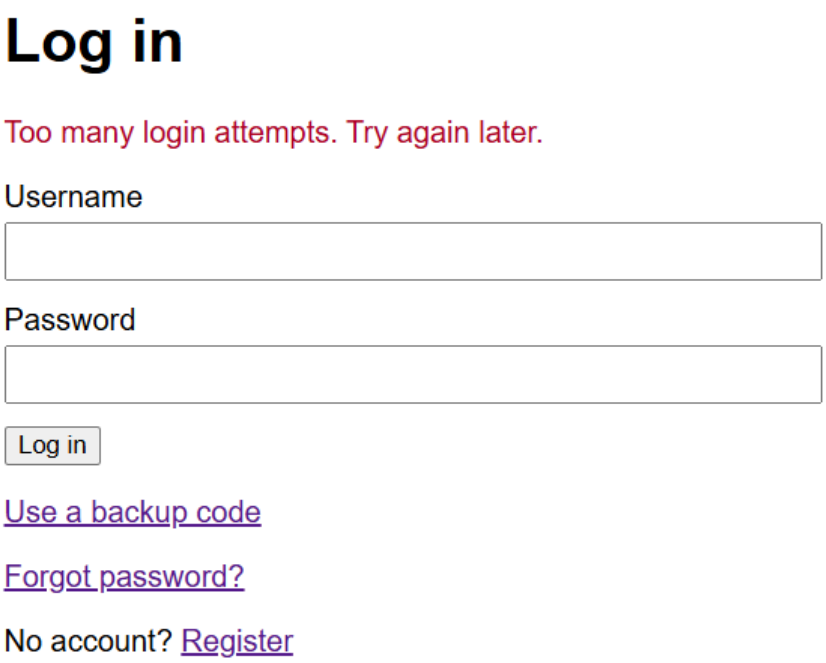


Figure 30. T9: the eleventh wrong login for one username is answered with HTTP 429.



Figure 31. T14: a login POST with the CSRF token removed is rejected with HTTP 400.

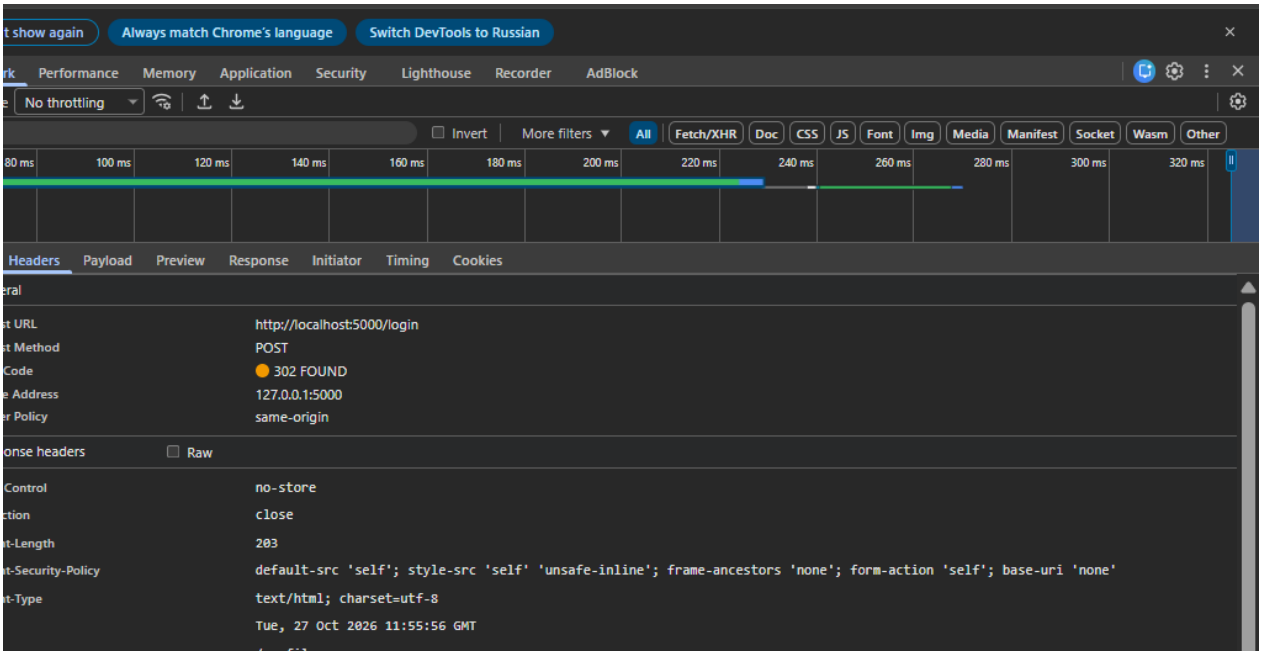


Figure 32. T13: the login response sets __Host-sid with Secure, HttpOnly and SameSite, alongside the security headers.

10.2 Full-solution tests

ID	Feature	Steps	Expected	Result	Evidence
T15	Authentication logging	Log in, log out, fail a login; open auth.log	Events recorded with user and IP; no password in the file	Pass	Figure 33
T16	Security headers	DevTools -> Network -> Response Headers on /login	CSP, X-Frame-Options, nosniff, Referrer-Policy, Cache-Control present	Pass	Figure 34
T17	Salt vs slow hashing	Run crack.py	SHA-256 cracks weak passwords instantly; Argon2id tens of thousands of times slower; strong password survives	Pass	Figure 35
T18	Password change	Change the password using the current one	Confirmation shown; all sessions signed out, including the current one	Pass	Figure 36
T19	Reset, no enumeration	Request a reset for a real email, then an unknown email	Identical message both times; a link printed only for the real email	Pass	Figure 37
T20	Reset applied, single-use	Open the link, set a new password; reuse the link	Old password refused, new one works; a reused link is invalid	Pass	Figure 38
T21	Backup codes	Generate codes; sign in with one	Codes shown once, stored hashed; a code works once	Pass	Figure 13
T22	Admin force reset	Admin forces a reset; the user tries to log in	Login blocked until the user resets via Forgot password	Pass	Figure 17

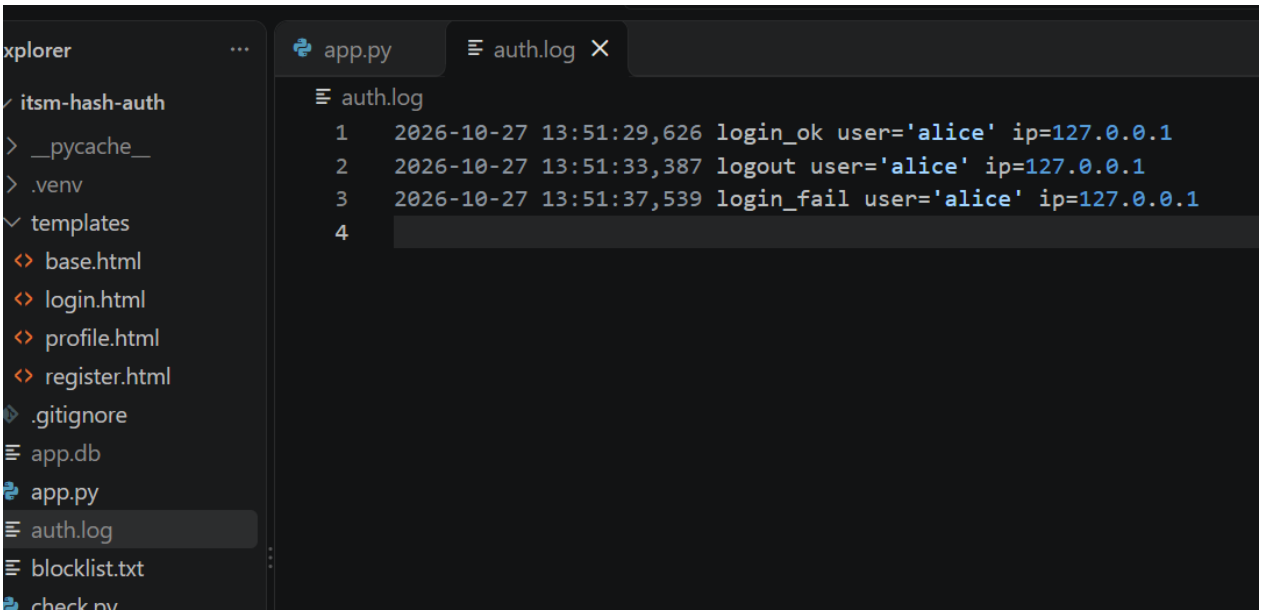


Figure 33. T15: auth.log records login, logout and failed login with user and IP; no password is written.

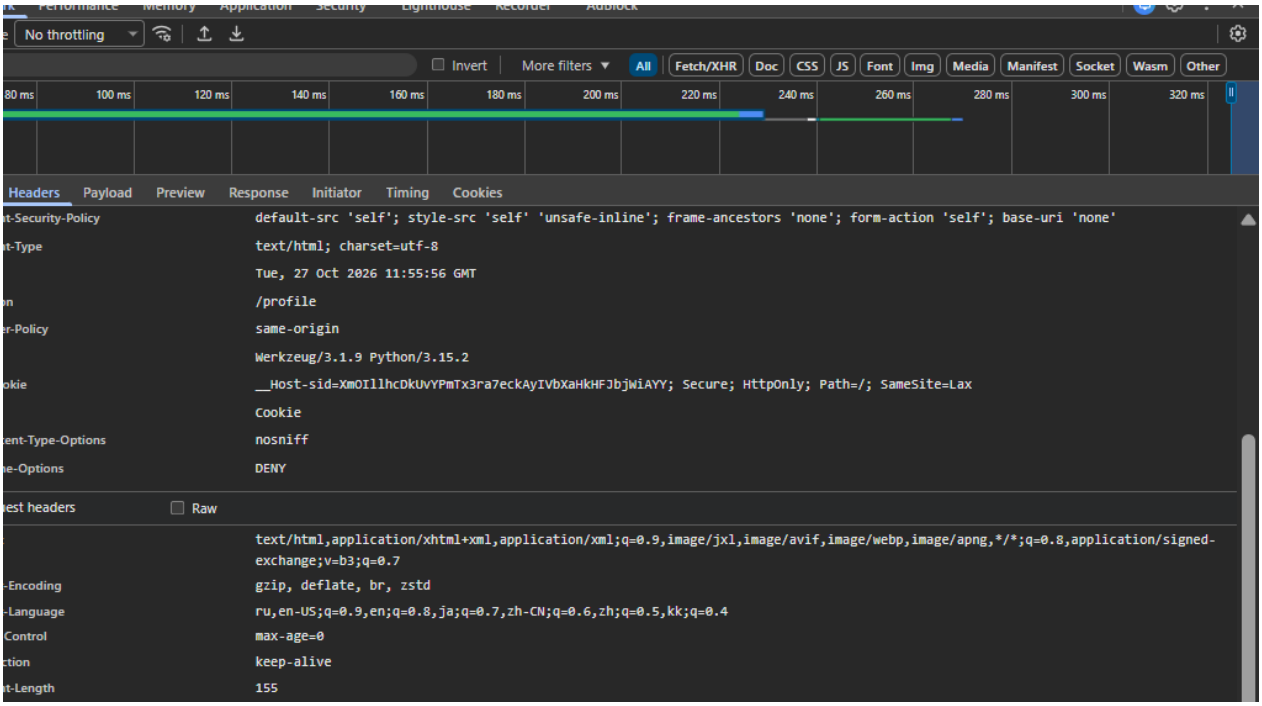


Figure 34. T16: the response headers on /login include CSP, nosniff, X-Frame-Options, Referrer-Policy and Cache-Control.

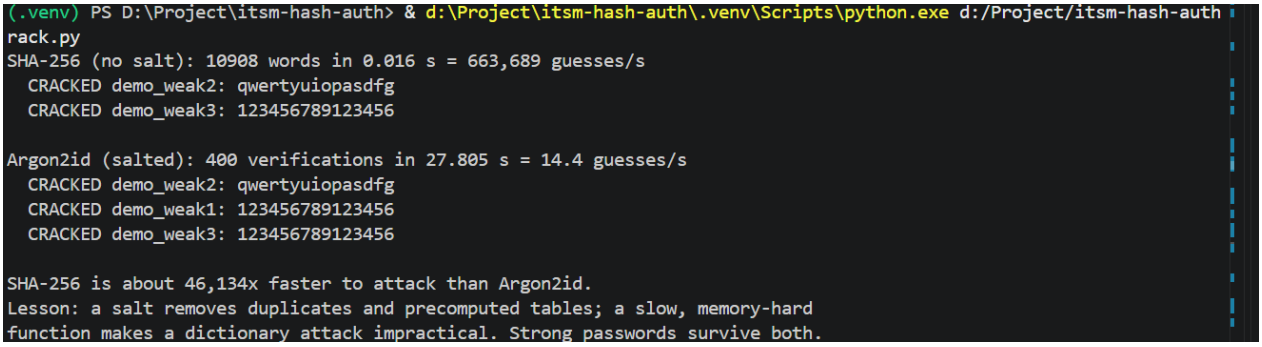


Figure 35. T17: crack.py: SHA-256 cracks the weak passwords instantly; Argon2id is about 46,000 times slower; the strong password survives.

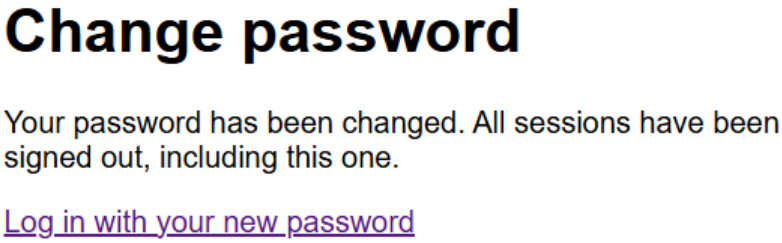


Figure 36. T18: after a password change, all sessions are signed out and the user must log in again.

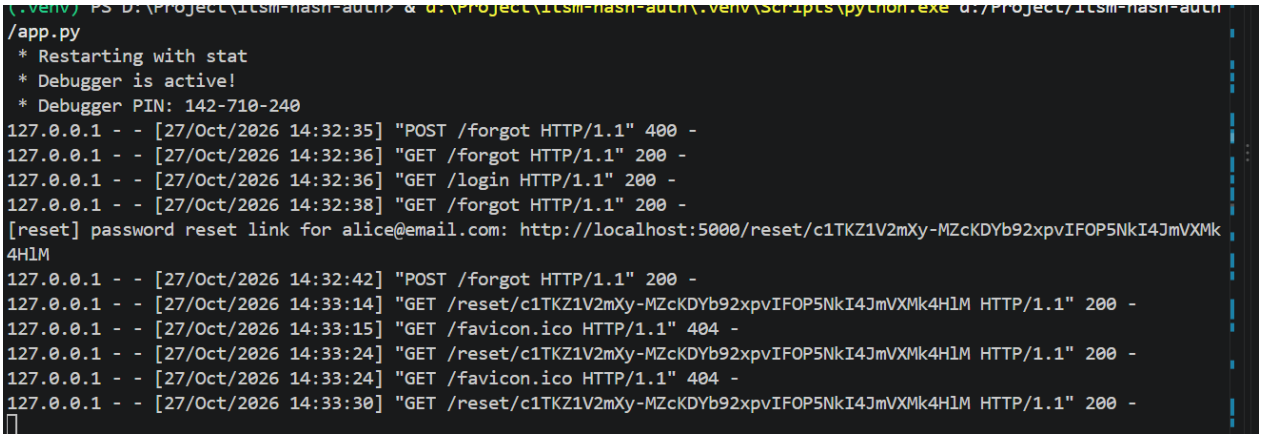


Figure 37. T19: the reset link is printed to the console only for a registered email; the page message is identical for any address.

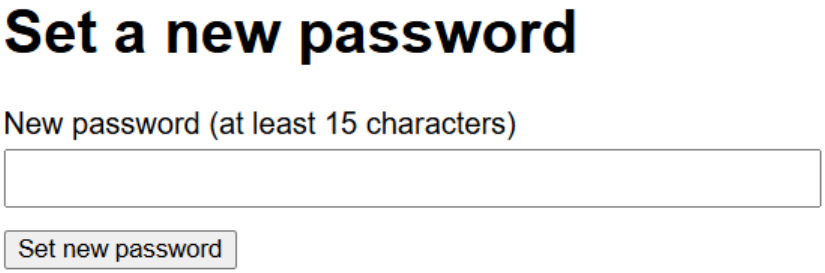


Figure 38. T20: setting a new password from the single-use reset link.

11. Independent penetration test

Ivann Aureus, who did not write the application, tested the finished build on 3 November 2026 as the team's breaker. The tests ran against the local application at `http://127.0.0.1:5000` in a Windows 11 VirtualBox guest, using Edge and Chrome, DevTools, console `fetch` requests and SQLite queries that were read-only or made on a backed-up database. Three dedicated accounts were used: `ivann-test`, `victim-test` and `admin-test`.

Each test records only what was observed. A pass means the stated attack failed under the stated conditions; it is not a guarantee against related attacks. Passwords, cookies and reset tokens are redacted in every screenshot. The full reports, with all evidence, are in Appendix A.

11.1 Results

ID	Attack attempted	Observed result	STRIDE	Result	Test log
PT-01	Register a new account and log in (baseline)	Login succeeded; profile shows role user, group students	-	Pass	T4
PT-02	Reuse a backup code that has already signed in	First use worked; the replay was refused with a generic message	S	Pass	T21
PT-03	Sign in as another user with one user's unused backup code	Refused with the same generic message	S	Pass	new
PT-04	Open a reset link again after it was used	"This reset link is invalid or has expired."	S	Pass	T20
PT-05	Open /admin as an ordinary user	HTTP 403 and a denial page	E	Pass	new

ID	Attack attempted	Observed result	STRIDE	Result	Test log
PT-06	Keep a second browser signed in, then change the password	The second session was sent to /login	S	Pass	T18
PT-07	Log in with the old password after a change	Old password refused; new password works	S	Pass	T18
PT-08	Six wrong logins in a row from one client	Attempts 1–5: HTTP 401; attempt 6: HTTP 429	S, D	Pass	T8
PT-09	Compare the error for a real and an unknown username	Same text and same HTTP 401	S, I	Pass	T5, T6
PT-10	SQL tautology ' OR '1'='1' -- as the password	Generic error; no session; no SQL error shown	S, T	Pass*	new
PT-11	POST /logout without a CSRF token	HTTP 400; the user stays signed in	T	Pass	T14
PT-12	Replay the session cookie after logout	Cookie was sent, but /profile redirected to /login	S	Pass	T7
PT-13	Use a reset link whose expiry lies in the past	Link refused; password unchanged	S	Pass	new
PT-14	Admin action with an empty or wrong admin password	"No action taken" until the correct password is given	E	Pass	new
PT-15	Admin forces a reset; user tries the old password, then recovers	Sessions 1→0; old password blocked; clean recovery clears the flag	S, R	Pass; F1	T22
PT-16	Disable an account that has an open session	Session ended; login refused with the generic error	E	Pass	new
PT-17	Revoke sessions for the whole students group	Three accounts, sessions 1→0 each; all sent to /login	E	Pass	new
PT-18	Read the session cookie's flags and access it from JavaScript	Secure, HttpOnly, SameSite=Lax, Path=/, no Domain; document.cookie empty	T, I	Pass	T13
PT-19	Unknown route, wrong HTTP methods, malformed POST	Generic 404, 405 and 400 pages; no traceback, path or debugger	I	Pass	new
PT-20	Inspect stored password values in SQLite	Argon2id v19, m=65536, t=3, p=4; three different salts; no plaintext column	I	Pass	T1

* PT-10 covers one payload in one field. The claim that SQL injection is not possible rests on the code, where every query passes user input as ? parameters (section 5.1), not on this single test.

All twenty tests passed. Twelve repeat a test from section 10, this time done by someone else. Eight cover attacks the builder's log did not try (marked new), mostly around the admin panel and the recovery flow.

11.2 Findings

F1. An admin action can be recorded twice (low). During PT-15 the target account showed `must_reset=yes` again after it had already recovered. The `audit_actions` table held two `force_reset` events for `victim-test`, at 13:57:38 and 14:04:40. A browser "Confirm Form Resubmission" prompt appeared during the test, so the likely cause is the browser re-sending the admin POST when the page was refreshed; which click caused it was not proven. A clean retest with no further admin action showed that recovery works and clears the flag.

Impact: an administrator can repeat a destructive action by accident, for example forcing a second reset just after the user recovered. Another user cannot trigger it, because the request still needs the admin role, the admin password and a valid CSRF token. Fix: answer every admin POST

with a redirect (Post/Redirect/Get), so that a refresh repeats only the harmless GET, and give each admin form a one-time action token. The audit trail is what made the duplicate visible, which shows the Repudiation defence working.

F2. Revoking your own group signs you out (informational). PT-17 confirmed the limitation already stated in section 7.3: the administrator was in the `students` group and lost their own session. Fix: exclude the acting administrator from group actions, or keep administrators in a separate group.

F3. Default error pages and debug mode (low). PT-19 found no sensitive detail in error responses, but the pages are the framework defaults. Custom handlers for 400, 404, 405 and 500 would give one consistent page, and a deployment would run behind a production WSGI server rather than the Flask development server. Figure 37 in section 10, taken on 27 October, shows the development server running with "Debugger is active"; PT-19 on 3 November found no debugger on error pages. The live demonstration and any deployment must run with debug mode off, because the Werkzeug debugger lets anyone who reaches it run code on the server.

11.3 What was not tested

The tester also wrote down what he did not test:

- Response time of failed logins for existing and unknown users. The dummy-hash design (section 5.3) is meant to make them equal, but the times have not been measured; mini task 9 gives the method.
- Rate limiting from several IP addresses or over the full one-hour window; PT-08 used one client.
- HTTPS. All tests ran over plain HTTP on the loopback address. Browsers treat `localhost` as a secure context, which is why the `Secure`, `__Host-` cookie works there; a real deployment needs TLS 1.2 or newer with HSTS.
- SQL injection beyond one payload, and JSON parsing: the malformed request in PT-19 was stopped by the CSRF check before the parser ran.
- The natural 30-minute expiry of a reset token; PT-13 simulated it by editing the expiry in the database.

12. Mini tasks

The lecture asks for a one-sentence result from each in-class mini task. The tasks below map onto work done in this project; the last column points to the test that shows it.

#	Mini task	Result in one sentence	Evidence
4	Cookie flag lab	The session cookie `__Host-sid` is invisible to page scripts (`document.cookie` returns an empty string), so a script injected into the page cannot steal it.	T13, PT-18
7	Why "slow" hashing	Argon2id made each guess about 46,000 times slower than unsalted SHA-256, and two users with the same password got different hashes because each hash has its own salt.	T17, T1

#	Mini task	Result in one sentence	Evidence
9	Username enumeration and timing	A wrong password and an unknown user give the same text and status code; the dummy hash should equalise the timing as well, but that has not been measured yet.	T5, T6, PT-09
10	Session fixation check	The session ID changes at login and the old cookie is useless after logout, because the server deletes the session record instead of only clearing the cookie.	T12, T7, PT-12
11	Rate limiting	The sixth wrong login in a minute from one client and the eleventh in an hour on one username get HTTP 429; the per-account limit stops distributed guessing but lets anyone throttle a known user for an hour.	T8, T9, PT-08
14	Account-recovery tabletop	Run against the real flow rather than on paper: the reset link works once, is refused after expiry, answers the same for unknown emails and ends every session after use; the rule we keep is that every credential change ends all sessions.	PT-04, PT-13, T19, PT-06

Mini tasks 1, 2, 3, 5, 6, 8, 12 and 13 are not recorded in this report.

13. Did backup, reset and revocation raise or lower security?

Both. The net effect is positive only because each addition was hardened. Every recovery feature is a new way into the account, so each one widens the attack surface before any defence is added. The question is whether the defence closes more than the feature opens.

Addition	What it lets an attacker try	How it was blocked	Net effect
Backup codes	A second credential to steal or guess; codes left valid forever	Stored as Argon2id hashes; shown once; one-time use; rate-limited login; random codes	Positive: a usable fallback without a reusable secret on the server
Credential reset	Hijack the reset link; discover which emails exist; reuse an old link	Single-use token, only its SHA-256 stored, 30-minute expiry; identical response for known and unknown emails; all sessions killed after reset	Positive, provided the email channel is trusted (the weakest link)
Admin revocation	A stolen admin session disables accounts; abuse goes untraced	Role check; re-authentication on every action; every action written to audit_actions and the log	Positive: it is itself a defence (fast containment), and it is audited

The independent test supports each 'blocked' claim in the table. Backup codes could not be replayed or used across accounts (PT-02, PT-03); reset links could not be reused or used after expiry (PT-04, PT-13); admin actions needed the admin password and reached exactly one user or one group (PT-14 to PT-17). It also found the one weakness the design missed, the repeated

admin action F1. So the conclusion stays the same: every new way into an account has to be protected as carefully as the login.

The recovery flow does shift risk toward the email account: anyone who controls the registered mailbox can reset the password. That is inherent to email-based recovery and is why the lecture calls account recovery the weakest link. In a production system the mitigation would be a second recovery factor (a backup code or a second passkey) for high-value accounts, plus notifying the old address on every reset.

So the additions raise security overall, because each new path is built with the same discipline as the password itself (hashed secrets, single use, expiry, rate limiting, audit) rather than left as a convenient back door.

14. Conclusion

The application secures its login with the mechanism of the topic: passwords are stored only as Argon2id hashes with a per-user salt, so a leaked database reveals no passwords and equal passwords give different records. The `crack.py` demonstration makes the point concrete, with unsalted SHA-256 guessing tens of thousands of times faster than Argon2id.

Around that core the project adds the controls that a password login needs in practice: rate limiting, a common-password blocklist, a generic error with a dummy hash against username enumeration, server-side sessions with secure cookies and rotation, CSRF protection, security headers, and an authentication log. The full solution adds a backup-code login, a single-use credential reset with no user enumeration, and an administrator panel that revokes or resets credentials for one user or a whole group, all audited.

A second team member then ran twenty attacks against the finished application, and all of them failed. The test found one low-severity issue, an admin action that a browser re-submission can repeat, and listed what remains untested: login timing, distributed guessing and a TLS deployment.

The STRIDE model and the ASVS Level 1 checklist show the design is sound against the common threats, with two small gaps (no show-password toggle, and no breached-password check, which is a Level 2 requirement). The main residual risk is inherent to email-based recovery. For the scope of this assignment, the mechanism is implemented correctly and the recovery and revocation features raise security rather than lower it.

References

1. ITSM Practical Work No. 2: Web Authentication Security, lecture slides, VILNIUS TECH, 2026–2027 fall semester.
2. OWASP Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
3. argon2-cffi API reference. https://argon2-cffi.readthedocs.io/en/latest/api.html
4. OWASP ASVS 5.0, V6.2 Password Security. https://cornucopia.owasp.org/taxonomy/asvs-5.0/06-authentication/02-password-security
5. OWASP ASVS 5.0, V11.4 Hashing and Hash-based Functions. https://cornucopia.owasp.org/taxonomy/asvs-5.0/11-cryptography/04-hashing-and-hash-based-functions
6. RFC 9106, Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications, IRTF, 2021.

7. SecLists, xato-net-10-million-passwords-1000000.txt. https://github.com/danielmiessler/SecLists

Appendix A. Independent penetration test reports

Below are Ivann Aureus's four test reports. Only the title pages were removed and the headings renumbered. Section 11 summarises them. Passwords, session cookies and reset tokens are hidden in every screenshot.

A.1 Report 1: PT-01 to PT-05: backup codes, reset token and admin access

ITSM Hash Authentication Web Application

Tester	**Ivann Aureus**
Application	ITSM Hash Authentication Web Application
Environment	Local Windows test environment
Application address	127.0.0.1:5000
Scope	Authentication, recovery, session handling, and authorization

Purpose: check, as someone who did not write the code, whether the login and recovery controls stop replay, use of one user's credentials for another user, and direct access to admin pages. Only what was actually seen during the test session is written down.

PT-01: Registration and Normal Authentication

Objective: Verify that a newly registered user can authenticate normally, establishing a baseline for adversarial testing.

1. Accessed the registration page and created the dedicated test account ivann-test.
2. Used a password satisfying the application's minimum 15-character requirement.
3. Authenticated using the newly created account.
4. Verified the resulting profile and authorization context.

Expected result: Valid credentials authenticate successfully and create an authenticated user session.

Actual result: Authentication succeeded and redirected to /profile. The profile identified ivann-test with role user and group students.

Result: PASS

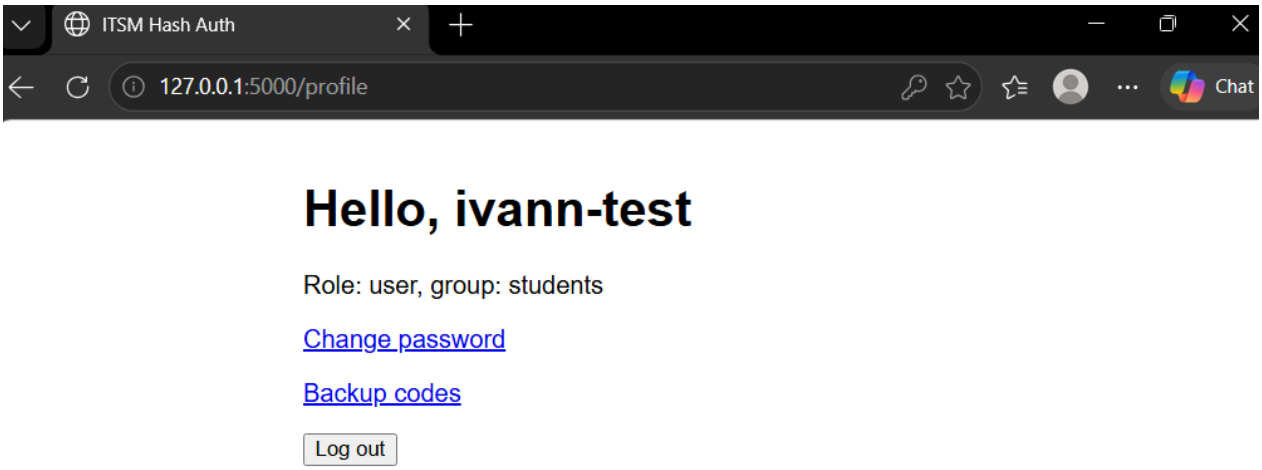


Figure PT-01. Successful authentication baseline: ivann-test profile, role user, group students.

PT-02: Backup Code Replay Attack

Objective: Determine whether a backup authentication code can be reused after it has already authenticated successfully.

5. Generated 10 backup codes for ivann-test.
6. Logged out and authenticated using one unused backup code.
7. Confirmed that the first use successfully authenticated the account.
8. Logged out and submitted the exact same backup code a second time.

Expected result: The first use succeeds; the consumed code is invalidated and a replay attempt is rejected.

Actual result: The first use succeeded. Reusing the same code returned 'Invalid username or backup code.'

Result: PASS

Conclusion: Backup codes were enforced as single-use credentials, preventing replay. The failure message was generic and did not disclose which field was incorrect.

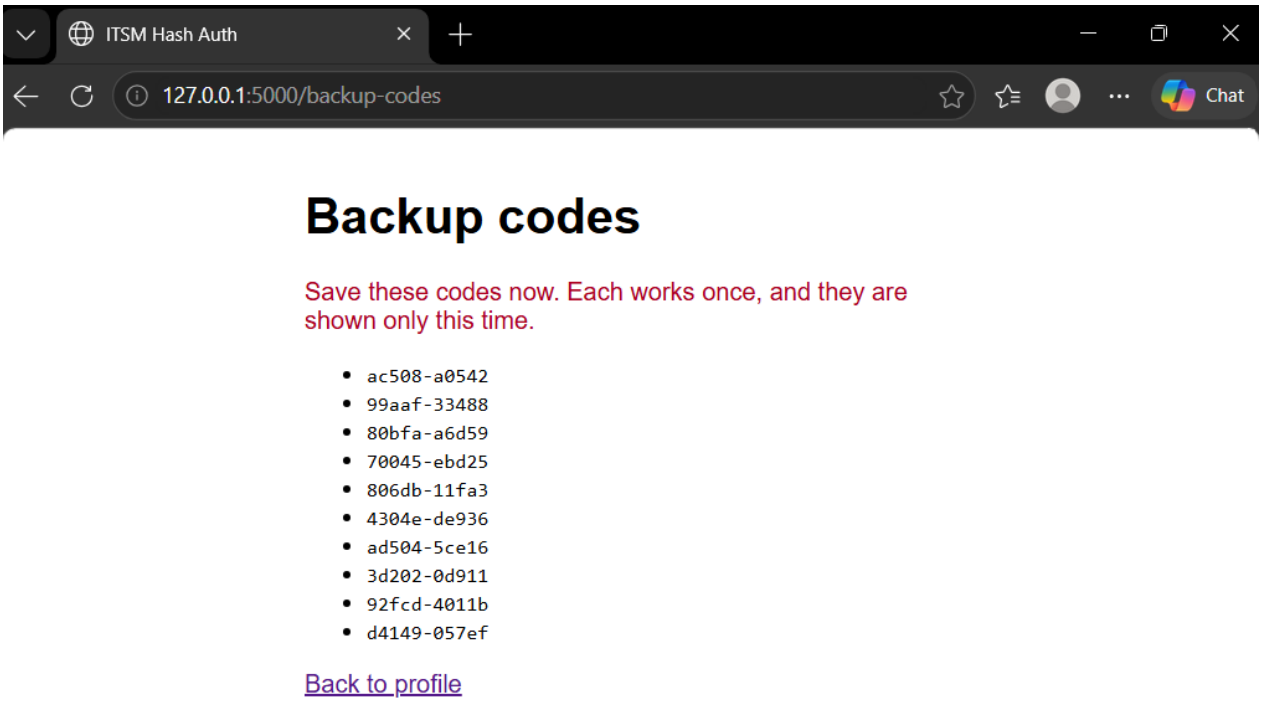


Figure PT-02A. Ten backup codes generated; the interface states that each code works once and is shown only once.

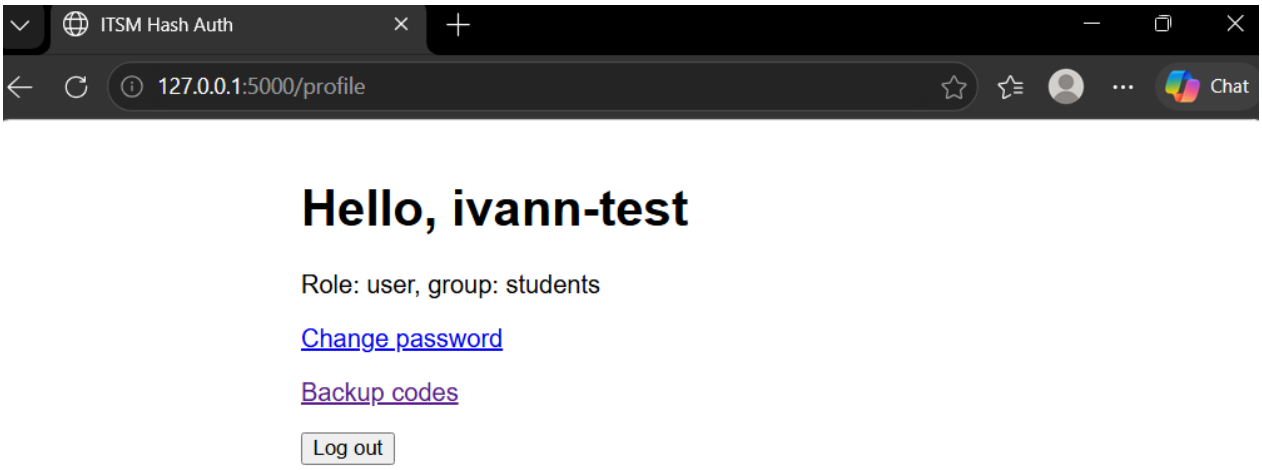


Figure PT-02B. First use of the selected backup code successfully authenticated ivann-test.

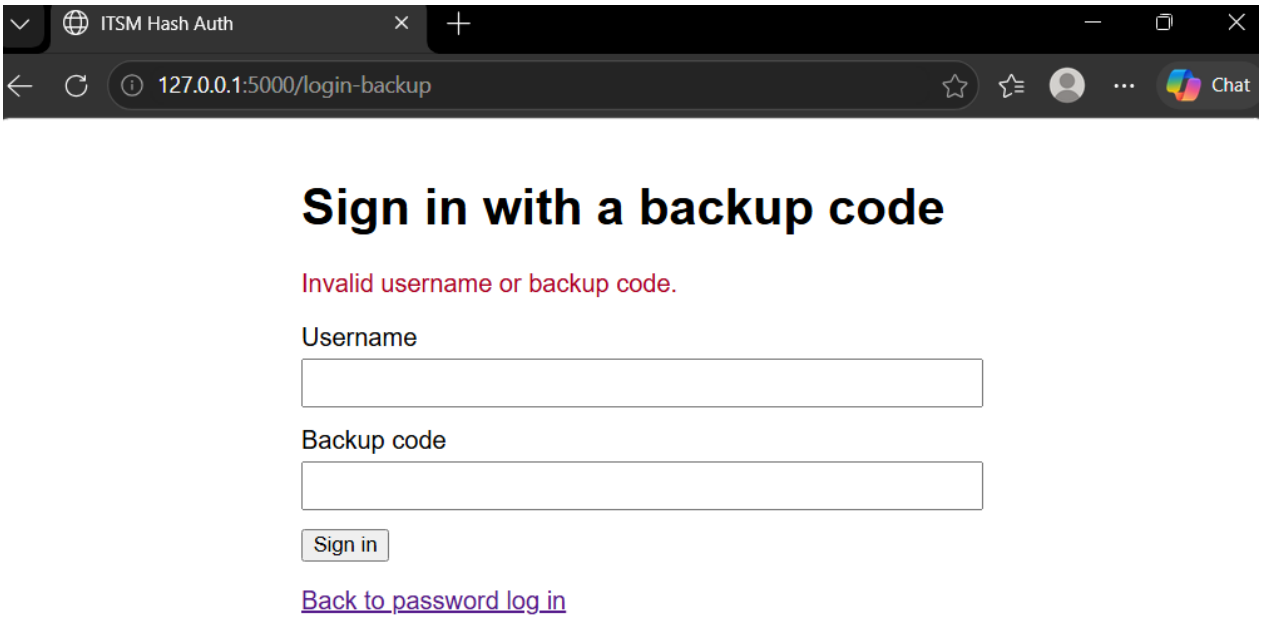


Figure PT-02C. Replay of the already-consumed backup code was rejected.

PT-03: Cross-Account Backup Code Attack

Objective: Determine whether a valid unused backup code issued to one user can authenticate a different user.

9. Created a second account named victim-test.
10. Selected a valid unused backup code previously generated for ivann-test.
11. Submitted victim-test as the username together with ivann-test's valid unused backup code.
12. Observed the authentication result.

Expected result: Authentication is rejected because backup codes must be bound to the account that generated them.

Actual result: The application rejected the attempt with 'Invalid username or backup code.'

Result: PASS

Conclusion: A valid backup code belonging to one account could not authenticate another account, demonstrating account binding of backup credentials.

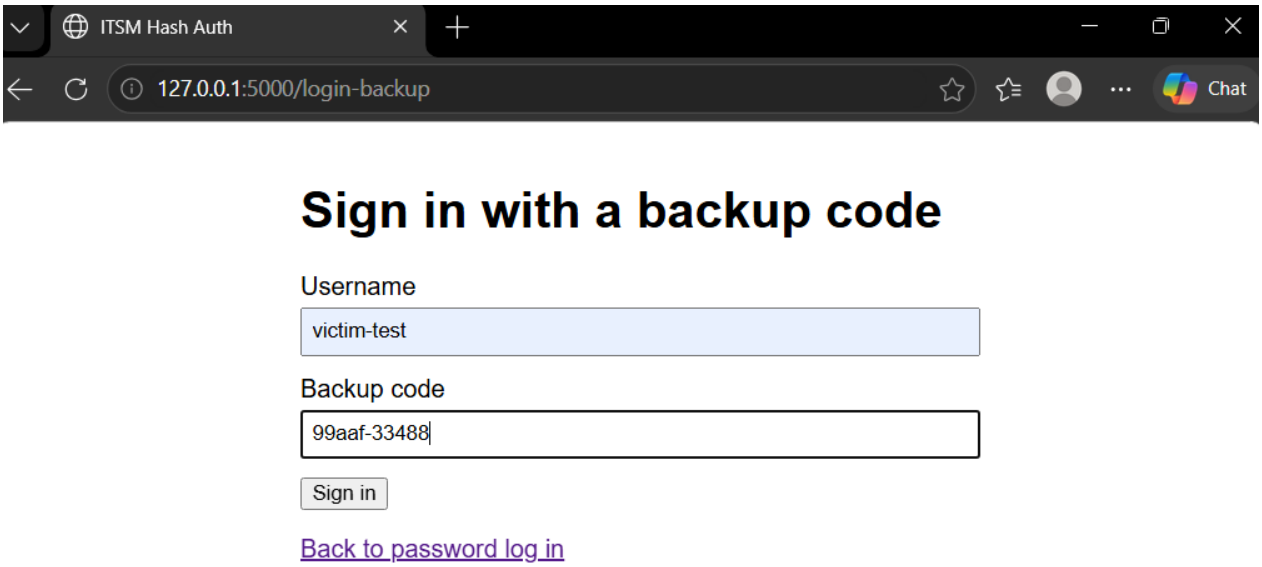


Figure PT-03A. Cross-account attempt: victim-test submitted with a valid unused backup code issued to ivann-test.

Note: there is no screenshot of the rejection for PT-03, only of the attempt. The result is written from what was seen on screen.

PT-04: Password Reset Token Replay Attack

Objective: Determine whether a password-reset token remains usable after a successful password reset.

13. Submitted the registered email address for ivann-test through /forgot.
14. Observed the generic recovery response and obtained the development reset URL from the Flask server console.
15. Opened the valid reset URL and changed the password successfully.
16. Observed that the application reported all sessions had been signed out.
17. Opened the exact same reset URL again using a fresh GET request.

Expected result: The token permits one successful reset and becomes unusable immediately afterward.

Actual result: The initial reset succeeded. Reuse of the identical reset URL returned 'This reset link is invalid or has expired.'

Result: PASS

Conclusion: The reset token was single-use and resistant to replay. The reset flow also invalidated existing sessions and used a generic forgot-password response that reduced account-enumeration disclosure.

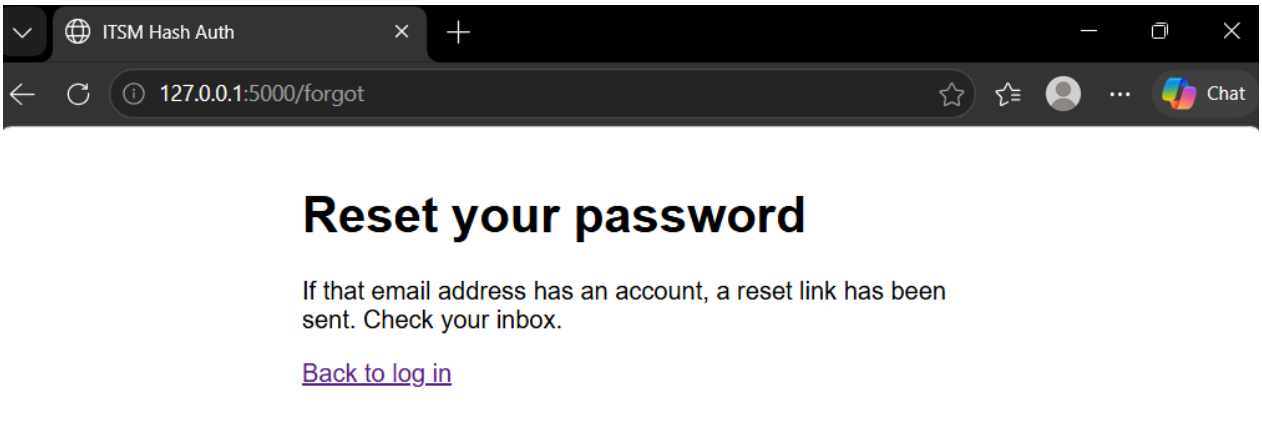


Figure PT-04A. Generic password-recovery response avoids confirming whether the submitted email exists.

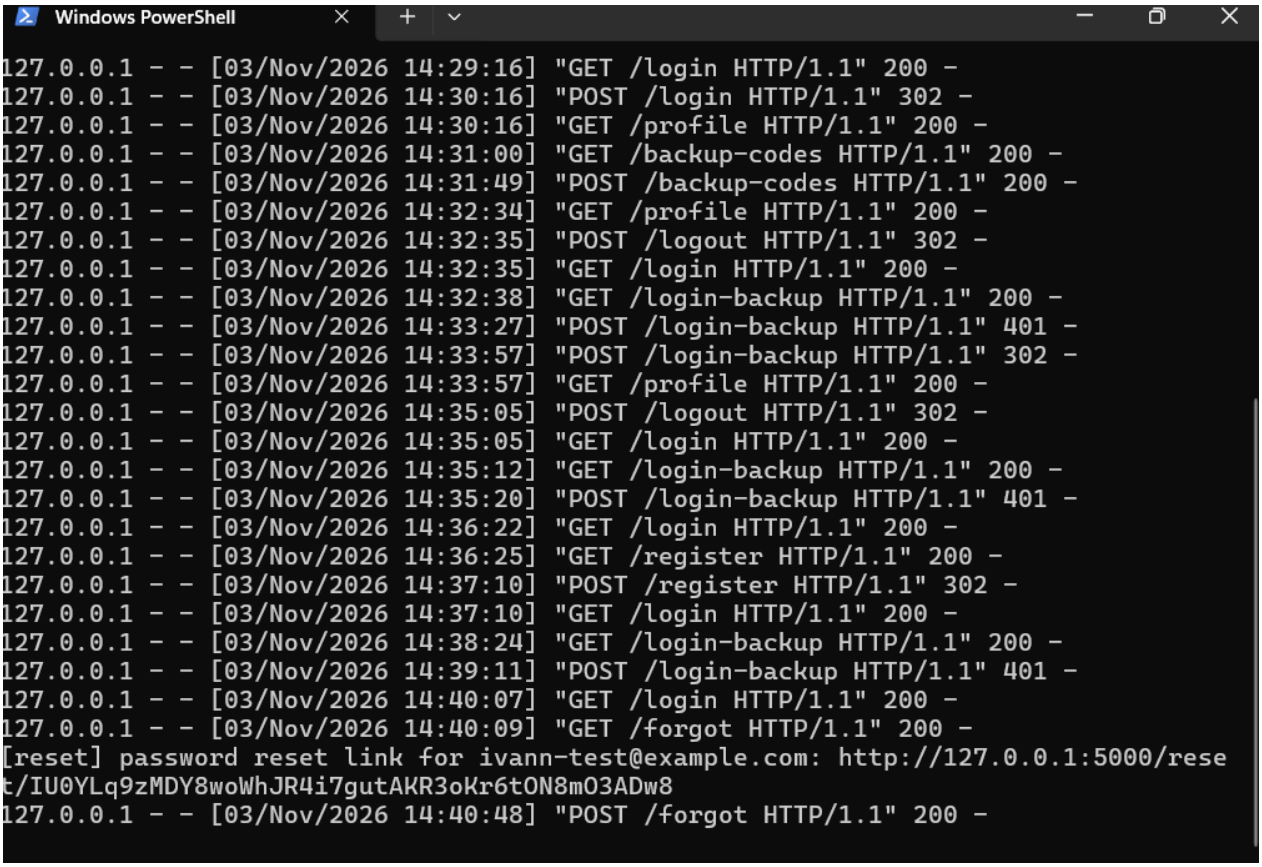


Figure PT-04B. Development server issued the password-reset URL for the test account.

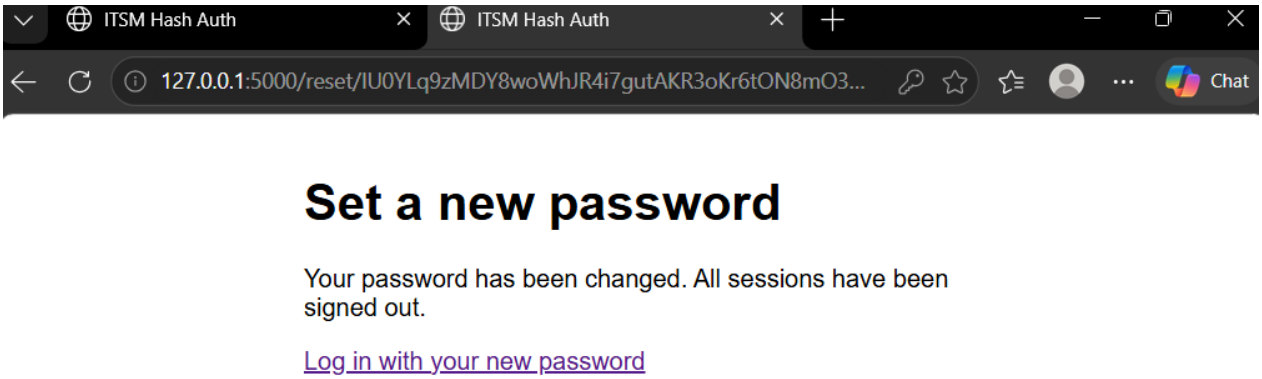


Figure PT-04C. Successful password reset; application reports that all sessions were signed out.

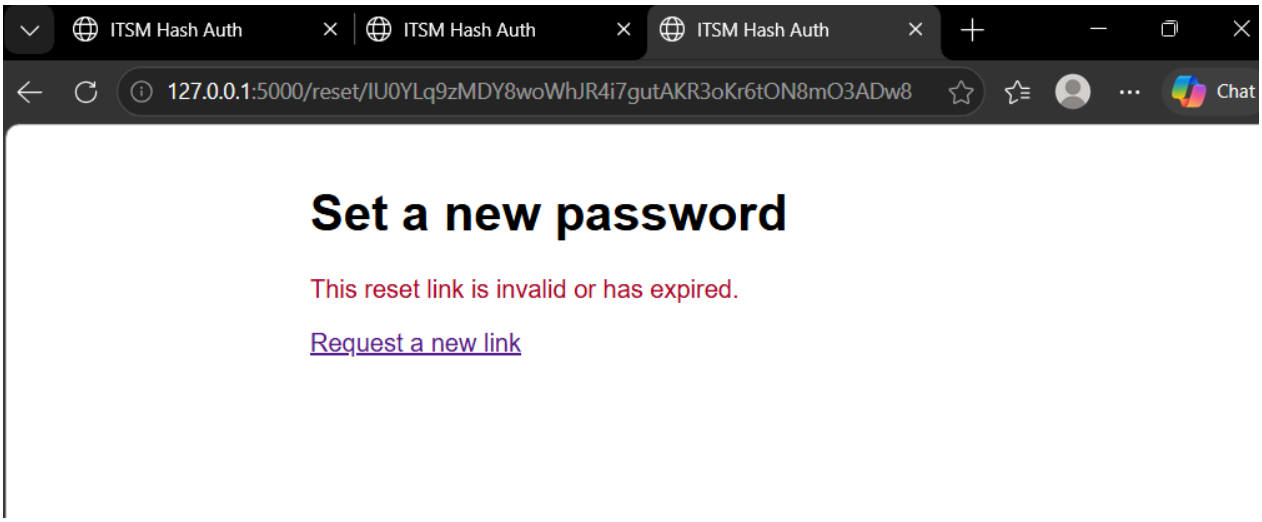


Figure PT-04D. Reuse of the same reset URL was rejected as invalid or expired.

PT-05: Direct Admin Authorization Bypass Attempt

Objective: Determine whether an authenticated ordinary user can access administrator functionality by manually requesting /admin.

18. Authenticated as ivann-test, an ordinary user account.
19. Manually requested /admin in the browser.
20. Observed the application denial page.
21. Verified the corresponding HTTP response in the Flask server log.

Expected result: Server-side authorization denies access and returns HTTP 403 Forbidden.

Actual result: The browser displayed 'You must be signed in as an administrator to view this page.' The Flask log confirmed GET /admin HTTP/1.1 403.

Result: PASS

Conclusion: Role-based access control was enforced server-side. Knowledge of the administrator route did not allow privilege escalation.

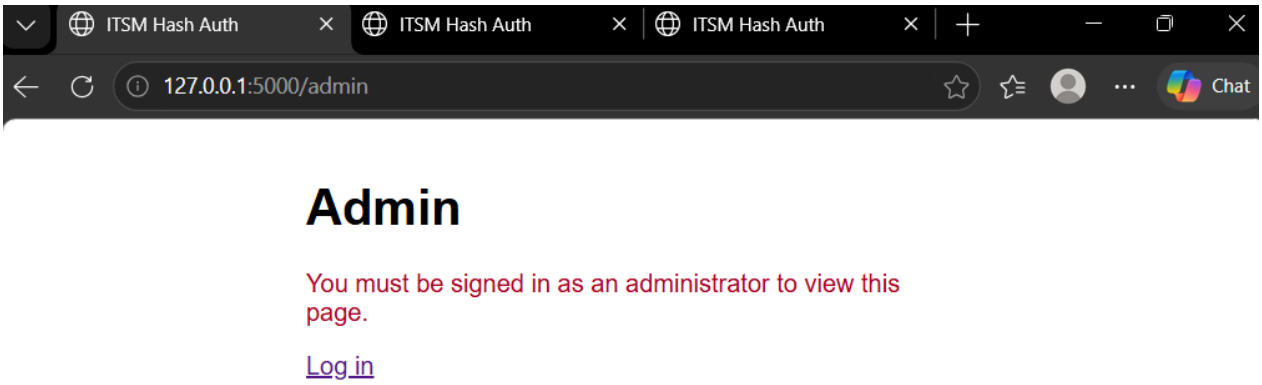


Figure PT-05A. Direct request to /admin by the ordinary user was denied.

```
127.0.0.1 - - [03/Nov/2026 14:52:35] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [03/Nov/2026 14:53:05] "POST /login HTTP/1.1" 401 -
127.0.0.1 - - [03/Nov/2026 14:53:23] "POST /login HTTP/1.1" 302 -
127.0.0.1 - - [03/Nov/2026 14:53:23] "GET /profile HTTP/1.1" 200 -
127.0.0.1 - - [03/Nov/2026 14:53:42] "GET /admin HTTP/1.1" 403 -
```

Figure PT-05B. Flask server log confirms HTTP 403 for the unauthorized /admin request.

Test Summary

Test	Security test	Expected defense	Observed result	Status
PT-01	Registration / normal login	Valid credentials authenticate	Authentication successful	PASS
PT-02	Backup-code replay	Consumed code cannot be reused	Replay rejected	PASS
PT-03	Cross-account backup code	Code valid only for owner	Cross-account use rejected	PASS
PT-04	Reset-token replay	Used token becomes invalid	Reused token rejected	PASS
PT-05	Direct /admin access	Unauthorized role denied	HTTP 403	PASS

Overall: all five tests behaved as expected and no way around the login or the admin check was found. Next to test: admin revocation, forced reset, disabled accounts, CSRF, input handling and more authorization cases.

A.2 Report 2: PT-06 to PT-10: password change, rate limiting, enumeration, SQL injection

ITSM Hash Authentication Web Application

Tester	Ivann Aureus
Test date	3 November 2026
Application	ITSM Hash Auth (local Flask web application)
Target	http://127.0.0.1:5000
Scope	Password changes, session revocation, rate limiting, username enumeration, and login SQL-injection probe
Assessment type	Independent manual tests of the team's authorized local application

Test overview

Test	Control tested	Result
PT-06	Session revocation after password change	**PASS**
PT-07	Old vs. new password after password change	**PASS**
PT-08	Login brute-force rate limiting	**PASS**
PT-09	Username enumeration via error responses	**PASS**
PT-10	SQL-injection authentication-bypass probe	**PASS***

* PT-10 tested only one payload in the password field, so it does not prove the app is safe from SQL injection in general.

All screenshots were taken during the test session. Results are based on what was visible in the browser and in the Flask log.

PT-06: Cross-session revocation after password change

Verify that changing an account password invalidates another concurrently authenticated session, not just the browser performing the change.

- Maintain a signed-in ivann-test session in an InPrivate browser window while also signed in through a normal window.
- Change the account password in the normal browser.
- Observe the password-change confirmation, then refresh the untouched InPrivate /profile session.

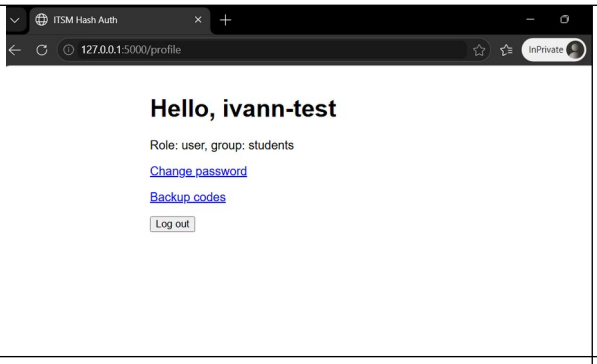
Expected: The password change revokes all existing sessions; the second browser must no longer access /profile.

Actual: The application displayed "Your password has been changed. All sessions have been signed out, including this one." Refreshing the previously authenticated InPrivate session returned it to /login.

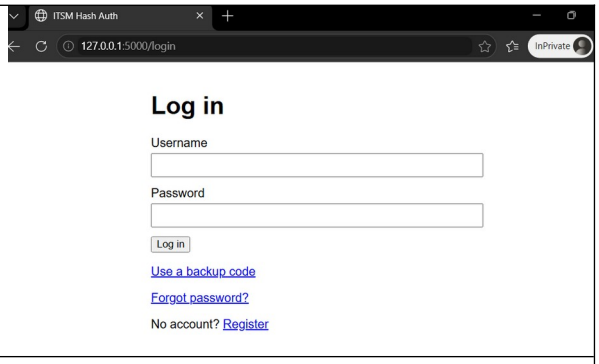
Result: PASS

Interpretation: the password change ended the session on the server, not only in the browser that made the change.

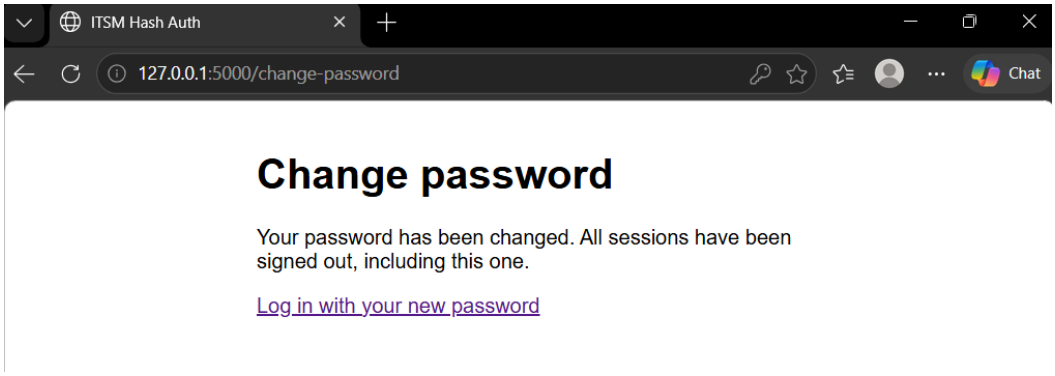
Evidence



PT-06A: InPrivate /profile authenticated before the change.



PT-06C: InPrivate window at /login after the change.



PT-06B: Password-change confirmation states all sessions were signed out.

PT-07: Old password rejected, new password accepted

Verify that an account password change replaces the login credential instead of allowing both passwords.

- Complete the password change for the existing ivann-test account.
- Attempt login using the former password and observe the rejection.
- Attempt login using the newly configured password and observe successful authentication.

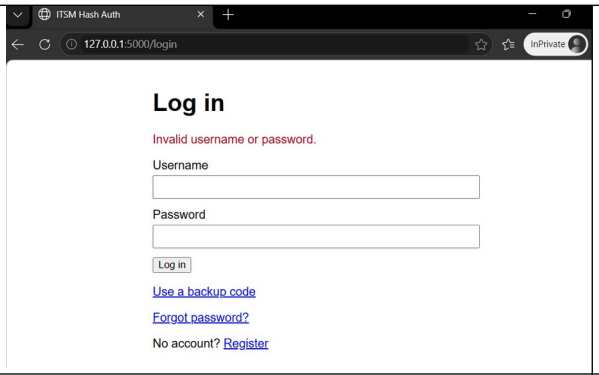
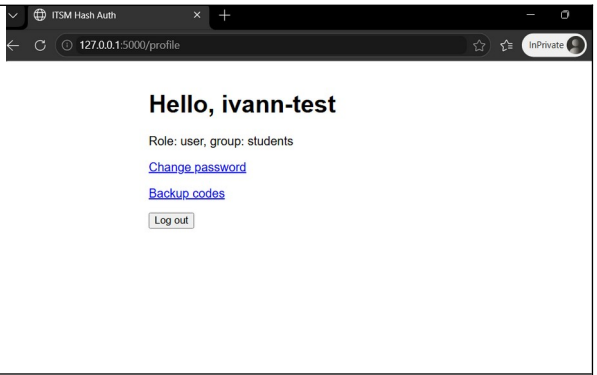
Expected: The former password fails; the replacement password authenticates the account.

Actual: The old-password attempt returned “Invalid username or password.” The new-password attempt reached /profile and displayed the authenticated ivann-test account.

Result: PASS

Interpretation: after the change only the new password works.

Evidence

	
PT-07A: Previous password rejected by /login.	PT-07B: New-password login reached /profile.

Note: the screenshots show the rejected and the successful login; the passwords themselves are not shown.

PT-08: Repeated failed login attempts / throttling

Verify that consecutive incorrect login requests from the local test client reach a rate limit rather than remaining unlimited.

- Submit repeated wrong-password login attempts for ivann-test from the local test browser.
- Observe the browser response on the sixth attempt.
- Compare with the local Flask access log.

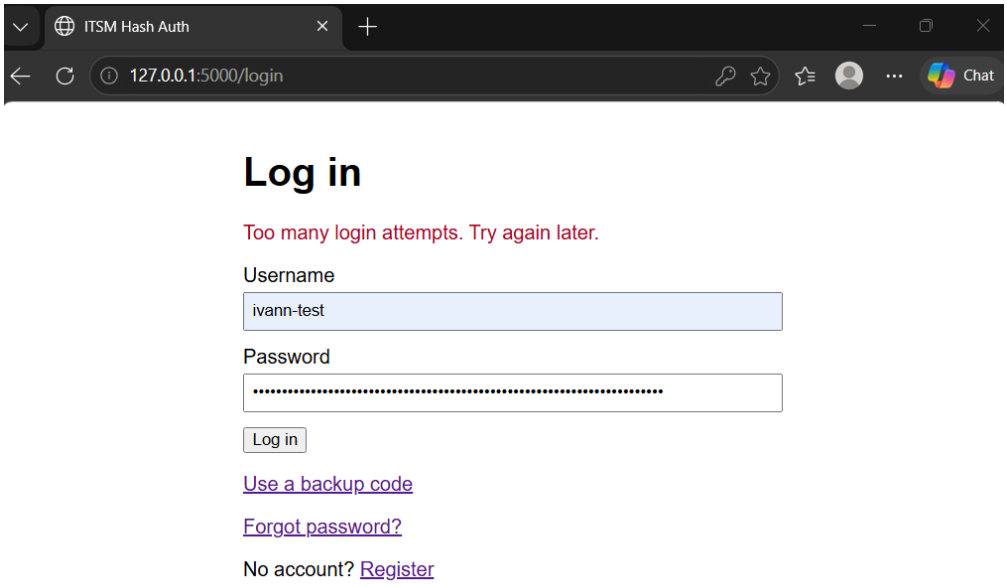
Expected: Excessive failed attempts are throttled and return a limit response.

Actual: The application returned the message “Too many login attempts. Try again later.” The recorded access log shows five POST /login 401 responses followed by a sixth POST /login 429 response.

Result: PASS

Interpretation: the limit works for six quick attempts from one browser. Attacks from many IP addresses or spread over a longer time were not tested.

Evidence



PT-08A: Browser displays the login rate-limit message.

```
127.0.0.1 - - [03/Nov/2026 15:35:49] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [03/Nov/2026 15:36:32] "POST /login HTTP/1.1" 401 -
127.0.0.1 - - [03/Nov/2026 15:36:51] "POST /login HTTP/1.1" 401 -
127.0.0.1 - - [03/Nov/2026 15:36:56] "POST /login HTTP/1.1" 401 -
127.0.0.1 - - [03/Nov/2026 15:37:00] "POST /login HTTP/1.1" 401 -
127.0.0.1 - - [03/Nov/2026 15:37:04] "POST /login HTTP/1.1" 401 -
127.0.0.1 - - [03/Nov/2026 15:37:07] "POST /login HTTP/1.1" 429 -
```

PT-08B: Flask access log: failed requests return 401; final request returns 429.

PT-09: Username enumeration through login errors

Compare the login error shown for a known account versus an account name not registered in the test application.

- Submit an incorrect password for the existing ivann-test account.
- Submit an incorrect password for a nonexistent username (ivann).
- Compare application messages and logged HTTP status codes.

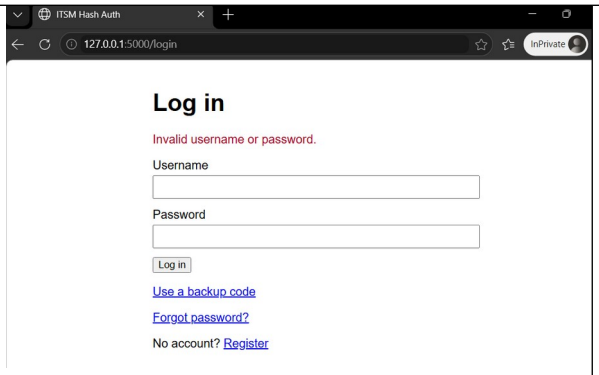
Expected: Both failures should use an indistinguishable generic error instead of revealing whether the username exists.

Actual: Both attempts displayed “Invalid username or password.” Flask access entries for the compared login failures were HTTP 401.

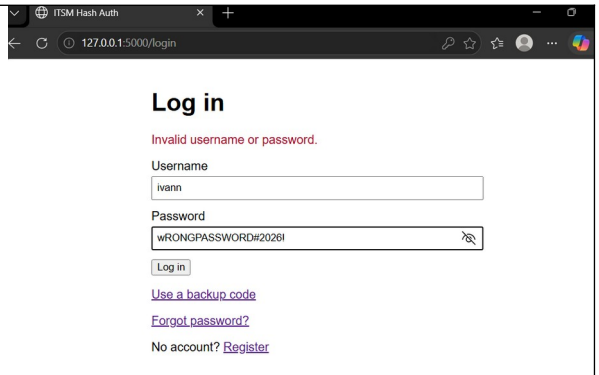
Result: PASS

Interpretation: the message and the status code are the same for both accounts. Response time was not measured.

Evidence



PT-09A: Wrong password for known account: generic error.



PT-09B: Nonexistent ivann username: same error.



PT-09C: Local access log includes HTTP 401 responses from the failed login tests.

PT-10: SQL-injection login bypass probe

Check whether a simple SQL tautology entered as the password can bypass the normal authentication check.

- Open /login in the local application.
- Enter ivann-test as the username.
- Submit the SQL-injection-style password value: ' OR '1'='1' --
- Observe the browser response and whether an authenticated profile page is reached.

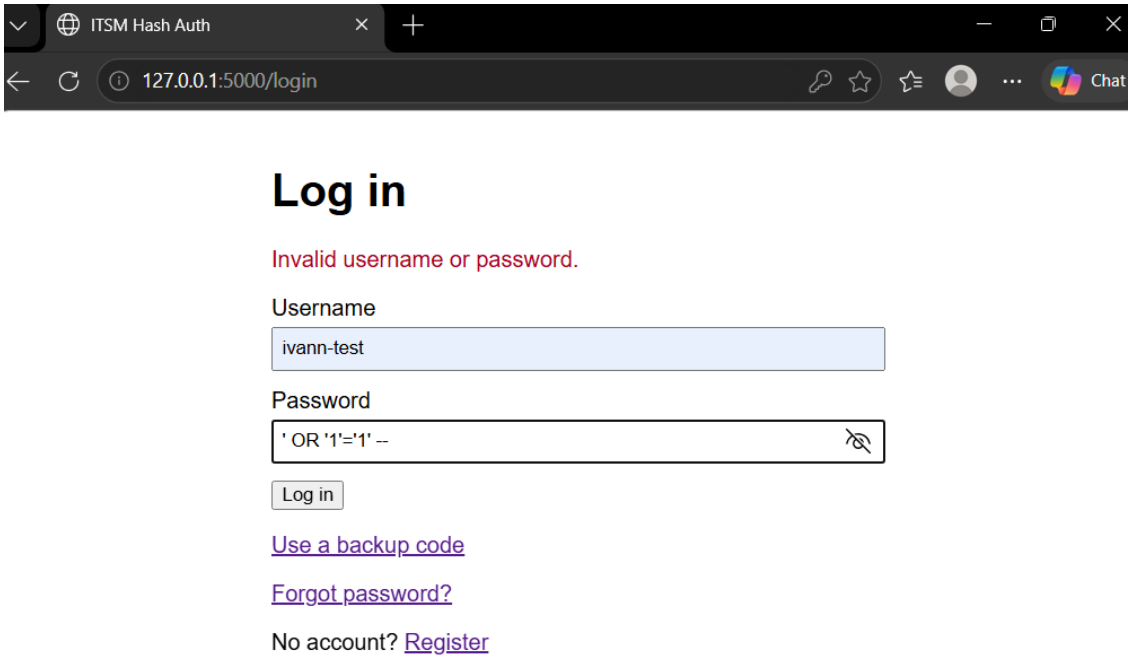
Expected: The injected string must not create an authenticated session.

Actual: The login page displayed “Invalid username or password.” The browser remained on /login rather than opening /profile.

Result: PASS

Interpretation: this payload did not get past the login, and no SQL error was shown.

Evidence



PT-10A: The SQL-injection-style password input was rejected with the generic login error.

Limitation: only one payload was tried, by hand, in one field. Saying more about SQL injection would need more inputs or a look at the code.

Overall conclusion

PT-06 to PT-10 all passed in the local setup. Old sessions did not survive a password change, the old password stopped working, the rate limit kicked in, the error messages did not give away usernames, and the SQL payload did not log in. These are targeted tests, not a full penetration test of the application.

A.3 Report 3: PT-11 to PT-15: CSRF, session replay, reset expiry, admin controls

ITSM Hash Auth, local Flask laboratory

Report details	**Value**
Prepared by	Ivann Aureus
Course	Information Technology Security Methods
Project	Hash (password + salt) with Argon2id
Date of testing	03 November 2026
System under test	Local application at http://127.0.0.1:5000
Assessment scope	Authorized local test accounts and test database
Evidence handling	Screenshots selectively redacted for passwords, cookies and reset tokens

Assessment statement

This report covers tests PT11 to PT15. PASS means the test passed under the conditions written here, nothing more. All tests ran in my local Windows 11 VirtualBox lab.

Screenshots are redacted copies of the originals.

Executive summary

Five tests covered session handling, CSRF protection, password recovery and the admin controls. All five passed. During PT15 a second Force Reset event showed up in the audit log; I looked into it and repeated the recovery cleanly.

ID	**Security control tested**	**Observed outcome**	**Assessment**
PT11	CSRF: POST /logout without token	HTTP 400 Bad Request; request denied	PASS
PT12	Session replay after logout	POST /logout 302; old cookie sent; /profile redirected to /login	PASS
PT13	Expired password-reset token	expires_at set to 0; reset URL rejected; prior password still worked	PASS
PT14	Admin password reauthentication	Empty/wrong password denied; correct password authorized revocation	PASS
PT15	Force-reset enforcement & recovery	Old session revoked, old password blocked, clean recovery clears flag	PASS

Test environment and approach

- Application: ITSM Hash Auth; local Flask server over 127.0.0.1:5000, backed by SQLite app.db.
- Browser: Microsoft Edge on a Windows 11 VirtualBox guest; normal and InPrivate windows isolated administrator and target-user sessions.
- Test accounts: ivann-test, victim-test and admin-test (administrator created specifically for admin tests).
- Techniques: browser DevTools, direct protected-page requests, console fetch, controlled SQLite queries, and application audit records.
- All changes were made only in the local lab. Mistakes and retests are included, not hidden.

Evidence and interpretation standards

Expected is how the control should behave; Observed is what the screenshot, console or database actually showed.

Safety: authentication cookies, reset URLs, and displayed passwords were redacted from embedded screenshots. The unredacted originals should not be committed to a public repository.

PT11: Cross-Site Request Forgery protection

Item	Detail
Objective	Verify that a POST request to /logout without a CSRF token is rejected.
Attacker action	From the authenticated browser, send fetch("/logout", {method:"POST", credentials:"same-origin", redirect:"manual"}) without a CSRF field.
Expected	Server rejects missing-token request and does not process logout.
Observed	POST /logout returned HTTP 400 Bad Request; no redirect location. The authenticated profile was still visible in the capture.
Result	PASS: the request without a token was refused.

Procedure and evidence

The first console request appeared to return HTTP 405 because the browser followed a redirect to a GET on the POST-only logout endpoint. The test was corrected with redirect: "manual" to reveal the POST response directly. The decisive result was HTTP 400 for the tokenless POST. The GET 405 is not itself evidence of CSRF protection.

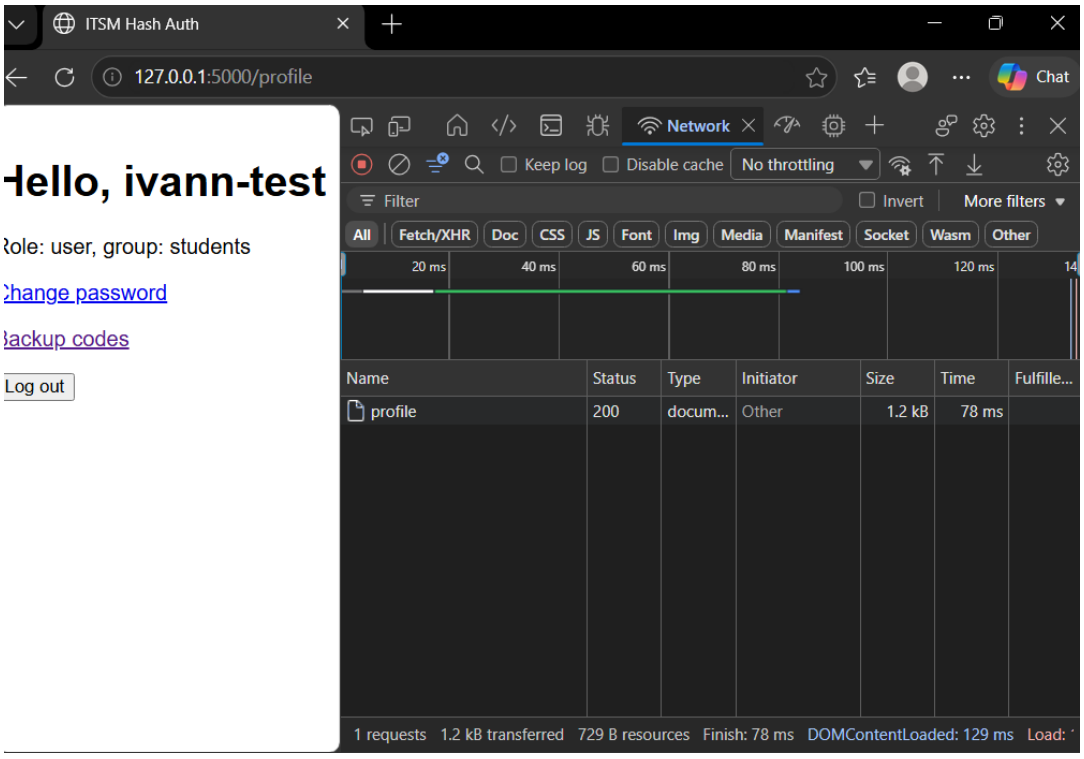


Figure PT11-01: Baseline: logged-in profile and HTTP 200.

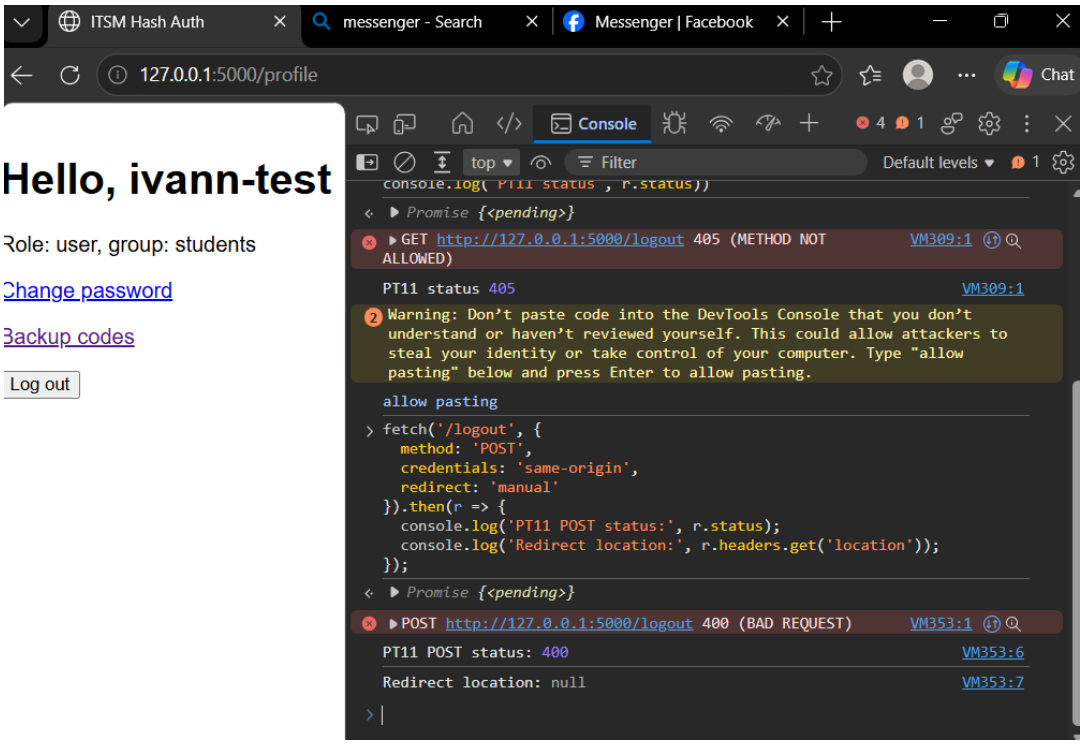


Figure PT11-02: CSRF-free POST /logout rejected with HTTP 400.

Limitation: there is no screenshot after refreshing the page. The test shows that this one POST without a token is refused, not that every form is protected.

PT12: Logout session replay

Item	Detail
Objective	Determine whether a previously valid authentication cookie still authorizes /profile after logout.
Setup	Authenticated ivann-test in Edge; saved the issued __Host-sid value locally.
Action	Successful normal logout; manually restored old cookie; requested /profile.
Expected	Server rejects the old session even if the browser sends its cookie.
Observed	POST /logout returned 302; cookie cleared; replayed __Host-sid appeared in the request headers; protected page redirected to /login.
Result	PASS: the old cookie no longer works.

Logout and cookie clearing

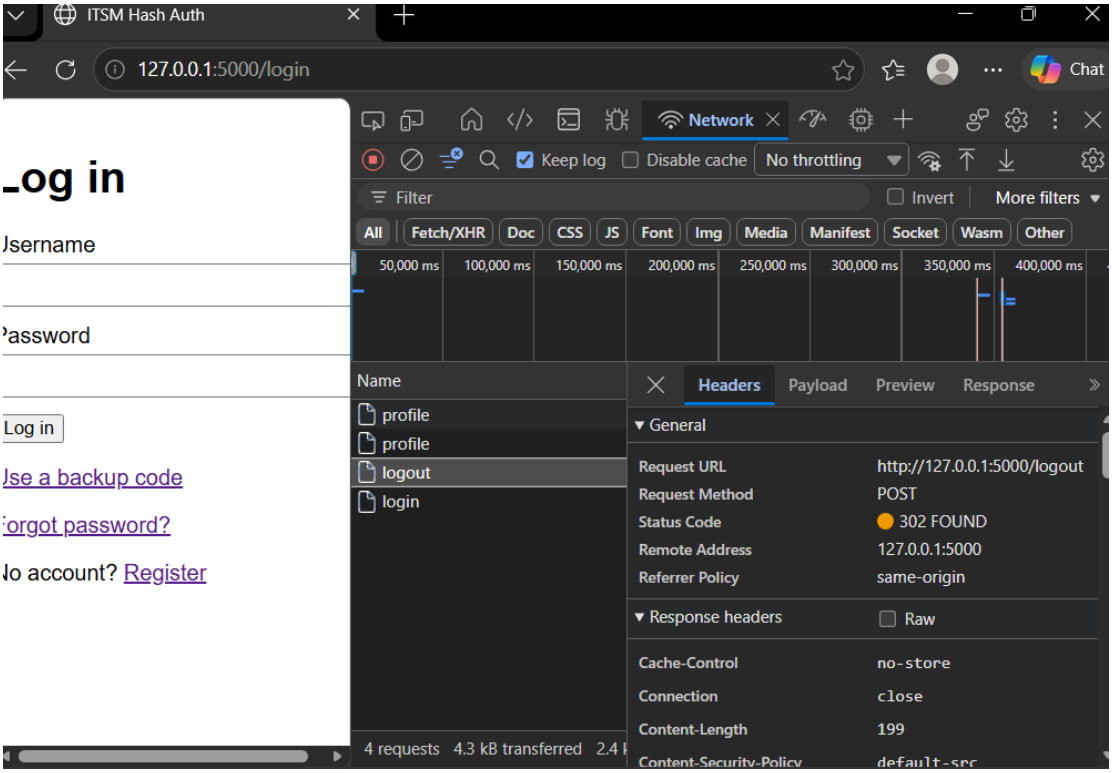


Figure PT12-01: Successful normal logout: POST /logout returns HTTP 302.

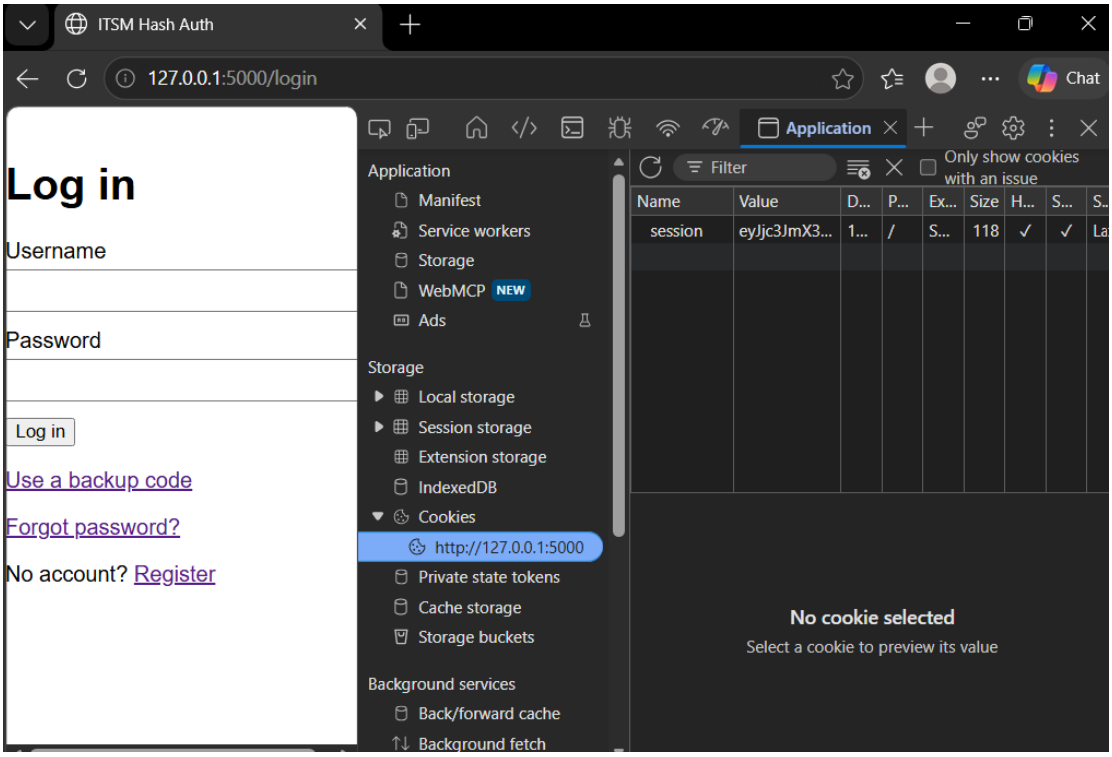


Figure PT12-02: After logout: __Host-sid removed from browser cookies.

Cookie restoration and replay result

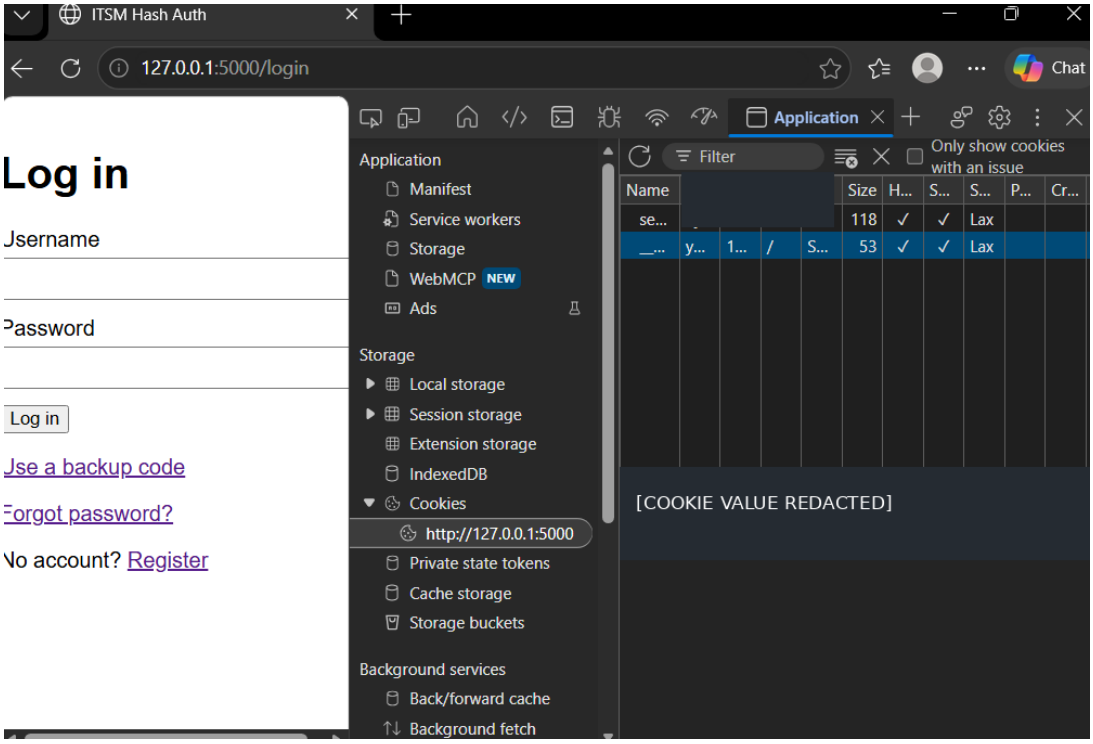


Figure PT12-03: Previously issued __Host-sid restored for replay test.

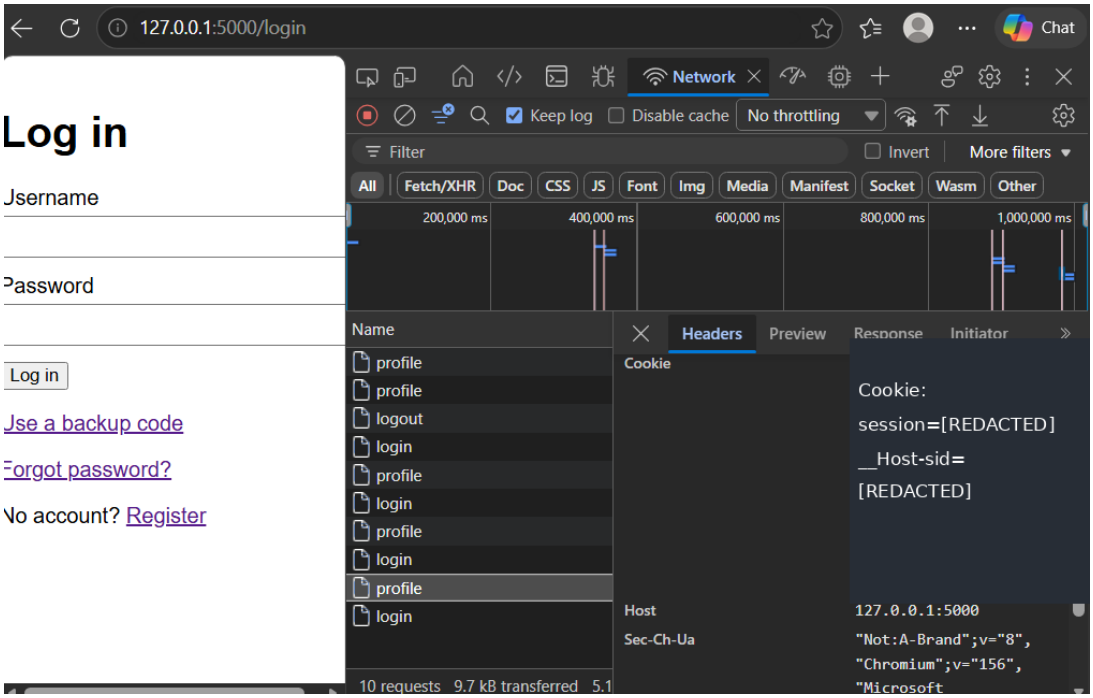


Figure PT12-04: Network request included __Host-sid (sensitive value removed).

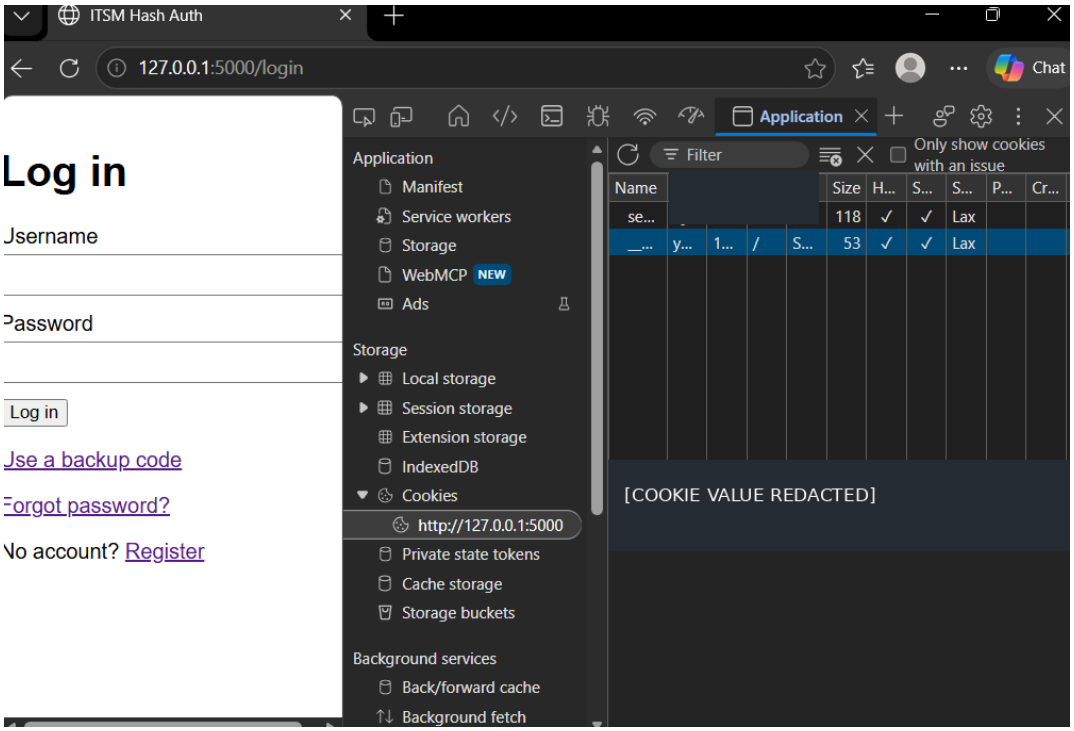


Figure PT12-05: Replay attempt ends at login instead of protected profile.

Interpretation: deleting the cookie in the browser proves nothing on its own. What matters is that the old cookie was sent again after logout and the server still refused it. An earlier attempt was lost when the VM crashed, so only the clean retest counts.

PT13: Expired password-reset token

Item	Detail
Objective	Check that a reset token with a past expiration cannot be used to change a password.
Setup	Generated a reset link for ivann-test; backed up app.db.
Attack simulation	Changed the test account's password_resets.expires_at to 0 in local SQLite.
Expected	Original reset link is invalid; password reset form is not usable; account retains its existing password.
Observed	One reset row updated; original link displayed "This reset link is invalid or has expired." Normal login with existing password succeeded.
Result	PASS: the expired link is refused.

Database change and rejection

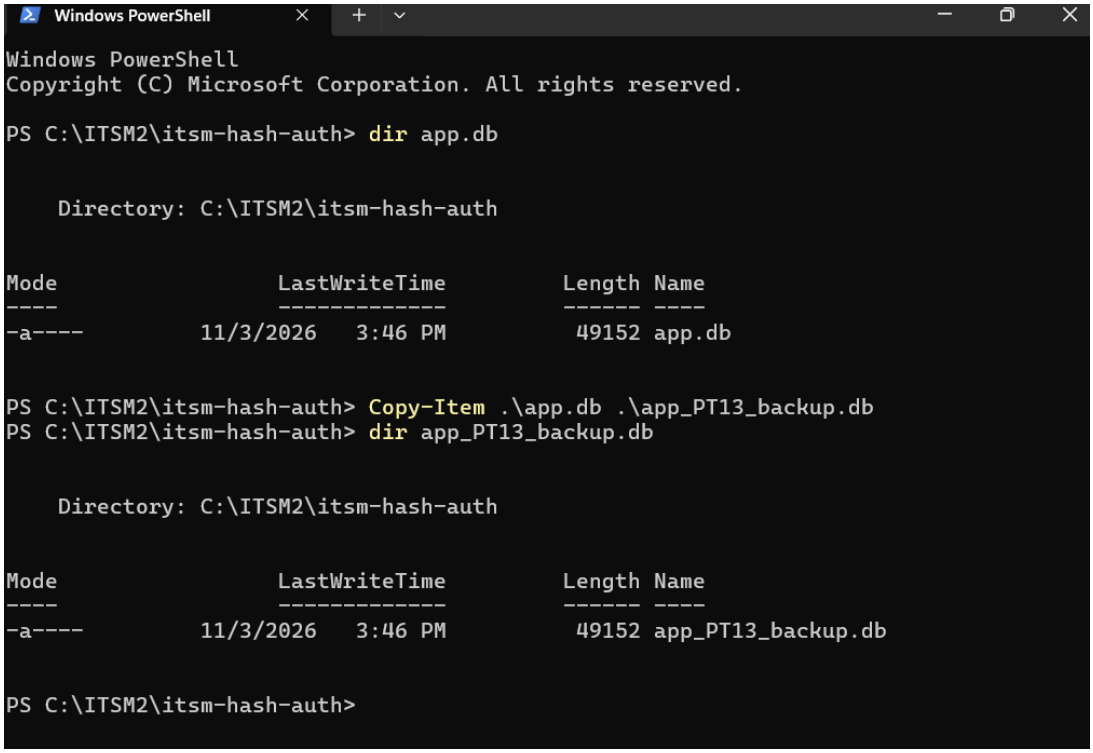


Figure PT13-01: SQLite backup file confirmed before token-expiry simulation.

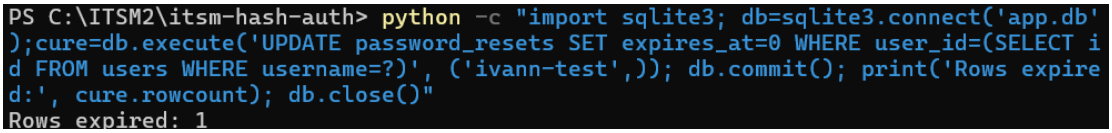


Figure PT13-02: SQL update affected one password-reset record.

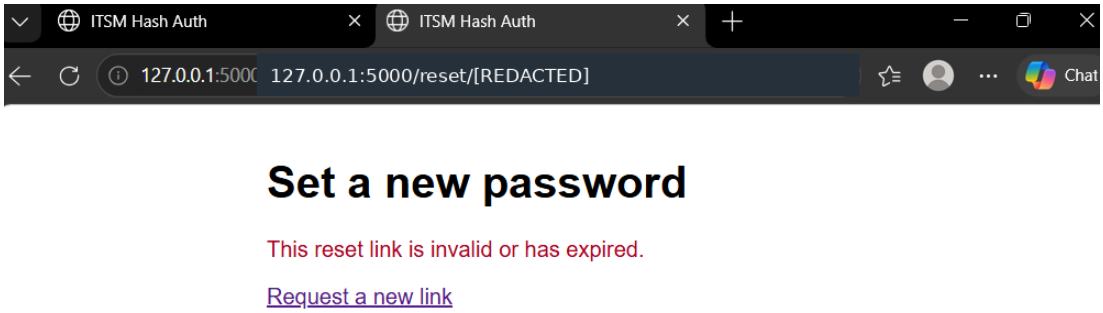


Figure PT13-03: Expired reset URL rejected with explicit expiration message.

Post-test account availability

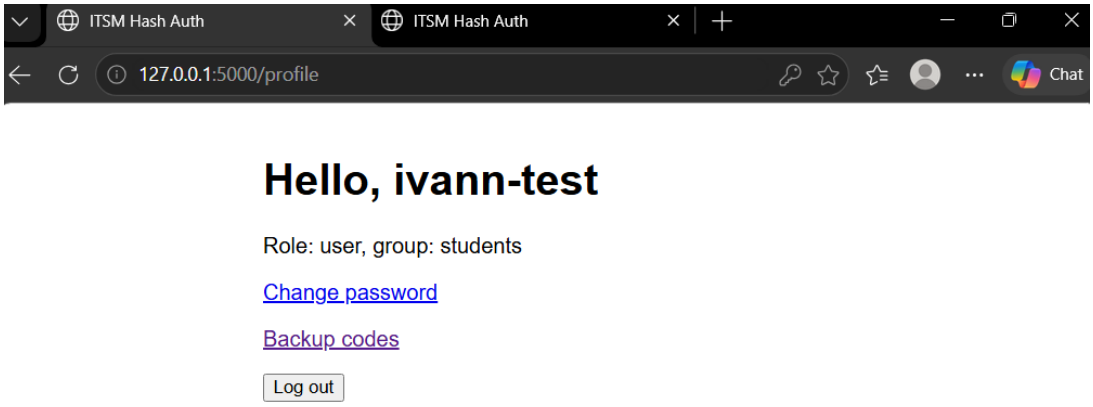


Figure PT13-04: Normal login confirms existing account remains usable.

The old password still worked afterwards, so the refused link did not change anything.

Limitation: the expiry was faked by editing the database instead of waiting 30 minutes.

PT14: Administrative reauthentication

Item	**Detail**
Objective	Verify that sensitive administrator actions require fresh confirmation of the admin password.
Setup	Registered admin-test, promoted it using make_admin.py, and logged in as administrator.
Test action	Revoke sessions for admin-test (one existing session).
Expected	Missing and wrong admin passwords are rejected; correct password authorizes action and invalidates sessions.
Observed	Missing and wrong passwords returned “Your admin password is incorrect. No action taken.” Correct password authorized revocation; sessions decreased from 1 to 0; subsequent /profile access redirected to /login.
Result	PASS: the server asks for the admin password.

Before attempting the action

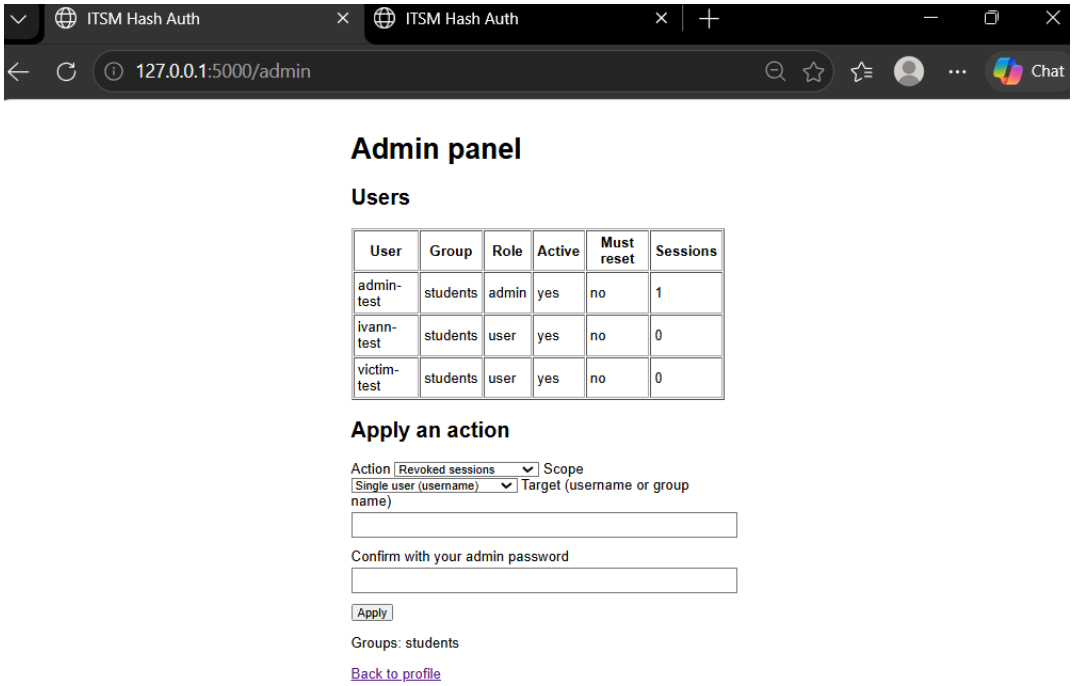


Figure PT14-01: Admin panel baseline: session count 1; password confirmation field.

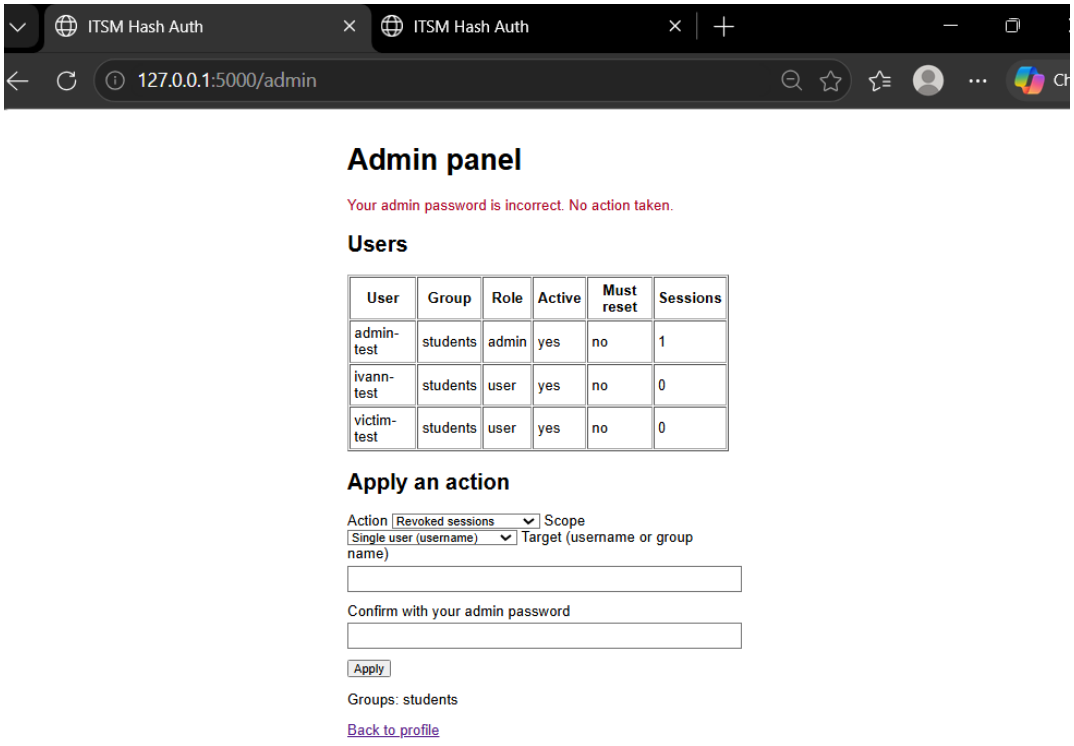


Figure PT14-02: Server rejects empty admin password; session remains 1.

Incorrect password and correct password comparison

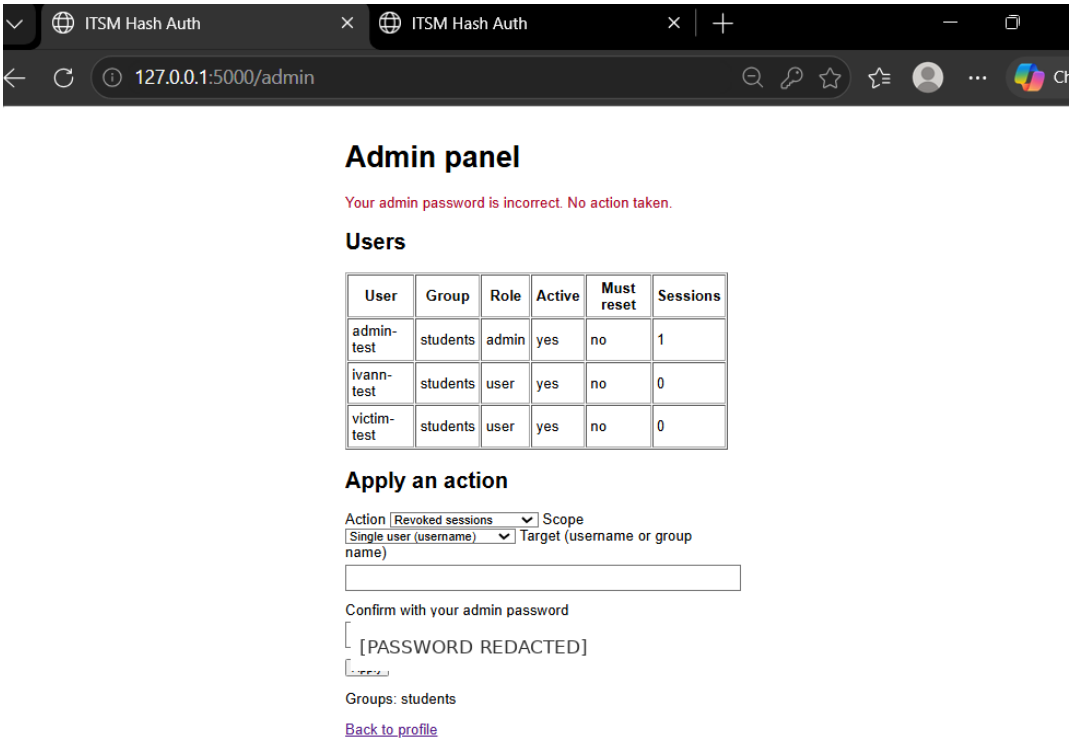


Figure PT14-03: Server rejects incorrect admin password; session remains 1.

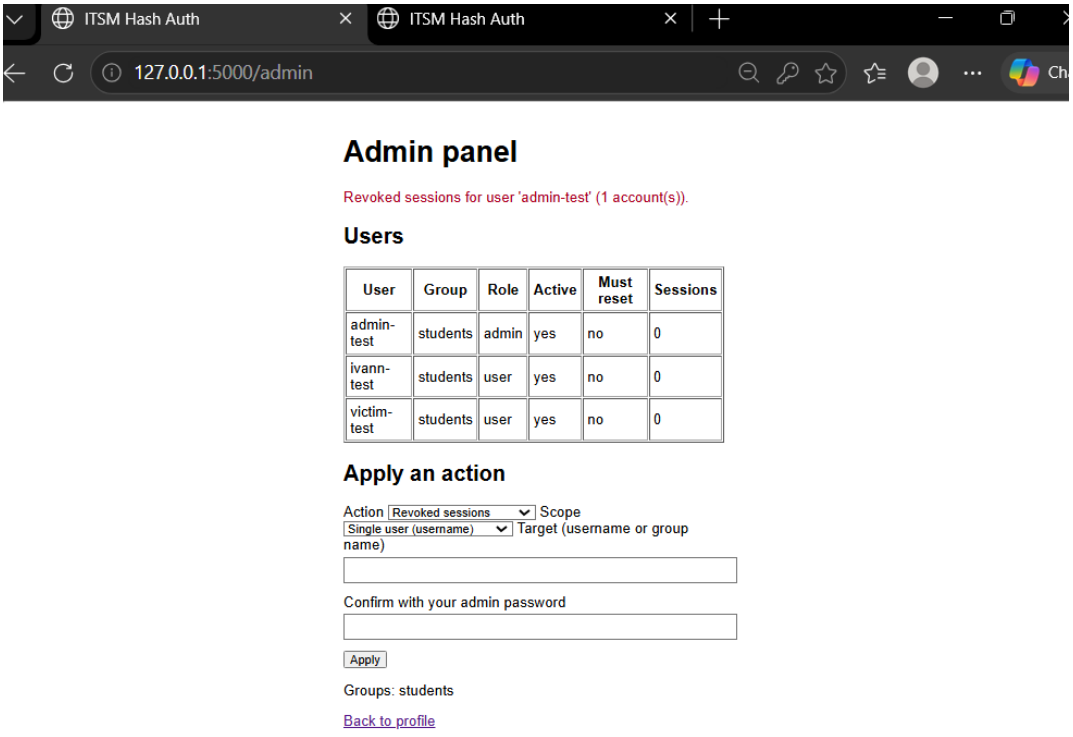


Figure PT14-04: Valid admin password: session revocation succeeds, count falls to 0.

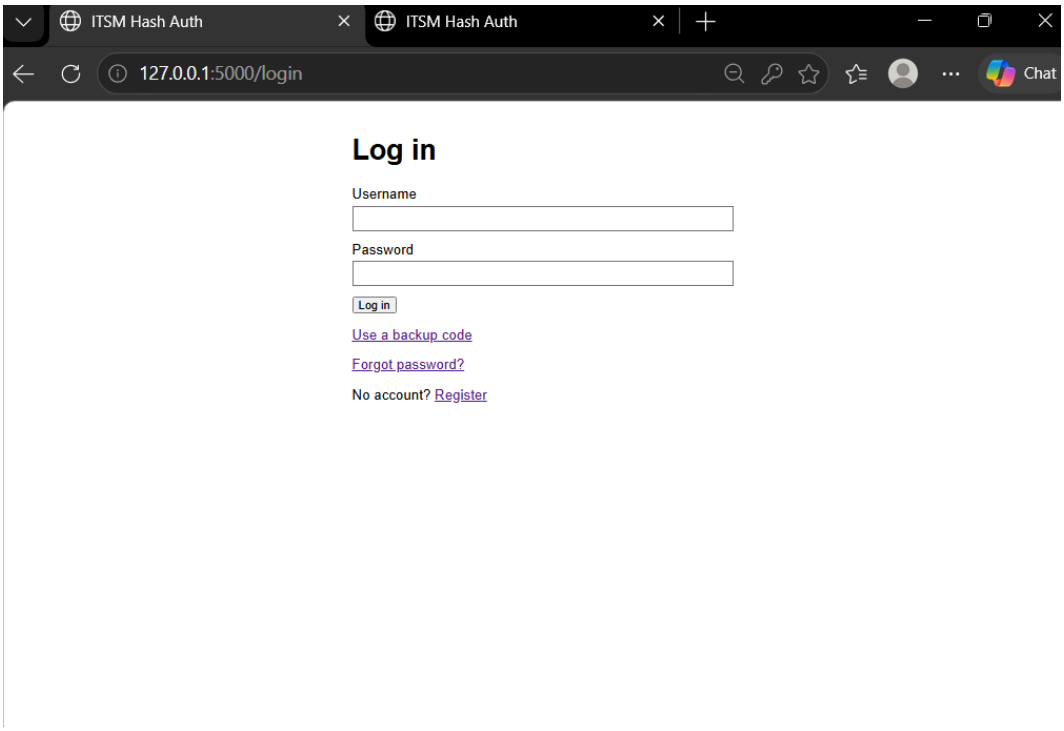


Figure PT14-05: Revoked administrator cannot access profile; browser at login.

The first try with an empty field was stopped by the browser's required-field check, so I removed that check to see what the server does. The PASS is based on the server's answer, not the browser's.

PT15: Forced password reset and recovery

Item	**Detail**
Objective	Confirm that a forced reset revokes sessions, blocks old-password login, and permits safe recovery.
Setup	victim-test logged in from InPrivate while admin-test was active in normal Edge.
Expected	Force Reset sets must_reset; removes sessions; old password cannot bypass; recovery clears must_reset and restores access.
Observed	Admin panel: must_reset no→yes and sessions 1→0; old password rejected with forced-reset notice. Later a clean recovery changed must_reset to 0, and victim-test logged in successfully.
Result	PASS: old password blocked, recovery works.

Forced reset enforcement

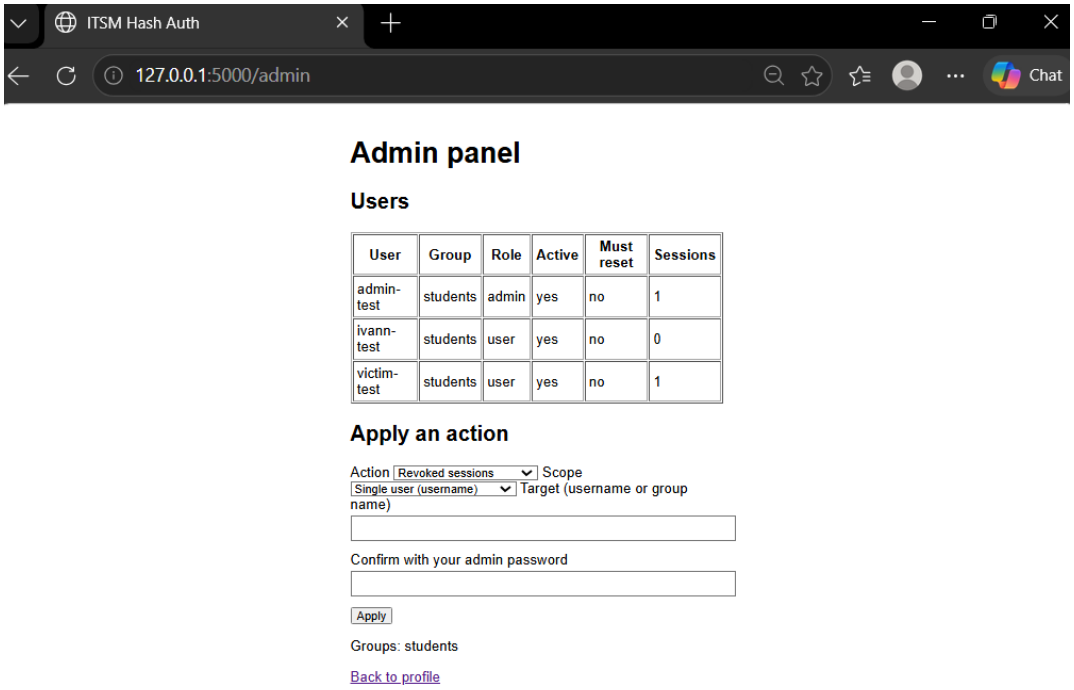


Figure PT15-01: Before admin action: victim-test requires no reset and has 1 session.

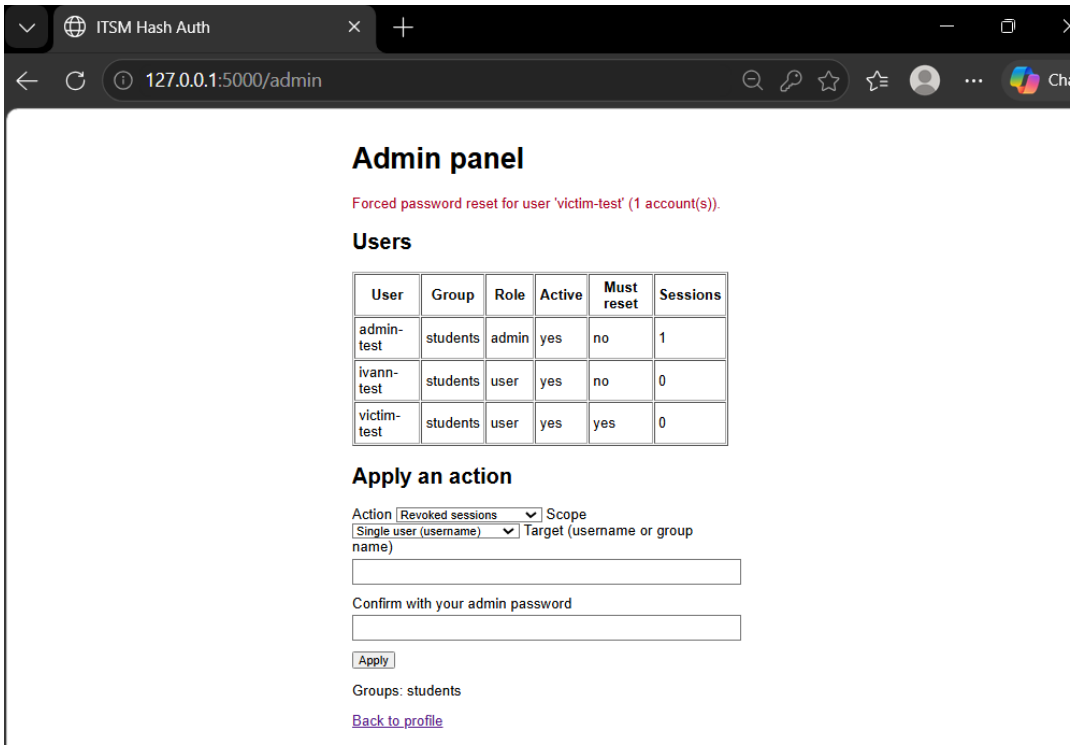


Figure PT15-02: After forced reset: must_reset=yes and victim sessions=0.

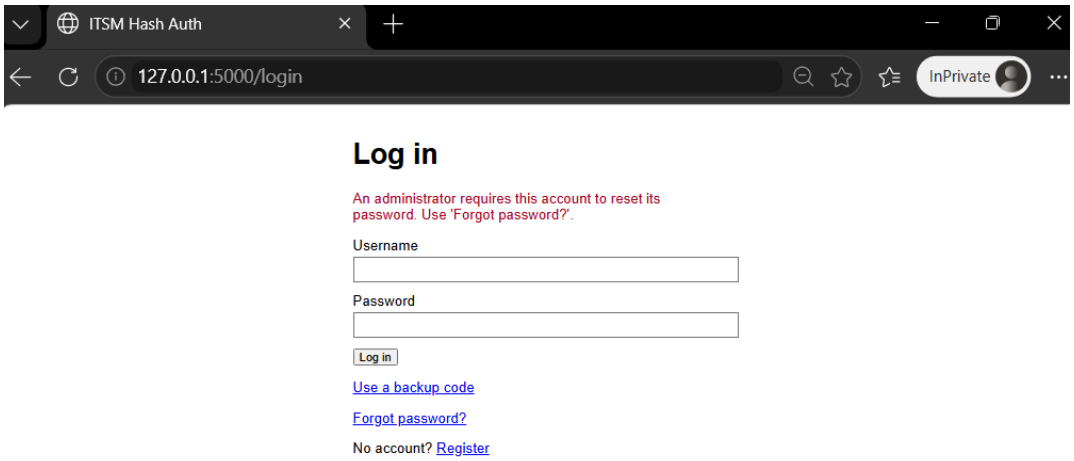


Figure PT15-03: Original password blocked; application demands password reset.

Unexpected repeat action and investigation

During the initial recovery check, the administrator view unexpectedly displayed must_reset=yes and 0 sessions after the user had apparently recovered. A direct SQLite query confirmed (user_id=2, username=victim-test, must_reset=1) and zero session records. The audit log then showed TWO force_reset events targeting victim-test, so the account had in fact been subjected to a repeated administrative reset.

```
PS C:\ITSM2\itsm-hash-auth> python -c "import sqlite3; db=sqlite3.connect('app.db'
); print(db.execute('SELECT * FROM audit_actions ORDER BY rowid DESC LIMIT 8').fet
chall()); db.close()"
[(3, 'admin-test', 'force_reset', 'user:victim-test', '2026-11-03 14:04:40'), (2,
'admin-test', 'force_reset', 'user:victim-test', '2026-11-03 13:57:38'), (1, 'admi
n-test', 'revoke_sessions', 'user:admin-test', '2026-11-03 13:50:43')]
PS C:\ITSM2\itsm-hash-auth>
```

Figure PT15-04: Audit confirms two distinct force_reset actions recorded.

Audit event times: 13:57:38 and 14:04:40. The browser showed a "Confirm Form Resubmission" warning, so the second event was probably the admin form being sent again, but the log cannot show which click did it.

Clean recovery retest

A fresh reset URL was requested and used without further administrator actions. After the password change confirmation, the database was queried immediately, before logging in or revisiting the admin panel.

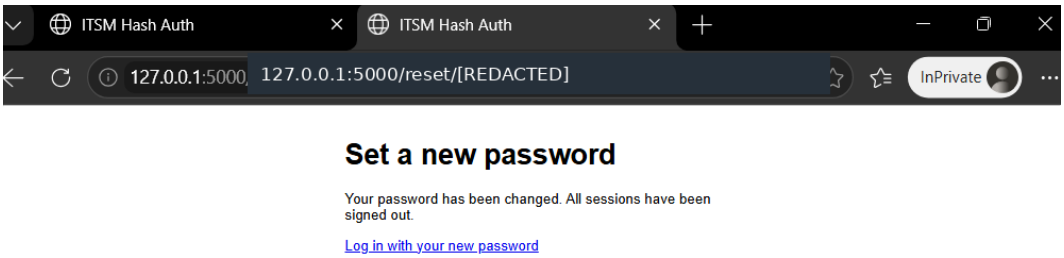


Figure PT15-05: Clean recovery retest: password change confirmed.

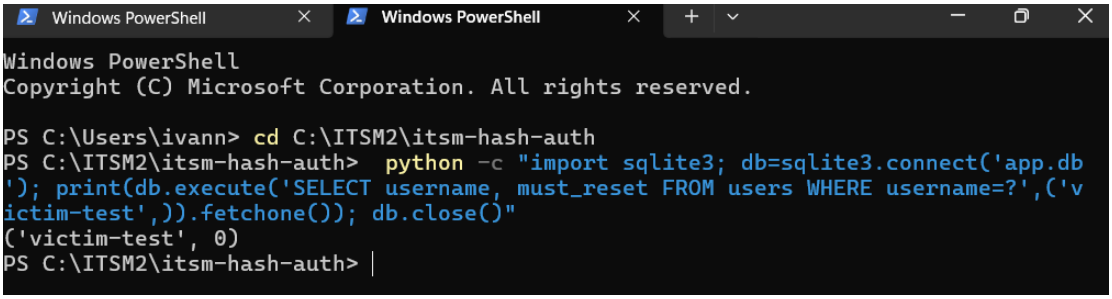


Figure PT15-06: Immediate SQLite query confirms must_reset=0.

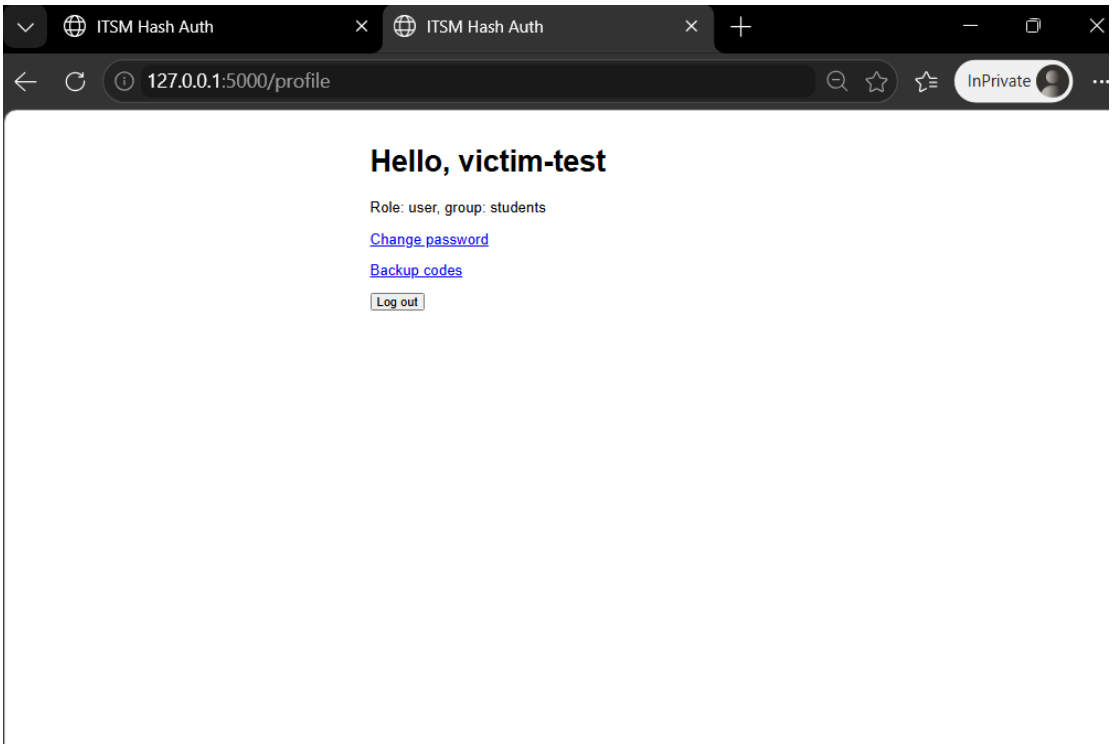


Figure PT15-07: Clean retest: victim-test logged in successfully.

The clean retest returned (victim-test, 0), so the reset flag was cleared, and victim-test could log in with the new password. Recovery works as long as no second admin reset gets in the way.

Finding: recovery cannot be bypassed and the flag does not get stuck. The possible double submission of the admin form is worth a separate note.

Conclusions and reproducibility

PT11 to PT15 all passed in the lab. The key results: a POST without a CSRF token got HTTP 400, an old session cookie was refused after logout, an expired reset link was refused, admin actions needed the right password, and a forced reset blocked the old password until a clean recovery.

Security observations and limitations

- These tests cover selected pages and steps, not the whole application.
- The reset expiry was faked in the database, and no real email was sent.
- Where the browser followed a redirect, I checked the status of the POST itself, not of the GET after it.
- The double admin action in PT15 is confirmed by two audit events, but what triggered it is only a guess.
- Passwords, reset tokens and cookies must stay redacted in anything published. Change the lab passwords before a public demo.

Evidence manifest

Evidence ID	**Screenshot subject (redacted copy)**
PT11-01	Baseline: logged-in profile and HTTP 200
PT11-02	CSRF-free POST /logout rejected with HTTP 400
PT12-01	Successful normal logout: POST /logout returns HTTP 302

PT12-02	After logout: __Host-sid removed from browser cookies
PT12-03	Previously issued __Host-sid restored for replay test
PT12-04	Network request included __Host-sid (sensitive value removed)
PT12-05	Replay attempt ends at login instead of protected profile
PT13-01	SQLite backup file confirmed before token-expiry simulation
PT13-02	SQL update affected one password-reset record
PT13-03	Expired reset URL rejected with explicit expiration message
PT13-04	Normal login confirms existing account remains usable
PT14-01	Admin panel baseline: session count 1; password confirmation field
PT14-02	Server rejects empty admin password; session remains 1
PT14-03	Server rejects incorrect admin password; session remains 1
PT14-04	Valid admin password: session revocation succeeds, count falls to 0
PT14-05	Revoked administrator cannot access profile; browser at login
PT15-01	Before admin action: victim-test requires no reset and has 1 session
PT15-02	After forced reset: must_reset=yes and victim sessions=0
PT15-03	Original password blocked; application demands password reset
PT15-04	Audit confirms two distinct force_reset actions recorded
PT15-05	Clean recovery retest: password change confirmed
PT15-06	Immediate SQLite query confirms must_reset=0
PT15-07	Clean retest: victim-test logged in successfully

A.4 Report 4: PT-16 to PT-20: disabled accounts, group revocation, cookies, errors, storage

Test range: PT16–PT20; Tester: Ivann Aureus; Application: ITSM Hash Auth (Flask / SQLite / Argon2id)

Environment: Local Windows 11 VirtualBox laboratory; Edge/Chrome browsers; application at 127.0.0.1:5000. Test evidence captured 3 November 2026.

Scope and evidence standards

This report covers five tests on the local application. PASS means the test did what was expected; it does not cover every related setup or attack. Passwords, session cookies and reset tokens are removed or blurred in the screenshots.

Test	Security objective	Observed outcome	Result
PT16	Disabled-account enforcement	Existing session invalidated; subsequent login rejected	PASS
PT17	Group-wide revocation	Three users' sessions reduced from one each to zero; old sessions denied	PASS
PT18	Session cookie security	Secure, HttpOnly, SameSite=Lax, Path=/, no Domain, JS-inaccessible	PASS
PT19	Error information disclosure	Generic 404/405/400 responses without observed sensitive details	PASS (tested cases)
PT20	Password hash storage	Argon2id v19, m=65536,t=3,p=4; three distinct salts	PASS

PT16: Disabled account and active session

Objective: Determine whether an administrator disabling an account terminates existing sessions and prevents subsequent login.

Setup: admin-test acted from a normal Edge window; victim-test had a separate authenticated session in Edge InPrivate. Before the action, victim-test was active (yes), did not require a password reset (no), and had one active session.

Test phase	Expected	Actual
Baseline	Active=yes; sessions=1	Confirmed on admin panel

Admin disables victim-test	Active=no; sessions=0	Admin panel displayed disabled confirmation; no/0 observed
Victim revisits /profile	Redirect to /login	Redirected to /login
Victim tries login	Login refused	Generic “Invalid username or password.” message

Result: PASS. Disabling the account ended its open session and blocked the next login, and the error message did not say the account was disabled.

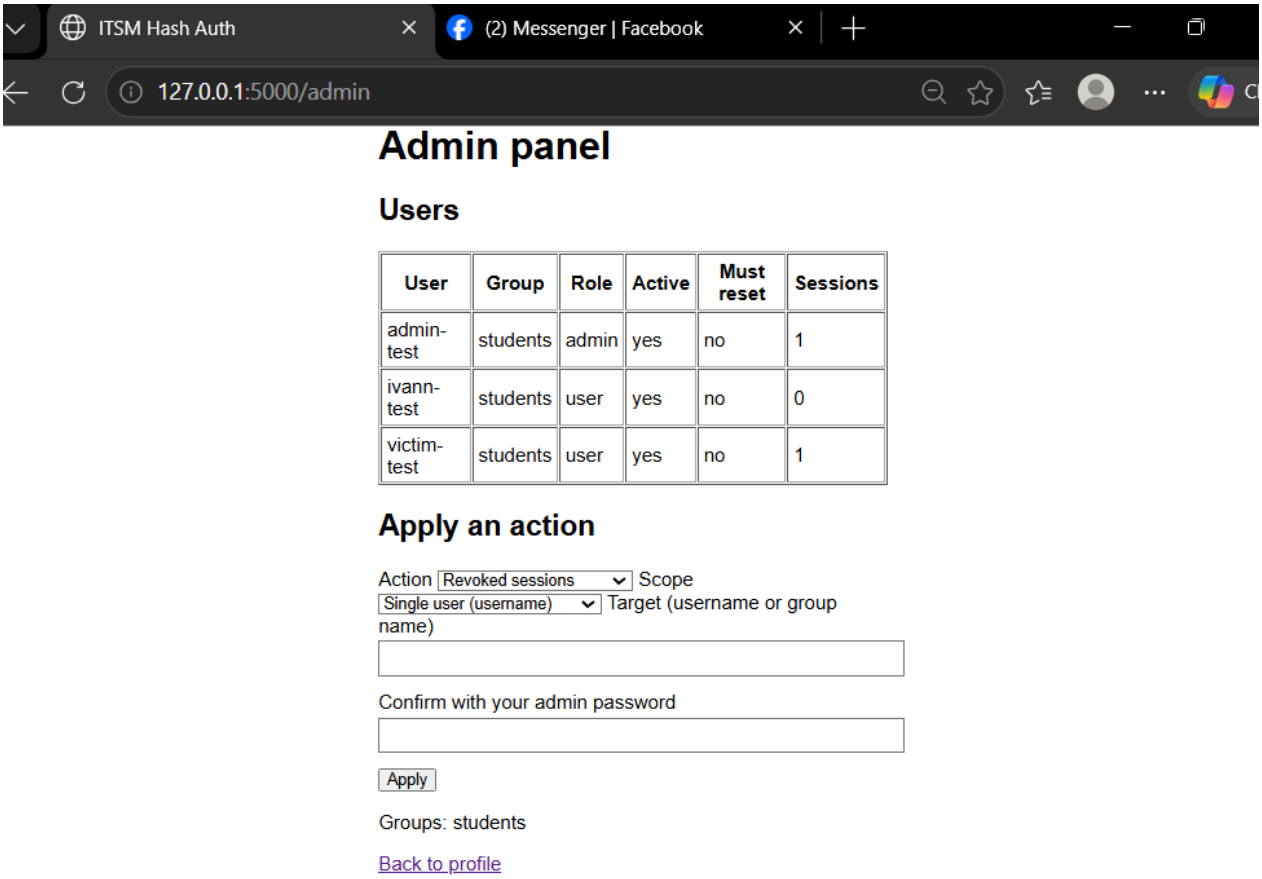


Figure PT16-01. Admin baseline: victim-test active with one session.

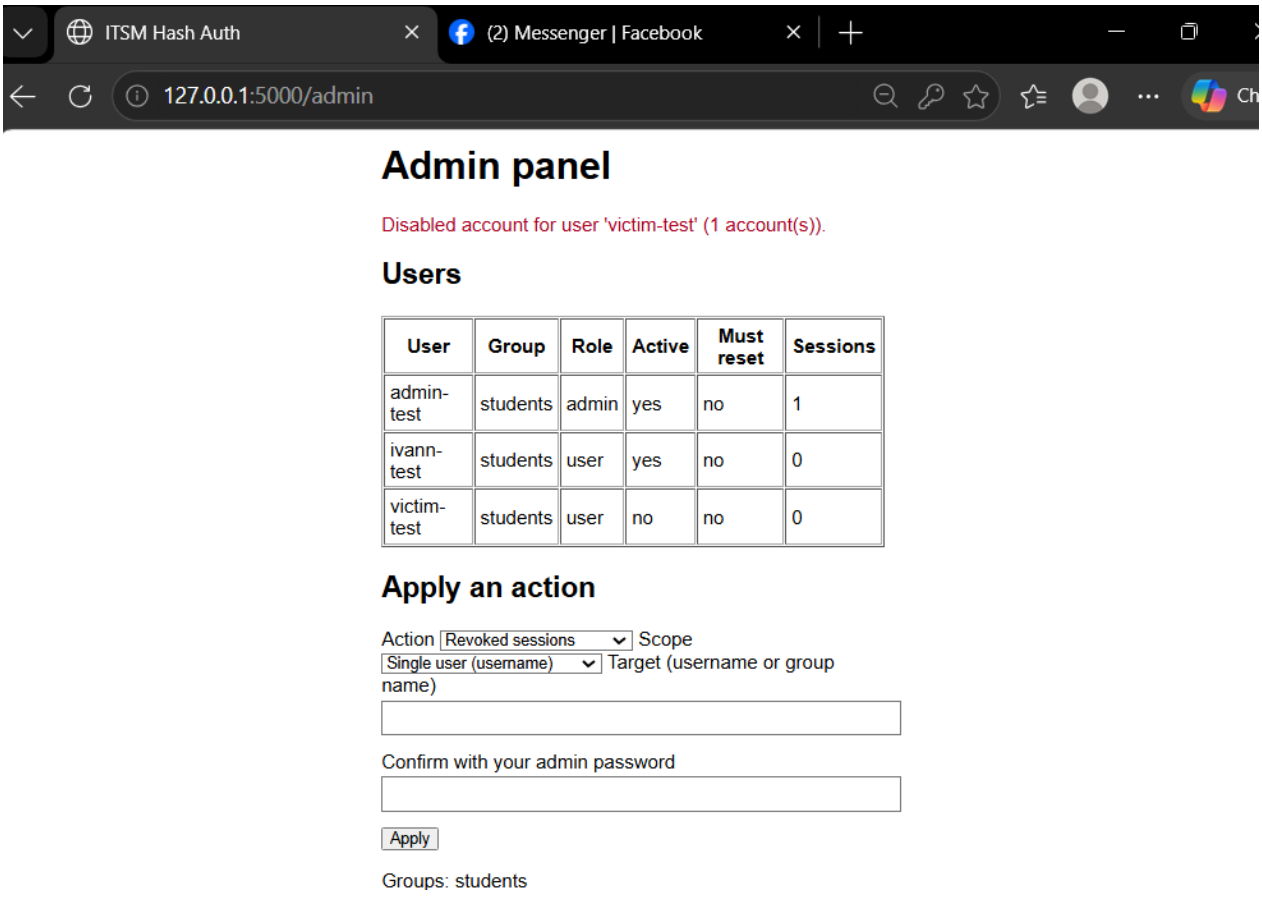


Figure PT16-02. Successful account disable: active no, sessions zero.

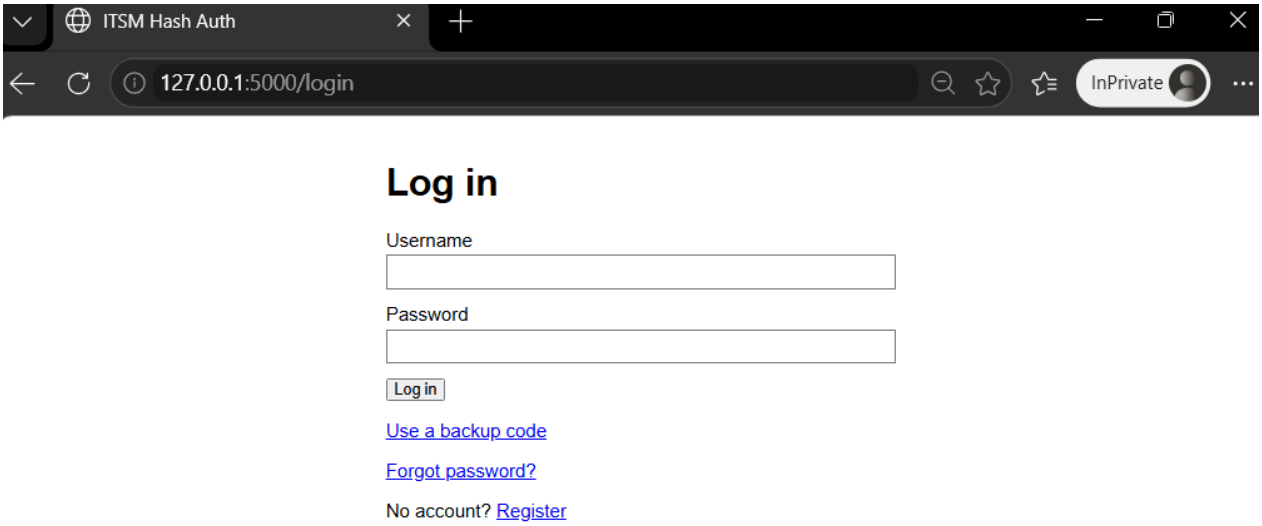


Figure PT16-03. Previously authenticated victim session redirected to login.

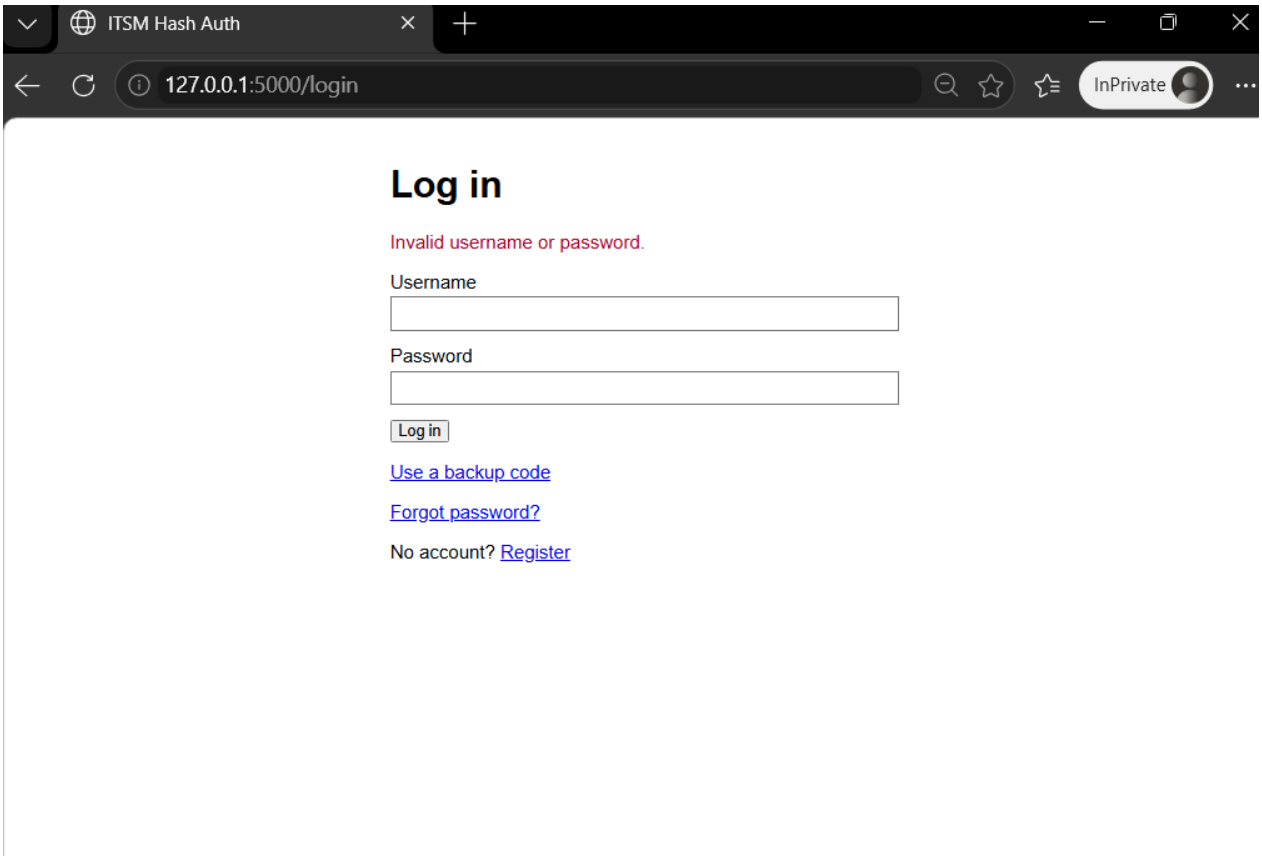


Figure PT16-04. Disabled-account login rejected with generic error.

PT17: Group-wide session revocation

Objective: Verify that an administrator can centrally revoke sessions for every member of a selected group and that those session credentials stop granting access.

Setup: Three accounts (admin-test, ivann-test, victim-test) were members of the students group. Each had one active session, verified before revocation. The administrator performed Revoked sessions with scope Whole group and target students.

User	**Before**	**After**	**Protected profile after revocation**
admin-test	1	0	Redirected to /login
ivann-test	1	0	Redirected to /login
victim-test	1	0	Redirected to /login

Result: PASS. The action reported “Revoked sessions for group students (3 account(s))”; the administrative panel showed zero sessions for all three users. Requests from the previously authenticated browser sessions redirected to the login page.

Operational note: The administrator belonged to the targeted group, so the administrative session was also revoked. A browser “Resubmit the form?” prompt was canceled rather than replaying the POST request.

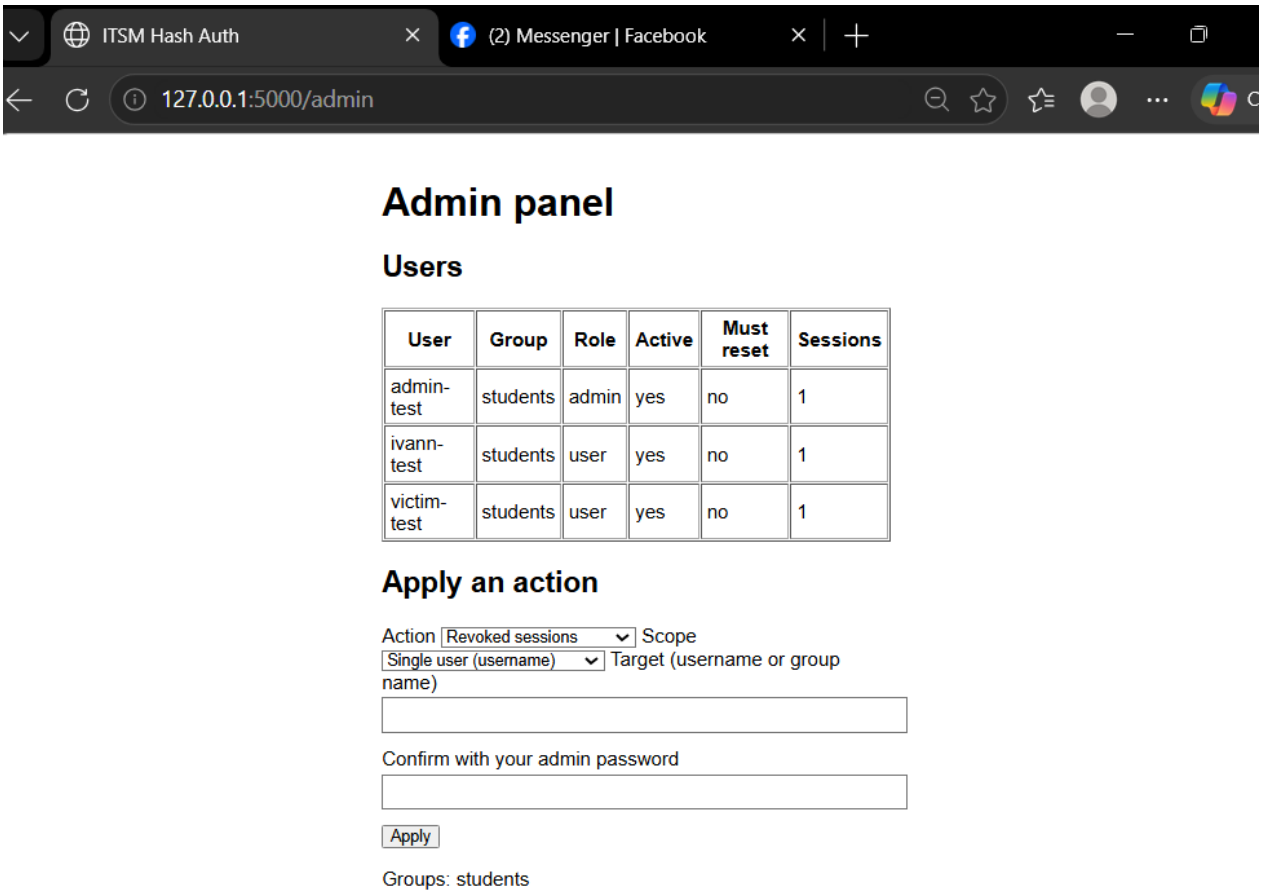


Figure PT17-01. Three students group members each have one session.

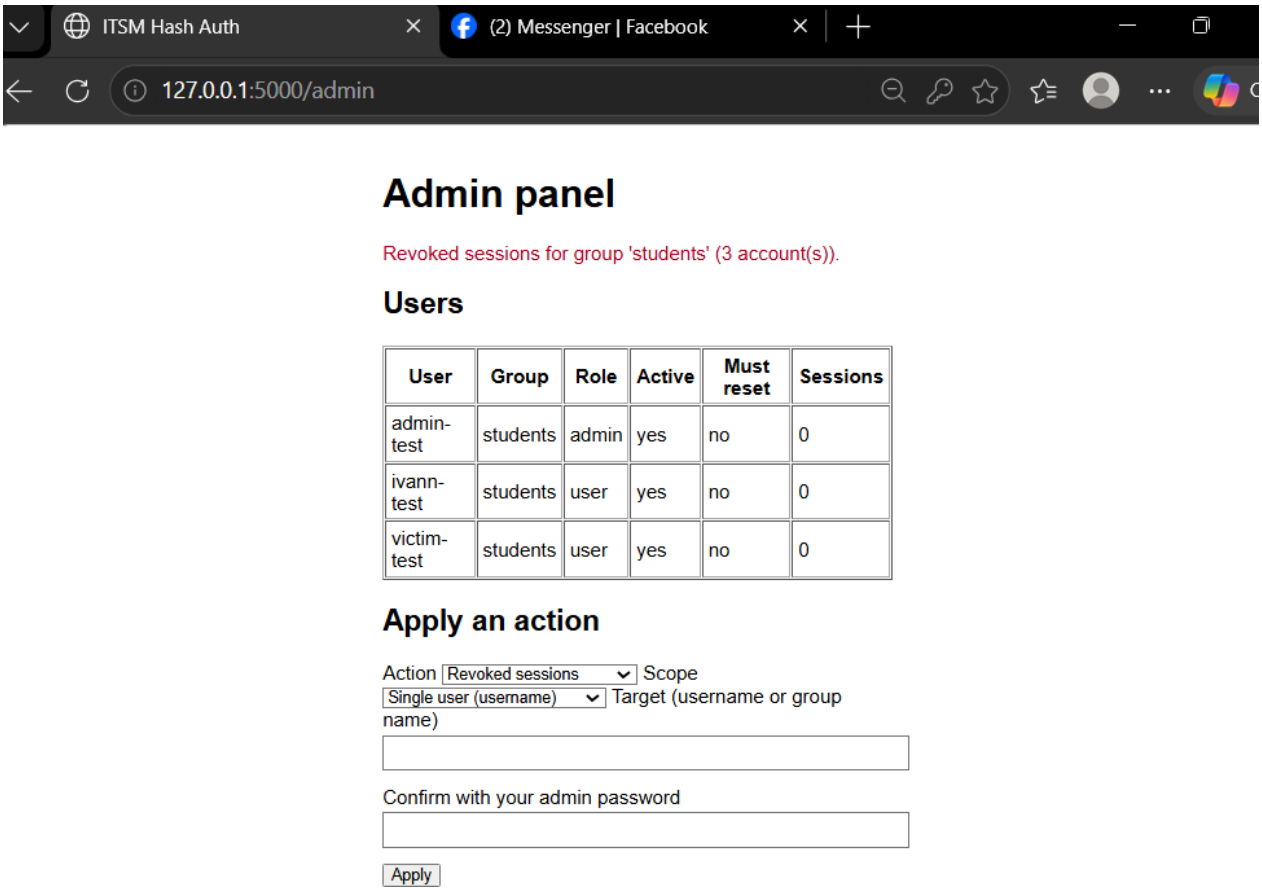


Figure PT17-02. Group session revocation: three accounts, zero sessions.

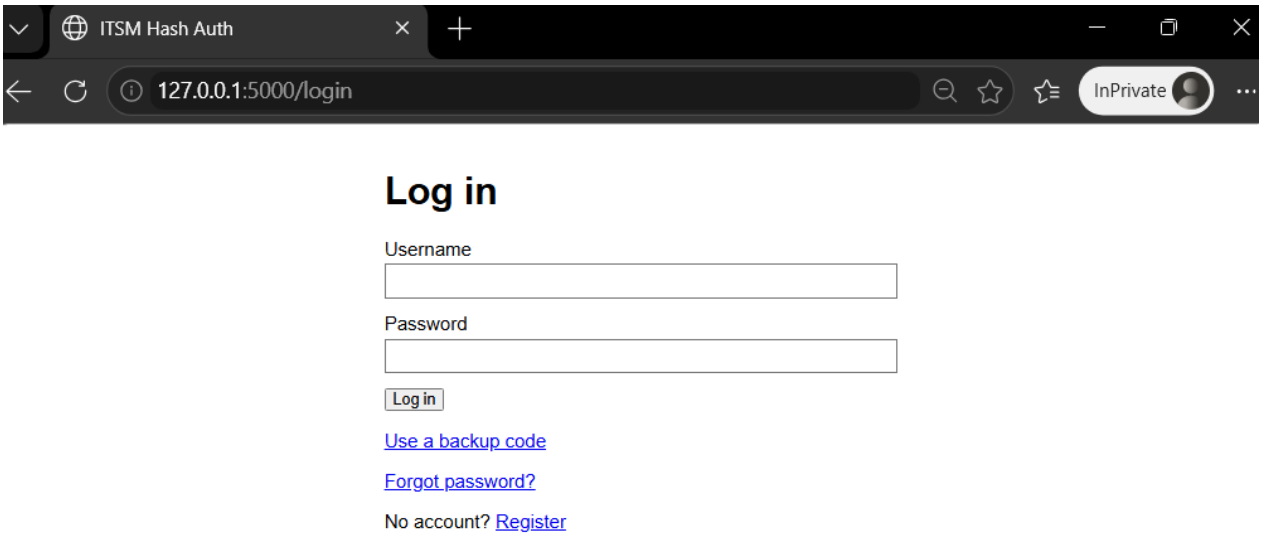


Figure PT17-03. Victim session denied after group revocation.

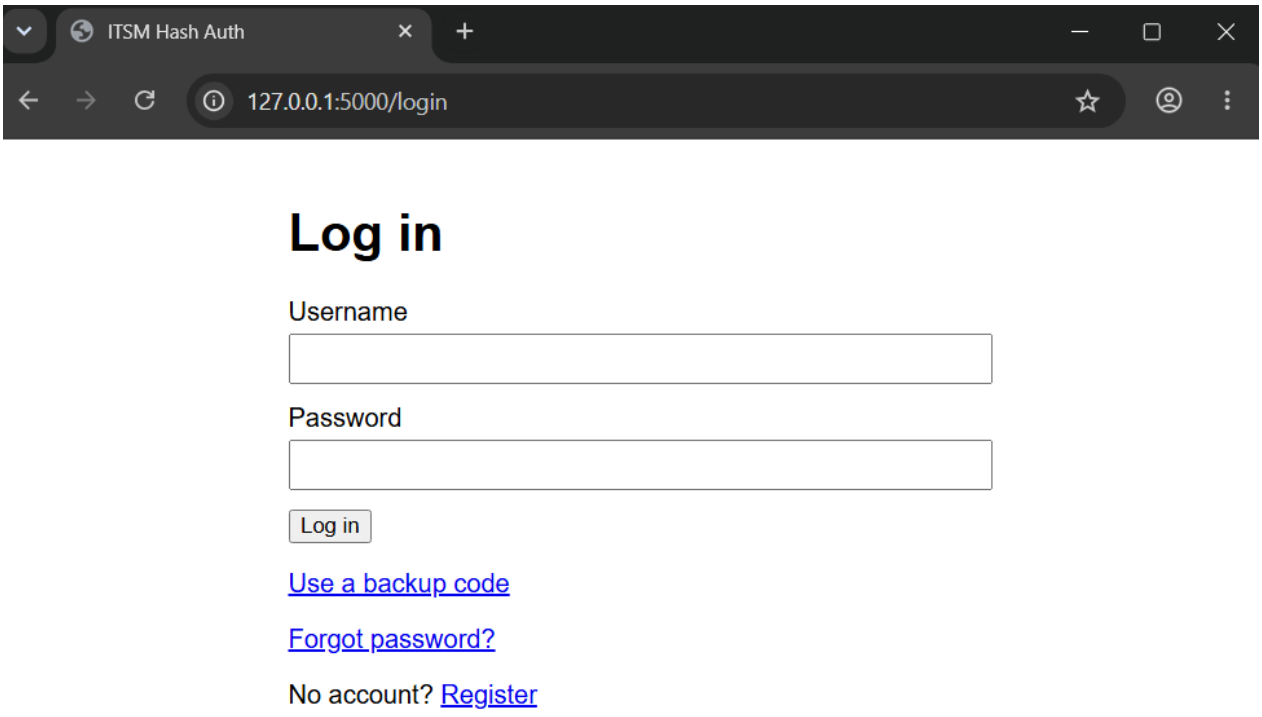


Figure PT17-04. Ivann session denied after group revocation.

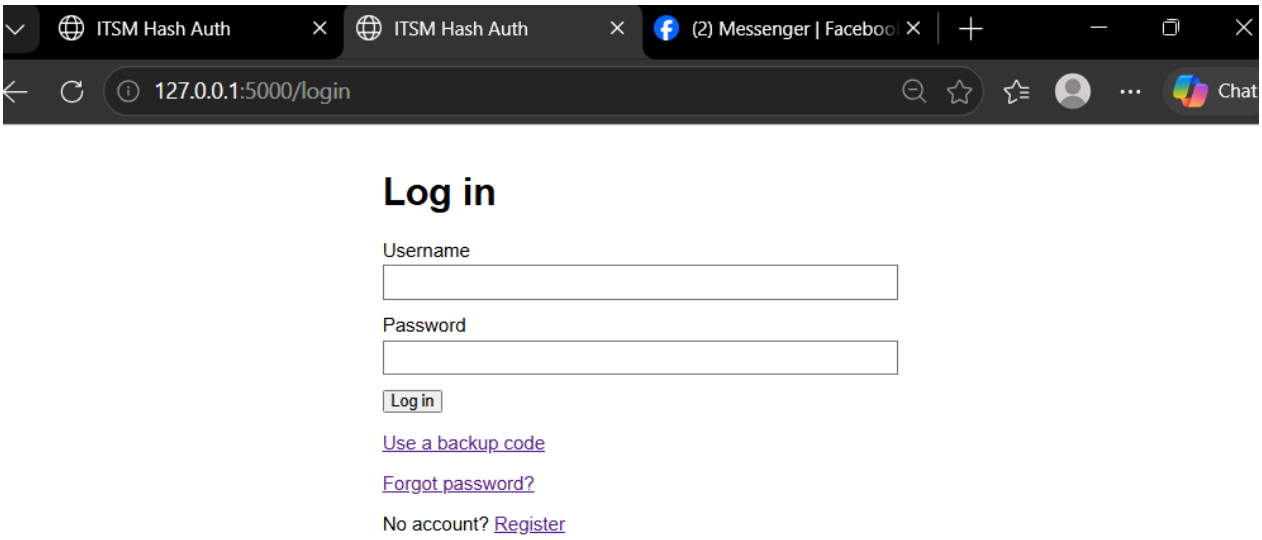


Figure PT17-05. Admin session denied after group revocation.

PT18: Session cookie security attributes

Objective: Verify protective attributes on the authentication cookie and check whether page JavaScript can read it.

Attribute	**Observed evidence**	**Assessment**
Name	__Host-sid	Matches expected naming convention
Secure	Present in Set-Cookie	PASS
HttpOnly	Present in Set-Cookie	PASS
Path	/	PASS
SameSite	Lax	PASS
Domain	No Domain attribute in Set-Cookie	PASS
JavaScript access	document.cookie returned empty string	PASS

Result: PASS for cookie configuration in the local browser. Edge DevTools showed the cookie attributes; inspecting the login response confirmed the Set-Cookie header had Secure; HttpOnly; Path=/; SameSite=Lax and no Domain attribute. document.cookie returned an empty string.

Limitations: this was plain HTTP on localhost, so it says nothing about a real HTTPS setup or about other browsers.

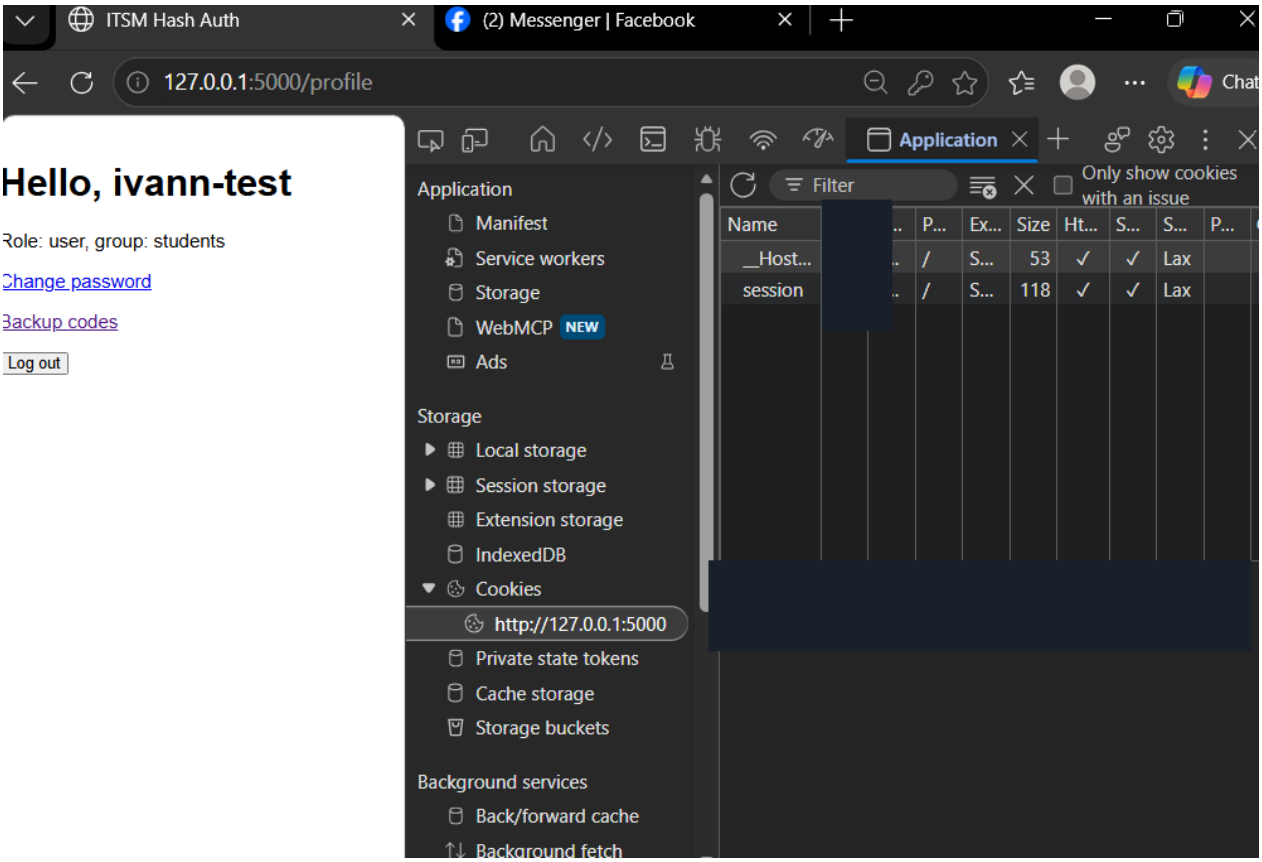


Figure PT18-01. Authentication cookie attributes in Edge DevTools.

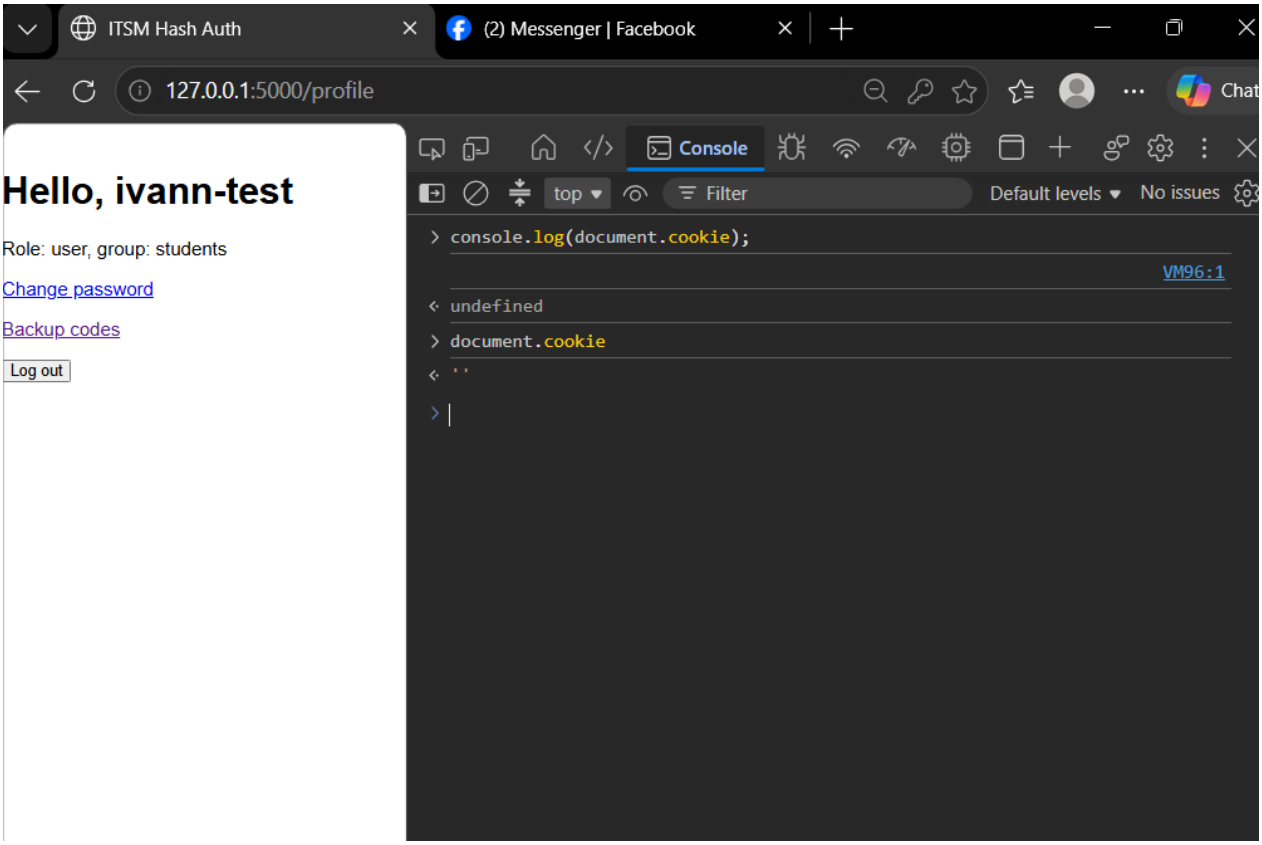


Figure PT18-02. document.cookie returned empty string.

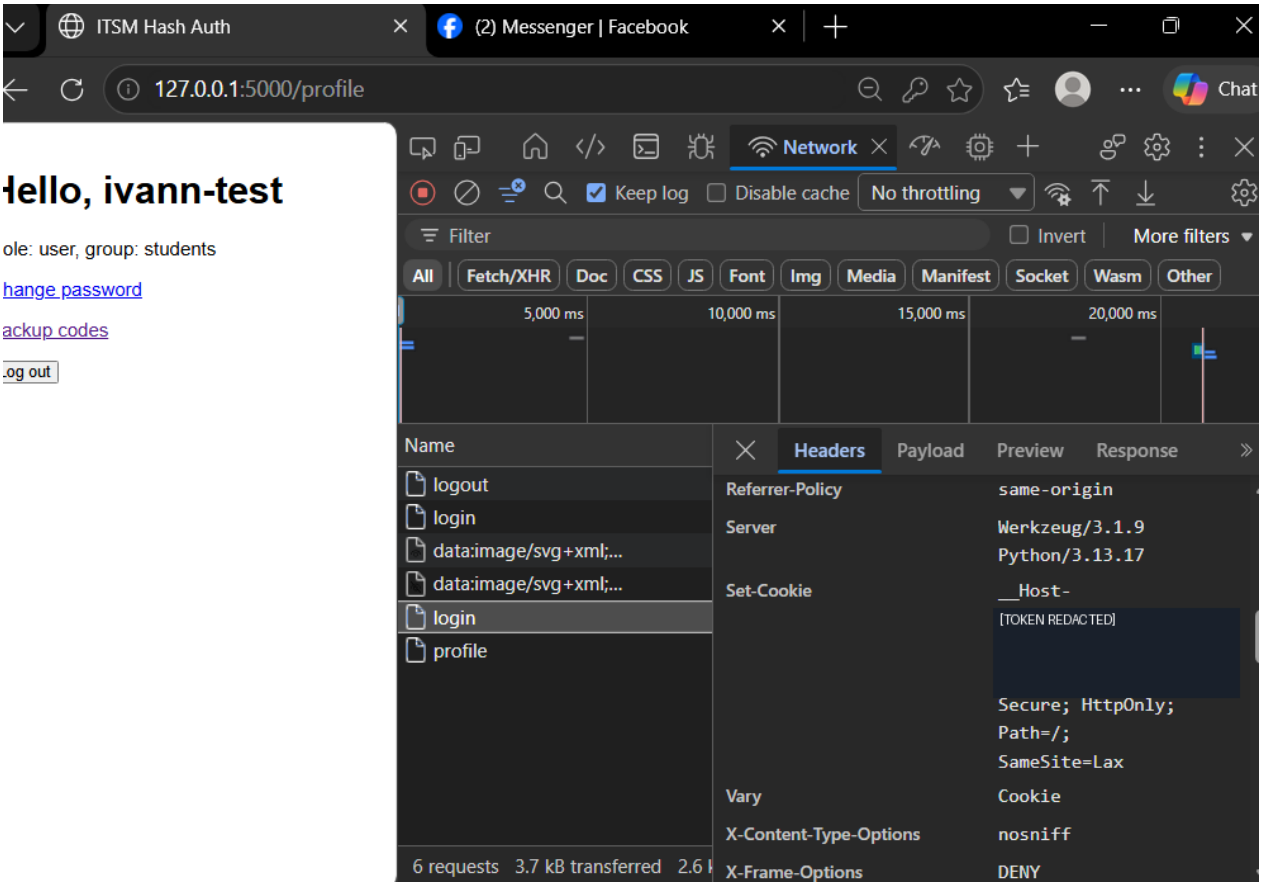


Figure PT18-03. Login Set-Cookie header and protective attributes.

PT19: Error information disclosure

Objective: Observe whether ordinary invalid routing and method/error conditions expose stack traces, filesystem paths, database details, secrets, or an interactive debugger.

Test case	**Observed response**	**Sensitive information observed?**
Nonexistent route	404 Not Found, generic message	No
GET /logout	405 Method Not Allowed, generic message	No
Malformed JSON POST /login, no CSRF token	400 Bad Request: CSRF token missing	No
PUT /login	405 Method Not Allowed, generic HTML	No

Result: PASS for the tested responses. No sensitive traceback, SQL error, file path, or interactive debug console was observed in the browser responses. In the two Console-based checks the automated keyword check also returned false.

Limitation: the malformed JSON request was stopped by the CSRF check before it reached the JSON parser, so this test says nothing about how the parser handles bad input. The error pages are the standard Flask/Werkzeug ones, not custom pages.

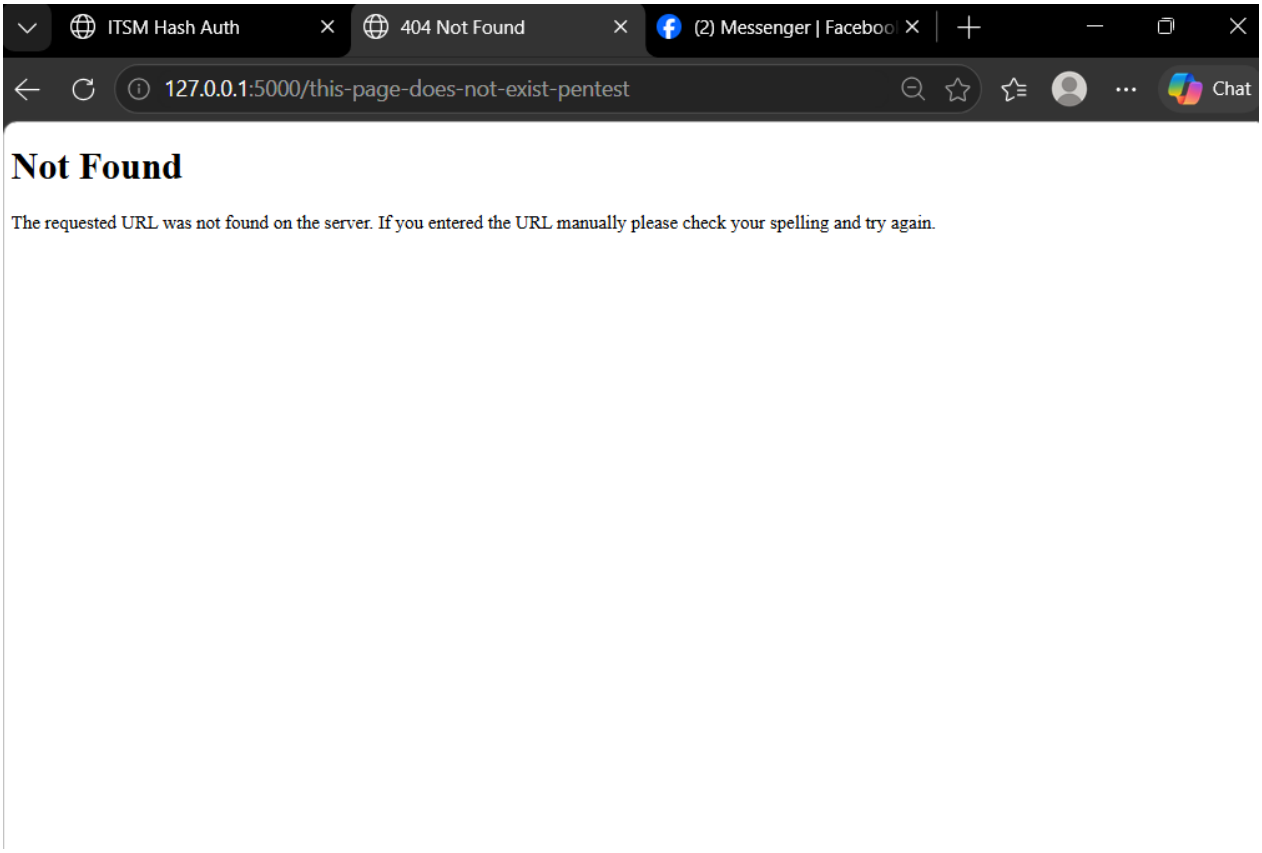


Figure PT19-01. Nonexistent route returns generic 404.

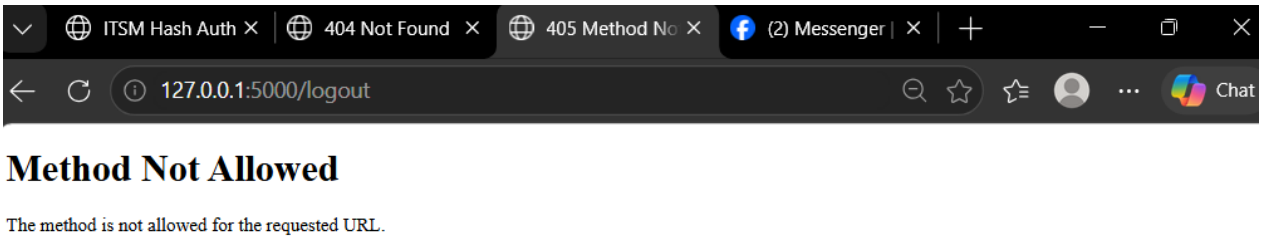


Figure PT19-02. GET /logout returns generic 405.

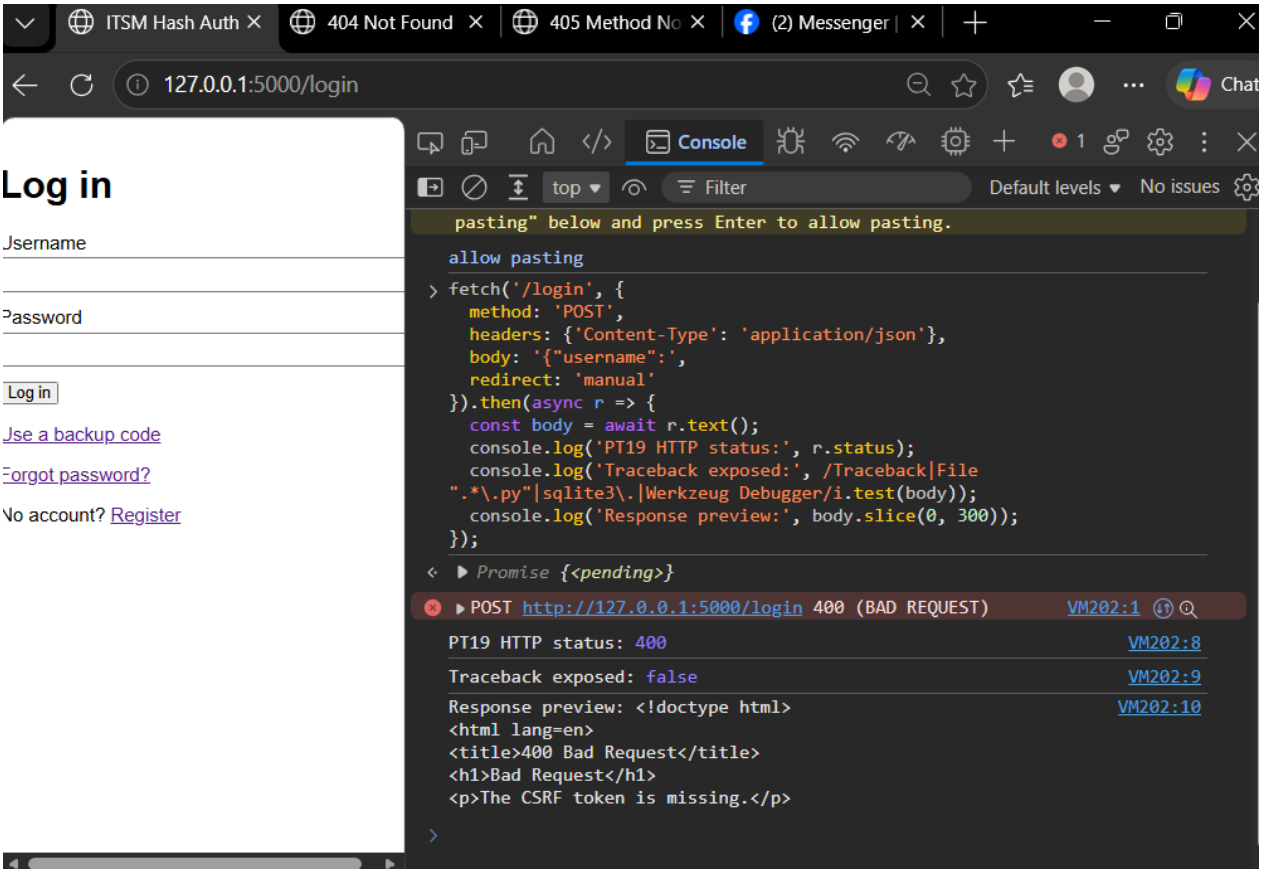


Figure PT19-03. Malformed JSON POST stopped by missing-CSRF check, HTTP 400.

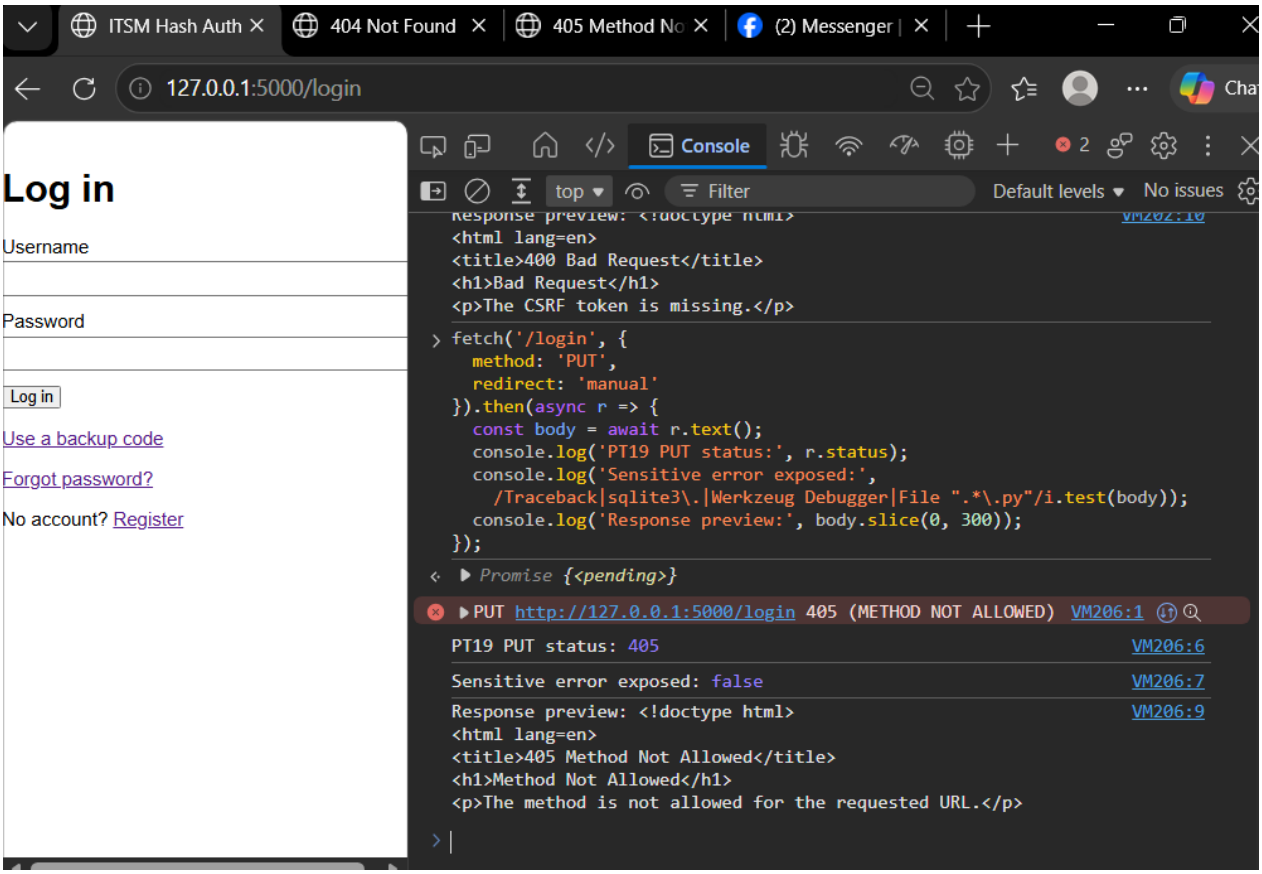


Figure PT19-04. PUT /login returns generic 405 without sensitive error.

PT20: Password storage verification

Objective: Examine database-stored password representations without printing salts or password-derived digests, confirming Argon2id format, encoded parameters, and salt differences.

Procedure: Read-only SQLite PRAGMA inspection of users table; query of stored password_hash fields for format/parameters; comparison of encoded salt segments; verification of encoded hash lengths and component count.

Check	**Observed result**
Schema	users.password_hash TEXT; no separate plaintext password column
Algorithm / version	argon2id; v=19 across ivann-test, victim-test, admin-test
Parameters	m=65536 (64 MiB), t=3, p=4 across the three accounts
Unique salts	Accounts: 3; Unique salts: 3; All salts unique: True
Stored value format	Each encoded string length 97; starts with $argon2id$; salt and digest segments present

Result: PASS for the three accounts. All three values are Argon2id strings, not plaintext, and each has a different salt.

Limitations: this only looked at what is stored, not at every place that writes to the database, and it did not try to crack the hashes. Full hashes are not shown on purpose.

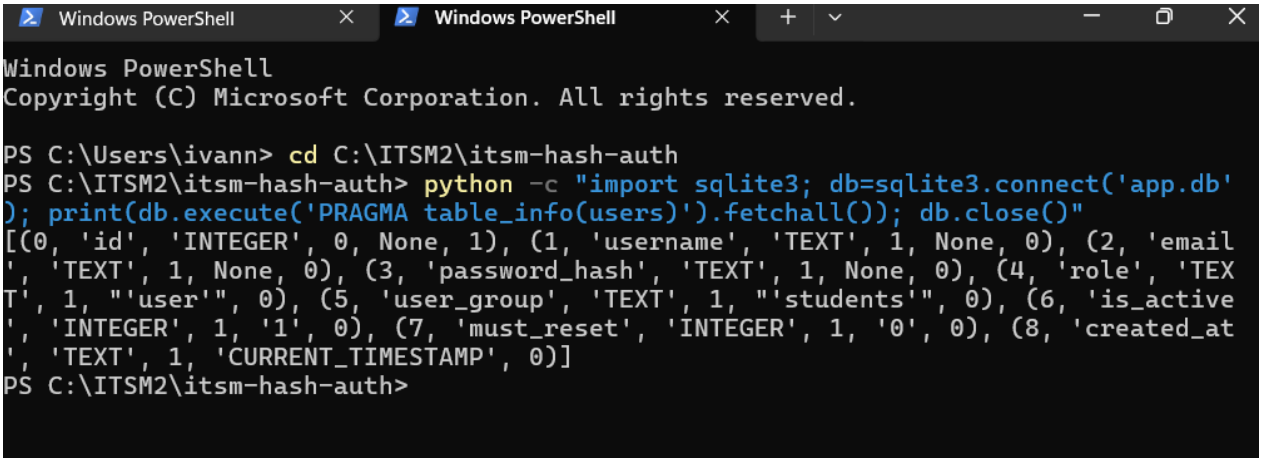


Figure PT20-01. Users table schema contains password_hash field.

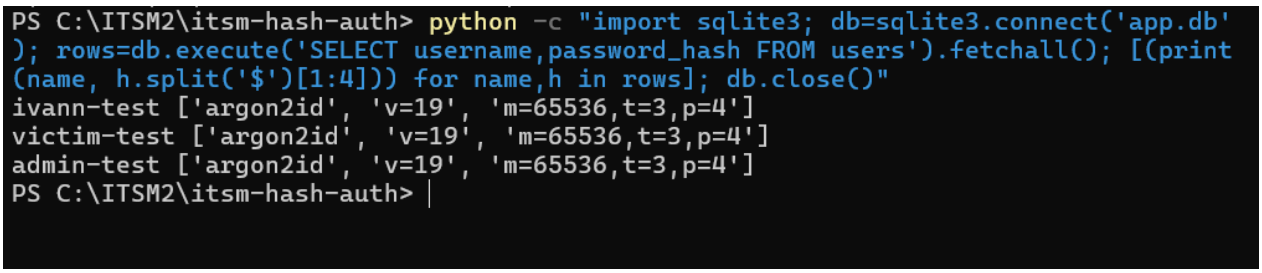


Figure PT20-02. Argon2id version 19 parameters for all three accounts.

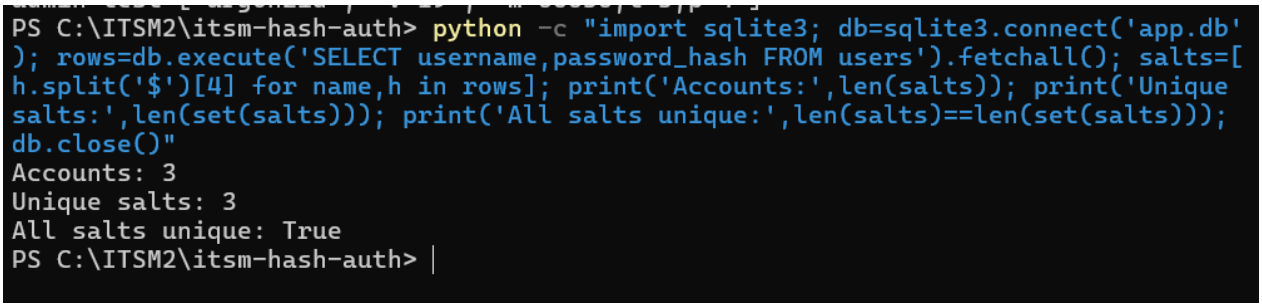


Figure PT20-03. Three distinct encoded salts for three accounts.

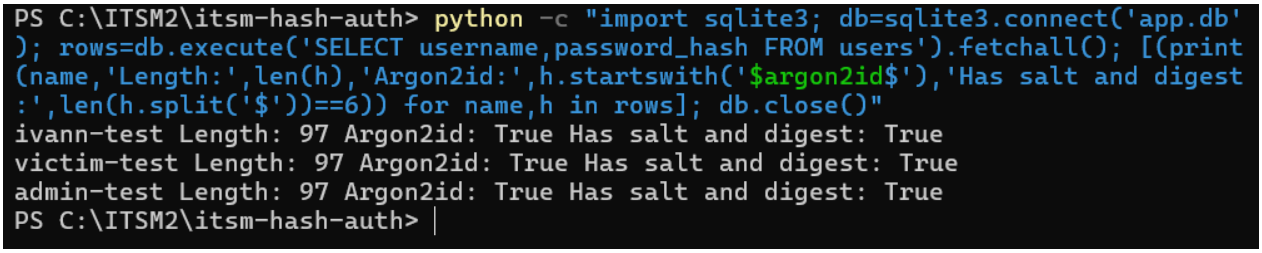


Figure PT20-04. Three encoded 97-character Argon2id hashes with salt and digest.

Overall conclusion

PT16 to PT20 all passed. PT16 and PT17 show that the admin can end sessions for one user or a whole group, PT18 that the session cookie has the right flags, PT19 that error pages leak nothing

sensitive, and PT20 that passwords are stored as salted Argon2id hashes. Still untested: bad input in general, a real TLS setup, and offline cracking of the stored hashes.

Evidence handling and reproducibility

- Evidence identifiers PT16-01 through PT20-04 correspond to labeled screenshots embedded in this report.
- Do not commit raw session cookies, account passwords, password-reset URLs, or the SQLite database to a public repository.
- The administrator account was restored after a VirtualBox snapshot rollback before PT16; PT16–PT20 measurements use the restored local environment.
- All SQL operations for PT20 were read-only; no password hashes or salts were printed in full.